



UxiLibris

Sistema Integral de Gestión y
Personalización de Bibliotecas Personales.

<https://github.com/UxiaRD/UxiLibris>

Uxía Rodríguez Domínguez

2º DAM

Proyecto Final Ciclo

Índice

1. Introducción.....	6
2. Objetivos.....	6
3. Situación previa.....	6
4. Tecnologías empleadas.....	7
Frontend (Capa de Presentación).....	7
Backend (Capa de Negocio).....	7
Base de Datos (Capa de Persistencia).....	7
Herramientas de Desarrollo.....	7
5. Solución propuesta.....	8
6. Planificación del proyecto.....	8
6.1 Planificación del proyecto con un diagrama de Gantt.....	8
6.2 Modelo de desarrollo de software a implementar.....	8
6.3 Análisis del proyecto.....	9
Diagrama de casos de uso.....	9
Universo de discurso de la base de datos.....	10
Diagrama entidad-relación.....	11
6.4 Diseño del proyecto.....	11
Modelo relacional de la base de datos.....	11
Diagrama de clases.....	12
Diagramas de flujo de cada caso de uso.....	13
Mockups de interfaz.....	27
Mockups de interfaz.....	28
6.5 Presupuesto completo.....	29
Mano de Obra (Personal).....	29
Hardware.....	30
Software e Infraestructura.....	30
Resumen Presupuesto Total.....	30
7. Desarrollo y ejecución.....	31
7.1 Diagrama de Despliegue.....	31
7.1.1 Despliegue en la Nube (Producción).....	32
7.1.2 Despliegue Local (Docker Compose).....	33
7.2 Incidencias técnicas resueltas durante el desarrollo.....	34
7.2.1 Backend (Spring Boot).....	34
7.2.2 Base de datos (MySQL).....	36
7.2.3 Frontend (Flutter).....	37
7.2.4 Despliegue.....	40
8. Conclusiones y reflexiones.....	42
8.1 Valoración global.....	42
8.2 Resumen de lo aprendido.....	42
8.3 Dificultades encontradas.....	44
9. Bibliografía y Webgrafía.....	45
10. Anexo I – Manual técnico de instalación o puesta en marcha.....	46
10.1 Introducción.....	46
10.1.1 Propósito del documento.....	46
10.1.2 Alcance.....	46
10.1.3 Descripción general del sistema.....	46

10.1.4 Modalidades de despliegue.....	46
10.2 Arquitectura del sistema.....	47
10.2.1 Diagrama de componentes.....	47
10.2.2 Descripción de los componentes del sistema.....	47
10.2.3 Relaciones entre componentes.....	49
10.3 Requisitos del entorno.....	50
10.4 Modalidades de despliegue.....	50
10.4.1 Opción A — Despliegue en la nube (Render + Neon).....	50
10.4.2 Opción B – Despliegue local con Docker.....	53
10.4.3 Opción C – APK con backend local.....	54
10.4.4 Opción D – Entorno de desarrollo.....	56
10.5 Configuración del backend.....	58
10.5.1 Estructura del fichero application.properties.....	58
10.5.2 Variables de entorno equivalentes para producción.....	58
10.5.3 La propiedad ddl-auto y por qué se usa update.....	59
10.5.4 Obtener y configurar la API key de Google Books.....	60
10.6 Compilación del APK.....	60
10.7 Verificación del sistema.....	62
10.7.1 El backend responde correctamente.....	62
10.7.2 Registro e inicio de sesión.....	63
10.7.3 Añadir y listar libros.....	63
10.7.4 Portadas de Google Books.....	64
10.8 Solución de problemas comunes.....	64
10.8.1 El servidor en Render no responde (arranque en frío).....	64
10.8.2 Las tablas no se crean en la base de datos.....	65
10.8.3 Error de dialecto o conexión con Neon.....	65
10.8.4 La aplicación no encuentra el backend (IP incorrecta).....	66
10.8.5 Render no detecta el puerto ("No open ports detected").....	66
10.8.6 Las portadas de los libros no se muestran.....	66
11. Anexo II – Documentación de uso (manuales de usuario).....	68
11.1 Introducción.....	68
11.1.1 ¿Qué es UxiLibris?.....	68
11.1.2 Plataformas disponibles.....	68
11.1.3 Requisitos del usuario.....	68
11.2 Primeros pasos.....	69
11.2.1 Inicio de sesión.....	69
11.2.2 Registro de cuenta.....	69
11.2.3 Cierre de sesión y cambio de modo de visualización.....	70
11.2.4 Modificación datos de la cuenta (username, email, contraseña).....	70
11.3 La biblioteca.....	71
11.3.1 Vista general (tarjetas de libros).....	71
11.3.2 Filtros y búsqueda.....	72
11.3.3 Estados de un libro (Pendiente, Leyendo, Leído).....	72
11.3.4 Detalle Libro.....	72
11.4 Gestión de libros.....	73
11.4.1 Añadir un libro manualmente.....	73
11.4.2 Añadir un libro por ISBN (escáner).....	74

11.4.3	Buscar un libro en Google Books.....	74
11.4.4	Editar un libro.....	75
11.4.5	Eliminar un libro.....	75
11.4.6	Marcar / desmarcar como favorito.....	76
11.5	Sagas.....	76
11.5.1	Crear una saga.....	77
11.5.2	Asignar un libro a una saga.....	77
11.5.3	Eliminar o abandonar una saga.....	78
11.6	Lista de deseos.....	78
11.6.1	Añadir un libro a la lista.....	79
11.6.2	Mover un libro a la biblioteca.....	79
11.7	Lecturas.....	80
11.7.1	Registrar fechas de una lectura.....	80
11.7.2	Consultar el historial de lecturas de un libro.....	80
11.8	Estadísticas.....	81
11.8.1	Selector de año.....	81
11.8.2	Tarjetas resumen.....	81
11.8.3	Gráfica mensual.....	82
11.8.4	Comparativa por años.....	82
11.8.5	Top autores.....	82
11.8.6	Mejores puntuaciones.....	82
11.8.7	Mis favoritos.....	83
11.9	Navegación.....	83
11.9.1	Menú lateral (drawer).....	83
11.10	Preguntas frecuentes.....	83
12.	Anexo III – Otra documentación.....	85
12.1	Test y Calidad del Código.....	85
12.1.1	Introducción y estrategia de pruebas.....	85
12.1.2	Pruebas del backend (Java / Spring Boot).....	85
12.1.3	Pruebas del frontend (Dart / Flutter).....	88
12.1.4	Integración continua – GitHub Actions.....	91
12.1.5	Cobertura y limitaciones.....	92

Índice de figuras

Figura 1: Diagrama de Gantt.....	8
Figura 2: Diagrama de Casos de Uso.....	9
Figura 3: Diagrama Entidad-Relación.....	11
Figura 4: Modelo Relacional.....	11
Figura 5: Diagrama de clases.....	12
Figura 6: UC-01 Formulario de Autoregistro.....	14
Figura 7: UC-02 Iniciar Sesión.....	15
Figura 8: UC-03 Añadir Libro.....	17
Figura 9: UC-04 Editar Libro.....	19
Figura 10: UC-05 Eliminar Libro.....	20
Figura 11: UC-06 Buscar Libro.....	21
Figura 12: UC-07 Escanear ISBN.....	23
Figura 13: UC-08 Gestionar Saga.....	26
Figura 14: UC-09 Asignar Libro Existente a Saga.....	27
Figura 15: Diagrama de Despliegue.....	31
Figura 16: Diagrama de componentes.....	46

1. Introducción

UxiLibris se define como una aplicación móvil de gestión de bibliotecas personales que rompe con la rigidez de las bases de datos convencionales. Su base principal radica en la Personalización Extrema, permitiendo que cada usuario defina qué metadatos son relevantes para su colección (desde el ISBN o la editorial, hasta el traductor o el formato de edición) mediante un sistema dinámico de propiedades.

2. Objetivos

Este proyecto busca dar solución a la rigidez de las aplicaciones actuales de gestión de bibliotecas personales mediante los siguientes objetivos:

- **Gestión integral de colecciones:** Permitir el registro de detallado de libros, autores y sagas, manteniendo la integridad de los datos.
- **Sistema de Personalización Dinámica (EAV):** Implementar un “Almacén de Propiedades” permite al usuario añadir campos personalizados sin necesidad de modificar el código o la base de datos. El modelo EAV es útil para gestionar datos donde el número de atributos es variable (García, s. f.).
- **Seguimiento de Lectura Avanzado:** Controlar el ciclo de vida de un libro (Pendiente, Leyendo, Leído) mediante registro automático de fechas y valoraciones con precisión decimal.
- **Arquitectura Robusta y Escalable:** Separar claramente las responsabilidades entre una interfaz móvil moderna (Frontend) y un servidor de datos seguro (Backend).

3. Situación previa

Actualmente, los usuarios que desean organizar su biblioteca personal suelen enfrentarse a tres escenarios principales, cada uno con sus limitaciones.

Aplicaciones Comerciales Cerradas

Plataformas populares (como *Goodreads* o *StoryGraph*) ofrecen grandes bases de datos sociales, pero presentan interfaces rígidas. El usuario está limitado a los campos que el desarrollador ha decidido incluir. Si un coleccionista desea registrar datos específicos, como el ilustrador de una edición especial o la ubicación física en su estantería, estas aplicaciones no ofrecen flexibilidad para añadir dichos campos.

Métodos Analógicos o de Ofimática

Muchos lectores recurren a libretas físicas o complejas hojas de cálculo (Excel/Google Sheets). Aunque estos métodos ofrecen total libertad, carecen de las ventajas tecnológicas modernas: no permiten el escaneo de códigos de barras para la entrada rápida de datos, no generan estadísticas visuales automáticas y la sincronización entre dispositivos es inexistente o rudimentaria.

Problemas de Integridad de Datos

En las pocas herramientas que permiten cierta personalización, la entrada de datos suele ser caótica. Es común encontrar duplicados de autores o sagas debido a erratas tipográficas (ej. "*Sanderson*" vs "*Sandersson*"), lo que fragmenta la biblioteca y arruina la precisión de las estadísticas de lectura.

4. Tecnologías empleadas

Frontend (Capa de Presentación)

Tecnología: **Flutter (Dart)** (Google, s. f.)

Razón: Permite un desarrollo ágil multiplataforma con una interfaz nativa y reactiva. Su gestión de estados es ideal para formularios dinámicos.

Librerías clave: *http* (comunicación con API), *image_picker* (gestión de portadas), *mobile_scanner* (lectura de códigos de barras), *fl_chart* (visualización de gráficas de estadísticas), *flutter_secure_storage* (almacenamiento cifrado de credenciales de sesión en el keychain/keystore del dispositivo) y *provider* (gestión reactiva del estado del tema claro/oscuro).

Backend (Capa de Negocio)

Tecnología: **Java Spring Boot 3** (Spring Team, s. f.)

Razón: Es el estándar en de la industria para aplicaciones empresariales. Ofrece seguridad, facilidad para crear APIs REST y una integración excelente con bases de datos relacionales.

Herramientas: Spring Data JPA (para el mapeo de objetos a tablas) y Lombok (para la limpieza del código).

Base de Datos (Capa de Persistencia)

Tecnología: **MySQL**

Razón: Motor relacional robusto que permite gestionar las relaciones complejas entre libros, sagas y las tablas dinámicas de propiedades mediante claves foráneas y restricciones de integridad.

Herramientas de Desarrollo

- **IDE Frontend:** VS Code / Android Studio
- **IDE Backend:** IntelliJ IDEA (Optimizado para Spring Boot)
- **Control de Versiones:** Git (Estructura de Monorepo)
- **Integración continua:** GitHub Actions — pipeline automático que ejecuta los tests de backend y frontend en cada push.

- **Testing backend:** JUnit 5 (framework de pruebas), Mockito (mocks), H2 (base de datos en memoria para tests), AssertJ (aserciones).
- **Testing frontend:** flutter_test (SDK de Flutter, incluido en el entorno).

5. Solución propuesta

UxiLibris se presenta como una solución que combina la potencia de una base de datos relacional profesional con la flexibilidad de una hoja de cálculo, todo ello bajo una interfaz móvil diseñada con Flutter y un motor de persistencia robusto en Java Spring Boot. La aplicación no solo soluciona la falta de personalización, sino que introduce mecanismos de control de errores (sugerencias inteligentes y validaciones en tiempo real) para garantizar que la biblioteca del usuario sea una fuente de datos fiable y organizada.

6. Planificación del proyecto

6.1 Planificación del proyecto con un diagrama de Gantt

El siguiente diagrama de Gantt representa las semanas de esfuerzo de jornada completa equivalentes aplicadas al proyecto.

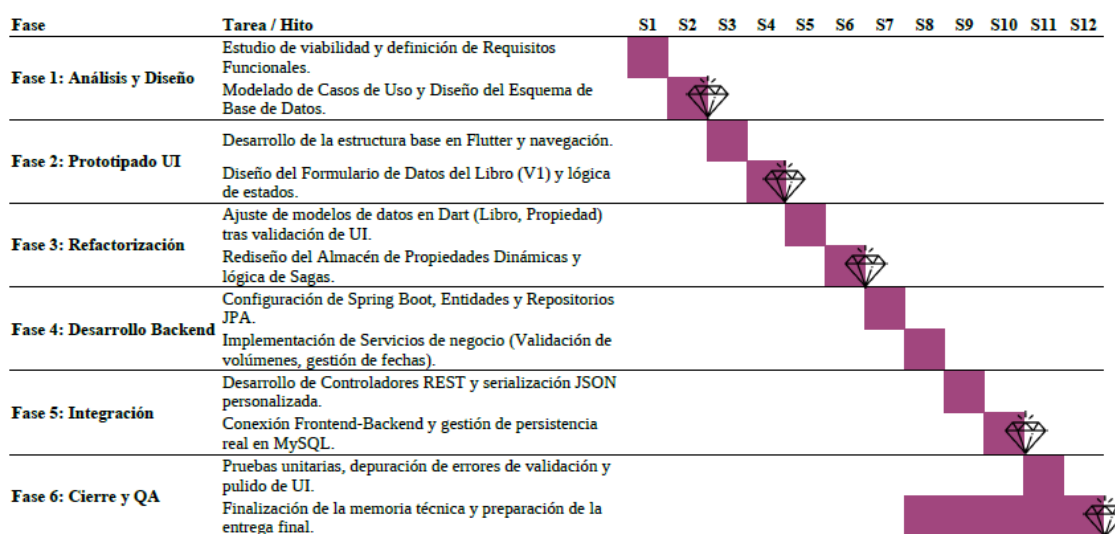


Figura 1: Diagrama de Gantt



Hitos del proyecto → indicación de finalización de etapas clave

6.2 Modelo de desarrollo de software a implementar

El proyecto se ha desarrollado siguiendo una metodología **iterativa e incremental**. A diferencia de los modelos tradicionales, este enfoque ha permitido validar la interfaz de usuario (Frontend) en fases tempranas (enero), lo que facilitó el descubrimiento de nuevas necesidades de datos.

Estas iteraciones permitieron que, al llegar a la fase de construcción del Backend en marzo, los modelos de persistencia y la lógica de negocio estuvieran perfectamente alineados con las necesidades reales de la aplicación (como el sistema de propiedades dinámicas y la validación de sagas).

6.3 Análisis del proyecto

Diagrama de casos de uso

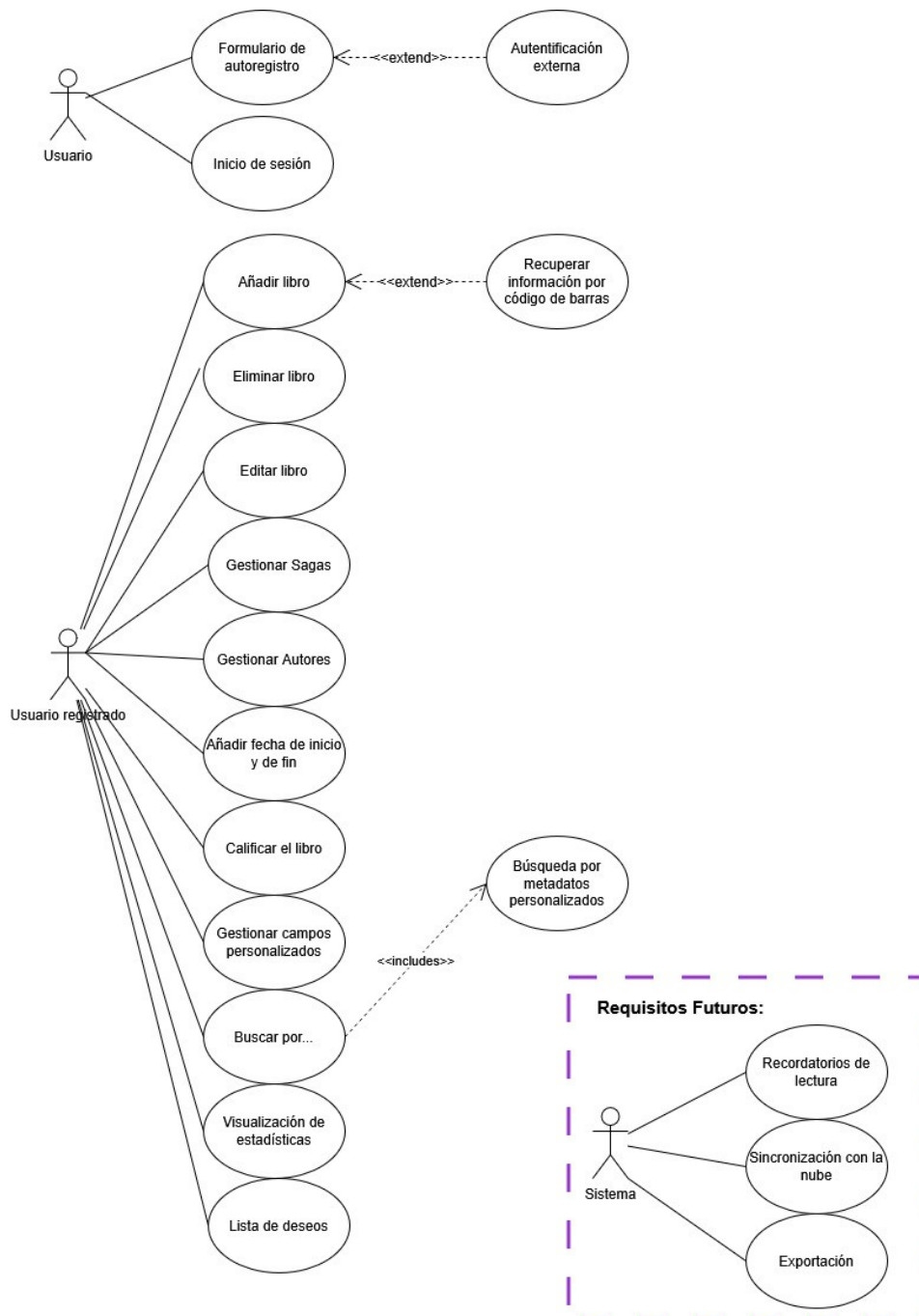


Figura 2: Diagrama de Casos de Uso

El diagrama de Casos de Uso de UxiLibris estructura las funcionalidades del sistema según los perfiles de **Usuario** (visitante), **Usuario Registrado** y el **Sistema** como actor secundario para procesos automatizados.

El sistema permite el acceso mediante un proceso de **Inicio de sesión** y un **Registro** que incluye obligatoriamente un formulario de autorregistro o el uso de servicios de autenticación externa. Estas relaciones se marcan como «*include*» al ser componentes necesarios para completar la acción principal.

El corazón de la aplicación reside en el **Formulario de datos del libro**, un componente reutilizable invocado tanto para **Añadir** como para **Editar** registros. En el cual se han modelado como extensiones («*extend*») aquellas funcionalidades condicionales que dependen de la lógica de negocio o de la voluntad del usuario:

- **Gestionar Sagas y Autores:** Solo se activan si el elemento introducido no existe previamente en la base de datos.
- **Fechas de lectura y Calificación:** Funciones vinculadas al estado del libro (En Proceso o Leído).
- **Gestionar campos personalizados:** Representa el motor de "Personalización Extrema", permitiendo al usuario ampliar el esquema de datos del libro a demanda.

El proceso de **Añadir** un libro extiende a la **Validación del número de entrega** para garantizar la integridad de las sagas (evitando duplicados y sugiriendo el siguiente volumen).

El usuario puede realizar **Búsquedas** que incluyen obligatoriamente el uso de metadatos personalizados, garantizando que el "Almacén de Propiedades" sea totalmente indexable. Además, se proporcionan módulos para la **Visualización de estadísticas** y la gestión de una **Lista de deseos**.

Se ha delimitado un bloque de **Requisitos Futuros** gestionados por el **Sistema**, que incluye la exportación de datos, sincronización en la nube y recordatorios automáticos, asegurando la escalabilidad del proyecto en fases posteriores.

Universo de discurso de la base de datos

El universo de discurso de la base de datos **UxiLibris** comprende la gestión integral de un catálogo bibliográfico digital. El sistema permite a los **Usuarios** autenticados organizar su **colección de Libros**.

Cada Libro se caracteriza por atributos como título, saga y una imagen de portada. Existe una relación de autoría, donde un **Autor** puede haber escrito múltiples obras presentes en el sistema.

Las restricciones principales establecen que el acceso a la modificación de datos está limitado a usuarios registrados, y que la integridad de la información depende de la unicidad de los identificadores de libros y autores. El sistema debe ser capaz de manejar datos binarios (imágenes) transformados a formato de texto para su persistencia con un sistema de codificación Base64.

Diagrama entidad-relación

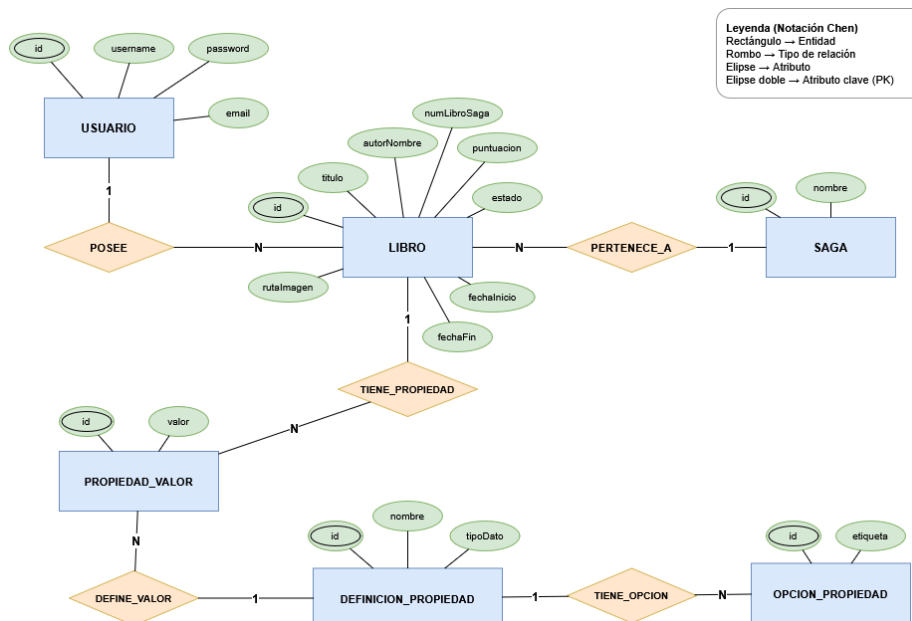


Figura 3: Diagrama Entidad-Relación

6.4 Diseño del proyecto

Modelo relacional de la base de datos

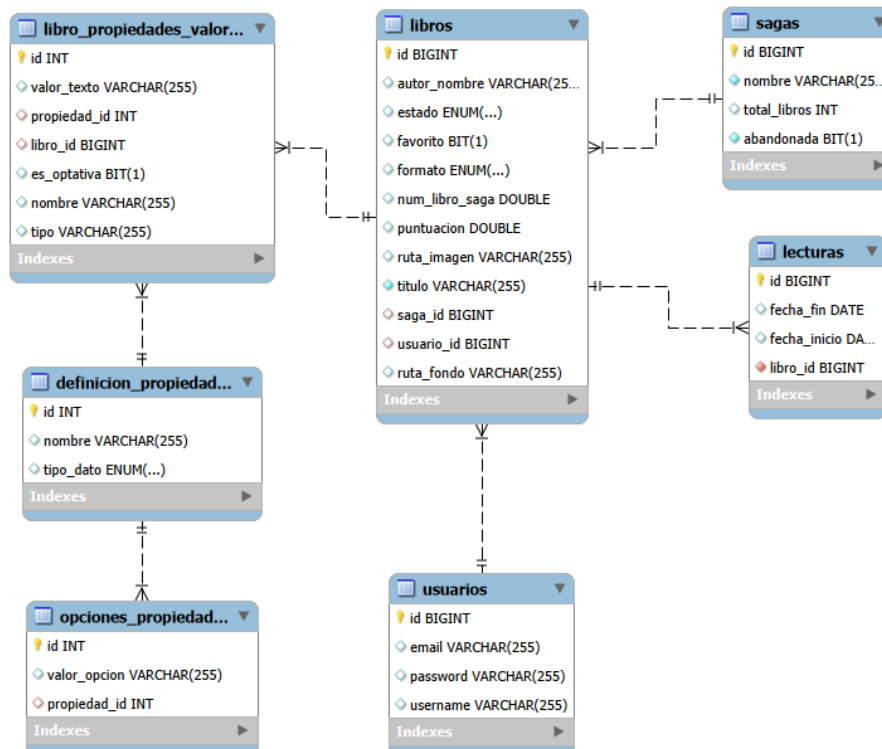


Figura 4: Modelo Relacional

The diagram illustrates the architecture of the Uxilibris application, organized into several packages and components:

- «package» com.uxia.uxilibris.model**: Contains enumerations for book status (Pendiente, Leído, Deseo), format (Físico, Digital), and type (Texto, Numérico, Fecha, Booleano).
- «Entity» @Table(libros) Libro**: Represents the book entity with attributes like id, title, author, price, status, format, favorite, image, and user.
- «Entity» @Table(usuarios) Usuario**: Represents the user entity with attributes like id, username, email, and password.
- «Entity» @Table(lecturas_libro) LecturaLibro**: Represents the book reading entity with attributes like id, book, start date, end date, and book.
- «Entity» @Table(sagas) Saga**: Represents the saga entity with attributes like id, name, total books, and abandoned status.
- «Entity» @Table(libro_propiedades_valores) PropiedadValor**: Represents the book property value entity with attributes like id, value, and definition.
- «Entity» @Table(definicion_propiedades) DefinicionPropiedad**: Represents the definition property entity with attributes like id, name, type, and definition.
- «Entity» @Table(opciones_propiedades) OpcionPropiedad**: Represents the option property entity with attributes like id, name, and definition.
- «interface» LibroRepository**: Defines the interface for the book repository, extending JpaRepository.
- «interface» UsuarioRepository**: Defines the interface for the user repository, extending JpaRepository.
- «interface» SagaRepository**: Defines the interface for the saga repository, extending JpaRepository.
- «DTO» LoginRequest**: DTO for login request with username and password.
- «DTO» LoginResponse**: DTO for login response with user ID, username, and email.
- «DTO» ActualizarUsuarioRequest**: DTO for user update request with username, email, password, and new password.
- «DTO» isbnResultDto**: DTO for ISBN result with title, author, and image.
- «Service» LibroService**: Implements the book repository interface, providing methods for saving, deleting, and finding books.
- «Service» UsuarioService**: Implements the user repository interface, providing methods for saving, deleting, and finding users.
- «Service» SagaService**: Implements the saga repository interface, providing methods for saving, deleting, and finding sagas.
- «Service» GoogleBooksService**: Implements the Google Books API service, providing methods for searching books by ISBN and text.
- «Test» @ExtendWith(MockitoExtension) UsuarioServiceTest**: Test class for the user service, using Mockito for mocking.
- «Test» @DataJpaTest [H2 in memory] UsuarioRepositoryTest**: Test class for the user repository, using H2 in-memory database.

12 / 92

Diagramas de flujo de cada caso de uso

UC-01	Registro de Usuario		
Actores	Usuario no registrado		
Pre condiciones	La app está abierta en la pantalla de Login. El backend Spring Boot está operativo.		
Post condiciones	El usuario queda registrado en la tabla «usuarios» y es redirigido a PantallaLogin.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa "¿Aún no estás registrado? ¡Haz clic aquí!".	El sistema navega a PantallaRegistro.
	2	El usuario rellena: username, email, password, confirmPassword.	-
	3	El usuario pulsa "Registrarse".	RegistroController valida localmente: a. email tiene formato válido. b. password contiene al menos una mayúscula. c. password y confirmPassword coinciden.
	4	-	ApiService envía POST /api/auth/registro {username, email, password}.
	5	-	UsuarioService comprueba unicidad de username y email.
	6	-	BCryptPasswordEncoder codifica la contraseña antes de persistirla.
	7	-	Se crea el registro en la base de datos (password almacenado como hash BCrypt).
	8	-	El backend devuelve true.
	9	-	El sistema muestra SnackBar verde y navega a PantallaLogin.
Excepciones	#	Acción (actor)	Reacción (sistema)
	3a	Validación local falla	SnackBar rojo con el mensaje de error. El formulario permanece abierto. Fin del caso de uso alternativo.
	5a	Username o email ya existen	backend devuelve false → SnackBar "Usuario o email ya existen en el sistema". El formulario permanece abierto. Fin del caso de uso alternativo.
Frecuencia	Baja (una vez por usuario)		
Importancia	Alta		

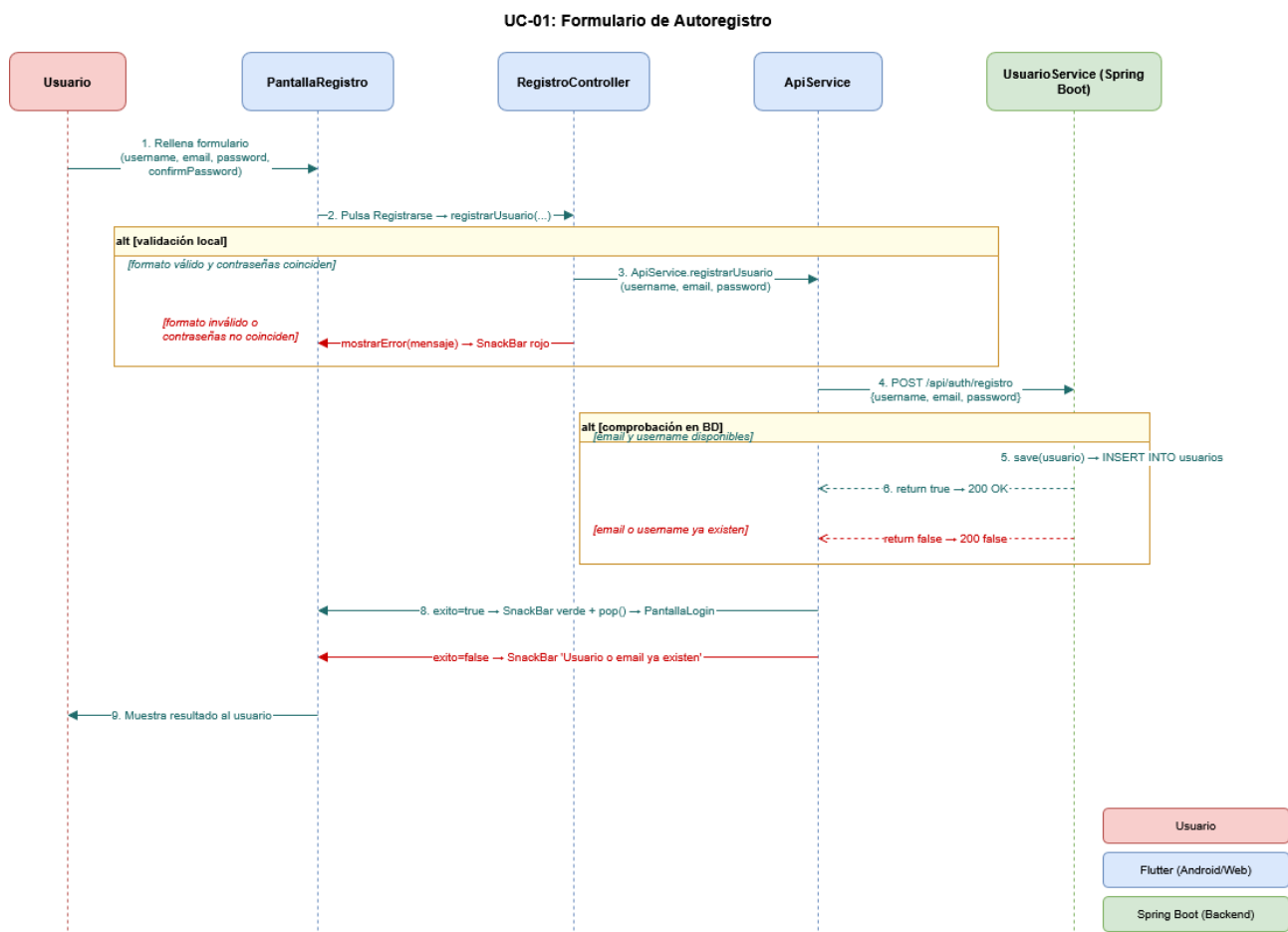


Figura 6: UC-01 Formulario de Autoregistro

UC-02	Inicio de Sesión		
Actores	Usuario registrado		
Pre condiciones	La app está en PantallaLogin. El backend está operativo.		
Post condiciones	El usuario accede a MenuPrincipal. SessionManager almacena usuarioId y username. Libreria.todosLosLibros está cargada.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario introduce username y password.	-
	2	El usuario pulsa "Iniciar sesión".	LoginController llama a ApiService.login(username, password).
	3	-	ApiService envía POST /api/auth/login {username, password}.

	4	-	UsuarioService busca el usuario por username.
	5	-	BCryptPasswordEncoder.matches(password, hash) verifica la contraseña.
	6	-	El backend devuelve LoginResponse {userId, username}.
	7	-	ApiService llama a SessionManager.iniciar(userId, username).
	8	-	La app navega a MenuPrincipal (pushReplacement).
Excepciones	#	Acción (actor)	Reacción (sistema)
	4a	Usuario no existe o BCrypt no coincide	Optional.empty() → 401 Unauthorized → false → Snackbar "Email o contraseña incorrectos". El usuario permanece en PantallaLogin.
Frecuencia	Alta		
Importancia	Alta		
Comentarios	La contraseña nunca se devuelve al cliente. SessionManager es una clase estática que persiste userId durante toda la sesión de la app.		

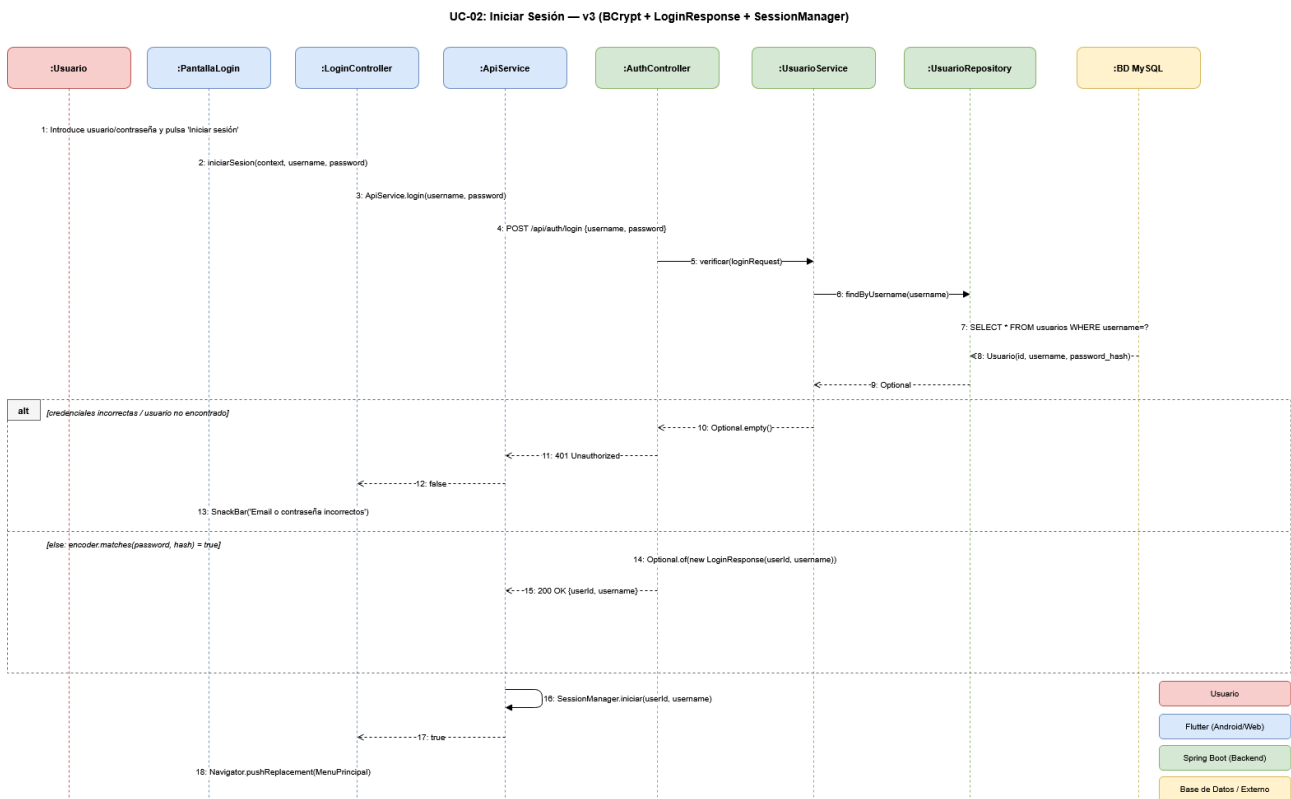


Figura 7: UC-02 Iniciar Sesión

UC-03	Añadir Libro (Manual)		
Actores	Usuario autenticado		
Pre condiciones	El usuario está en MenuPrincipal / PantallaColeccion. El usuario tiene sesión activa (SessionManager.usuarioId != null). El backend está operativo.		
Post condiciones	El libro queda en la tabla «libros» con usuario_id apuntando al usuario que lo creó. Si tiene saga, la saga existe en «sagas». El GridView de la colección muestra el nuevo libro.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa el FAB (+).	El sistema navega a SeleccionMetodoCarga.
	2	El usuario elige "Añadir Manualmente".	El sistema navega a PantallaGestionLibro (modo creación).
	3		FormularioLibro.initState carga sugerencias de autores y sagas mediante GET /api/libros/autores y GET /api/libros/sagas.
	4	El usuario rellena: título, autor, saga, volumen, estado, puntuación, fechas e imagen (opcional).	Si el usuario selecciona una saga del autocomplete, el sistema sugiere el siguiente volumen disponible (GET /api/libros/sugerir-volumen).
	5	El usuario pulsa "Registrar Libro".	FormularioLibroController valida que el título no esté vacío.
	6	-	FormularioLibroController asigna libro.usuarioId = SessionManager.usuarioId.
	7	-	Si sagaNombre está vacío, numLibroSaga se fuerza a null.
	8	-	ApiService envía POST /api/libros {libro sin id, con usuarioId}.
	9	-	LibroService.guardarLibro: a. resolverUsuario(usuarioId) → findById → carga la entidad Usuario. b. libro.setUsuario(usuario) → establece la FK @ManyToOne. c. Llama a autoCrearSagaSiNecesario → INSERT INTO sagas si no existe. d. Resuelve la entidad Saga por nombre (FK). e. INSERT INTO libros (con usuario_id).
	10	-	El backend devuelve el Libro con id asignado.
	11	-	FormularioLibro muestra Snackbar verde y llama Navigator.pop(true).
	12	-	PantallaColeccion refresca el GridView.

Excepciones	#	Acción (actor)	Reacción (sistema)
	5a	Título vacío	FormatException → SnackBar "El título es obligatorio". No se realiza ninguna llamada al backend.
	8a	Error de red	SnackBar "Error de conexión".
Frecuencia	Alta		
Importancia	Alta		

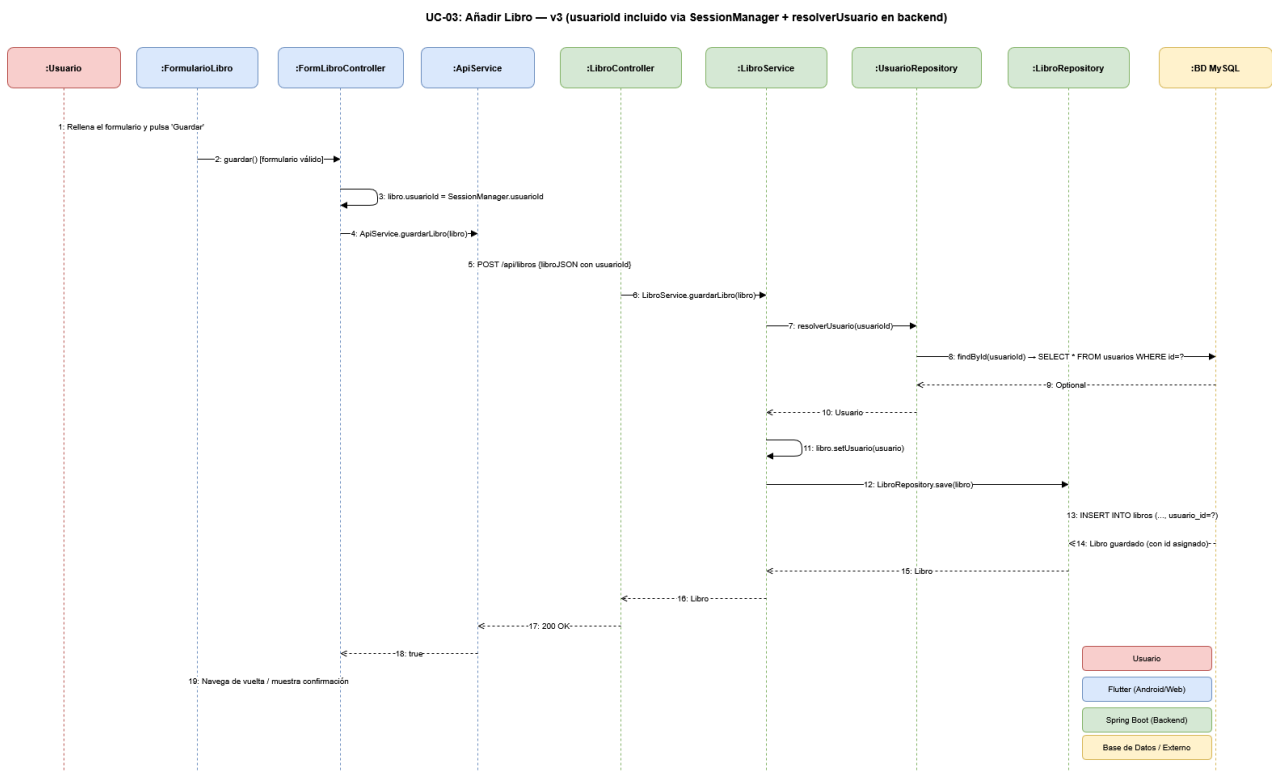


Figura 8: UC-03 Añadir Libro

UC-04	Editar Libro		
Actores	Usuario autenticado		
Pre condiciones	El libro existe en la base de datos y en Libreria.todosLosLibros. El usuario tiene sesión activa (SessionManager.usuarioId != null).		
Post condiciones	El libro queda actualizado en la base de datos y el GridView refleja los cambios.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario toca la card de un libro en el GridView.	GridColeccion navega a PantallaGestionLibro(libroExistente).

	2	-	FormularioLibro pre-rellena todos los campos con los datos actuales.
	3	El usuario modifica los campos deseados.	-
	4	El usuario pulsa "Actualizar Libro".	FormularioLibroController asigna libro.usuarioId = SessionManager.usuarioId.
	5	-	ApiService envía POST /api/libros {libro con id y usuarioId}.
	6	-	LibroService.guardarLibro (rama id != null): a. resolverUsuario(usuarioId) → findById → carga la entidad Usuario. b. libro.setUsuario(usuario) → mantiene la FK @ManyToOne. c. autoCrearSagaSiNecesario si cambia la saga. d. findById(id) → carga la entidad gestionada por Hibernate. e. Actualiza campos uno a uno (evita problema orphanRemoval). f. libroRepository.save(existente) → UPDATE libros SET ...
	7	-	El backend devuelve el Libro actualizado.
	8	-	FormularioLibro muestra SnackBar y llama Navigator.pop(true).
	9	-	PantallaColeccion refresca el GridView.
Excepciones	#	Acción (actor)	Reacción (sistema)
	6a	id no encontrado	orElseGet → INSERT (creación implícita).
	5a	Error de red	SnackBar "Error de conexión".
Frecuencia	Media		
Importancia	Alta		
Comentarios	El doble pop fue corregido en v3. Solo FormularioLibro llama a Navigator.pop. PantallaGestionLibro únicamente ejecuta libreria.editarLibro() en el callback alGuardar.		

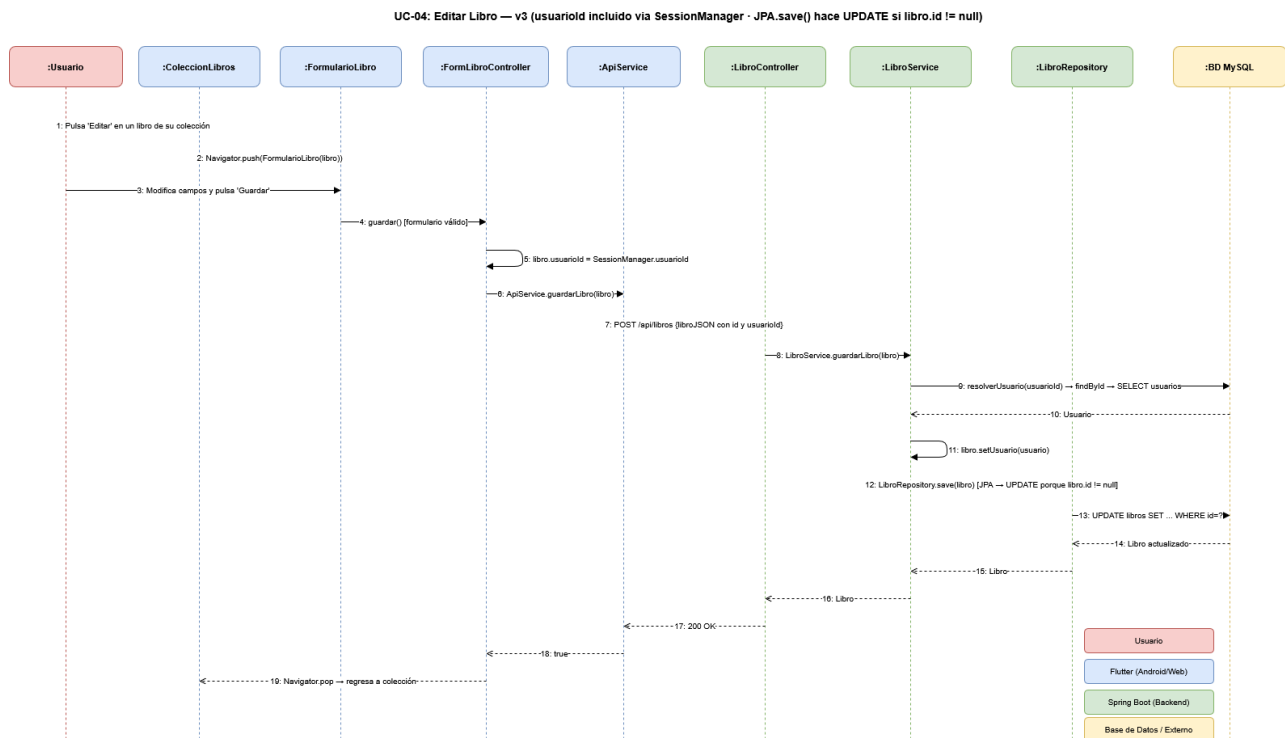


Figura 9: UC-04 Editar Libro

UC-05	Eliminar Libro		
Actores	Usuario autenticado		
Pre condiciones	El usuario está en PantallaGestionLibro (modo edición).		
Post condiciones	El libro es eliminado de la base de datos y de Libreria.todosLosLibros. El GridView lo omite.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa el icono de eliminación en el AppBar.	El sistema muestra AlertDialog "¿Eliminar libro?".
	2	El usuario confirma.	GestionLibroController.eliminarLibro llama a ApiService → DELETE /api/libros/{id}.
	3	-	LibroService.eliminarLibro: existsById → deleteById → CASCADE elimina las propiedades dinámicas del libro.
	4	-	El backend devuelve 204 No Content.
	5	-	libreria.eliminarLibro(libro) elimina el objeto de la lista local.
	6	-	El sistema navega de vuelta a PantallaColeccion.

Excepciones	#	Acción (actor)	Reacción (sistema)
	2a	El usuario cancela	el AlertDialog se cierra sin ninguna acción.
	3a	id no encontrado	EntityNotFoundException → SnackBar de error.
Frecuencia	Baja		
Importancia	Media		

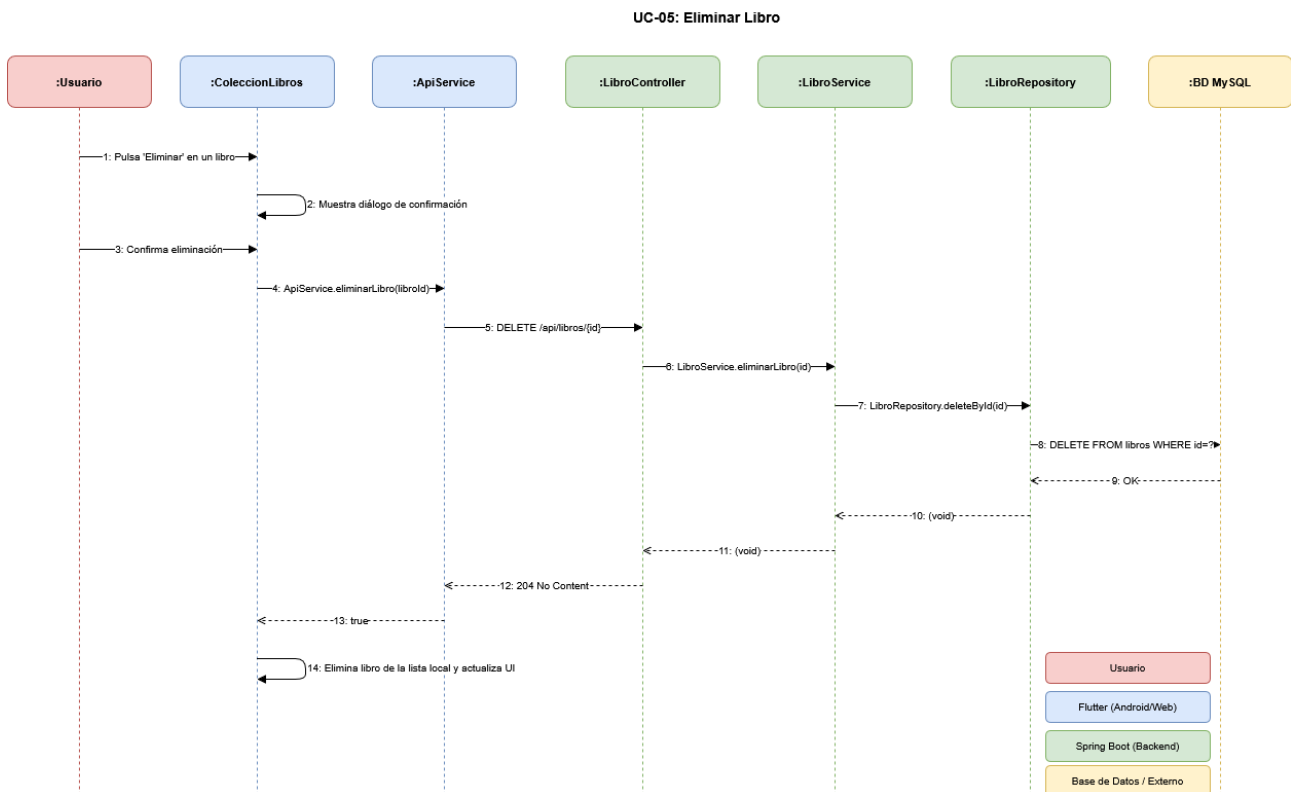


Figura 10: UC-05 Eliminar Libro

UC-06	Buscar / Filtrar Libro		
Actores	Usuario autenticado		
Pre condiciones	Al menos un libro está cargado en Libreria.todosLosLibros.		
Post condiciones	El GridView muestra solo los libros que cumplen el filtro de pestaña y el texto de búsqueda.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa una pestaña del BottomNavigationBar (Todos / Leyendo / Pendientes / Leídos).	MenuPrincipal actualiza el filtro; PantallaColeccion.didUpdateWidget dispara _buscarLibros("").

	2	-	Libreria.filtrarPorEstado filtra todosLosLibros por estado.
	3	-	El GridView se reconstruye con la lista filtrada.
	4	El usuario escribe en la BarraBusqueda.	onChanged dispara _buscarLibros(consulta).
	5	-	Libreria.filtrarPorBusqueda aplica búsqueda por título, autorNombre y sagaNombre (case-insensitive) sobre la lista ya filtrada por estado.
	6		El GridView se actualiza en tiempo real.
Excepciones	#	Acción (actor)	Reacción (sistema)
	-	No aplica (la búsqueda es local, no hay llamadas al backend).	-
Frecuencia	Muy Alta		
Importancia	Alta		

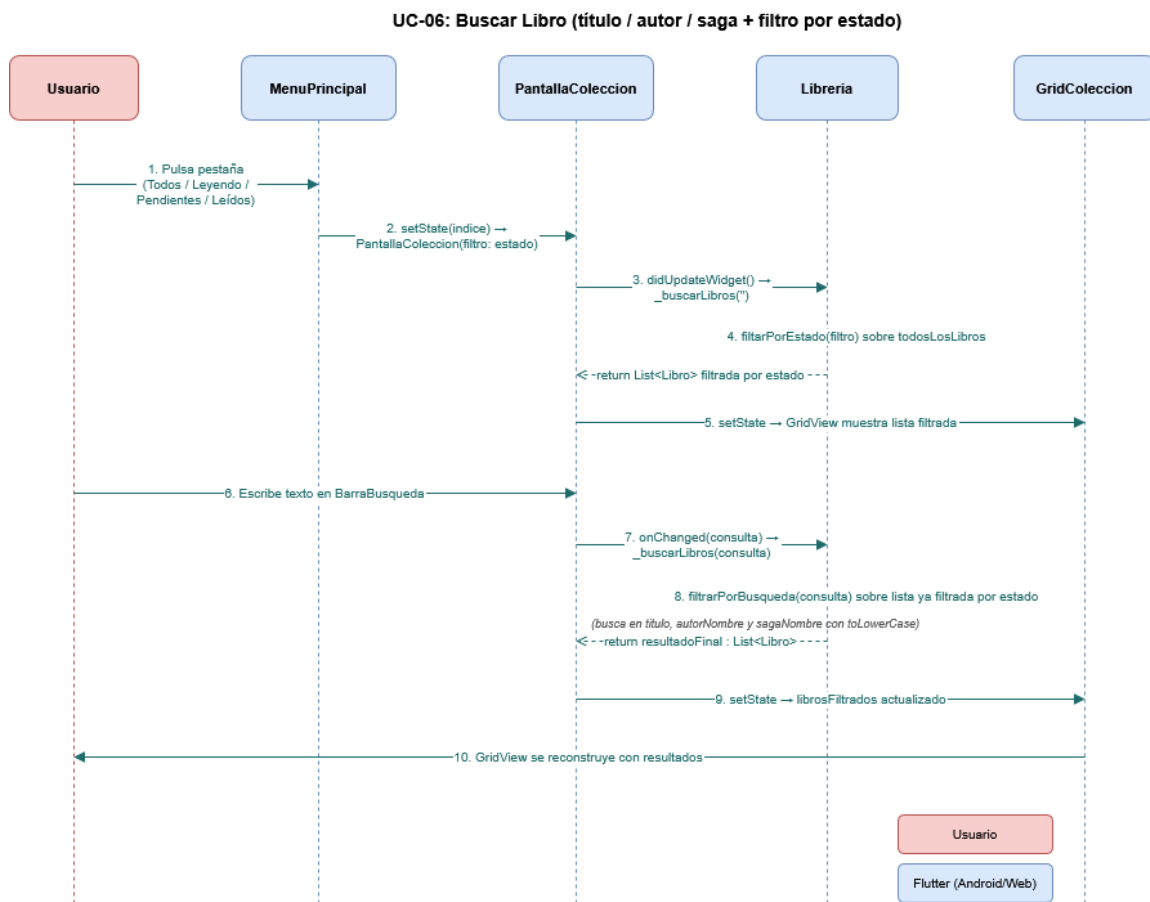


Figura 11: UC-06 Buscar Libro

UC-07	Escanear ISBN		
Actores	Usuario autenticado		
Pre condiciones	El dispositivo tiene cámara disponible. El backend está operativo con acceso a internet para consultar Google Books API. La API key de Google Books está configurada en application.properties (cuota: 1000 req/día).		
Post condiciones	PantallaGestionLibro se abre con los campos pre-rellenados (título, autor, portada, saga, volumen).		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa FAB (+) → SeleccionMetodoCarga → "Escanear ISBN".	El sistema abre PantallaEscanerISBN con la cámara activa.
	2	El usuario apunta la cámara al código de barras del libro.	MobileScanner detecta el código y llama _procesarBarcode(isbn).
	3	-	EscanerController valida que el ISBN tenga 10 o 13 dígitos.
	4	-	ApiService envía GET /api/libros/isbn/{isbn}.
	5	-	LibroController → GoogleBooksService consulta la Google Books API (con API key si está configurada).
	6	-	GoogleBooksService extrae del JSON: • volumeInfo.title → título • volumeInfo.authors[0] → autor • imageLinks.thumbnail → portada (URL http) • seriesInfo.shortSeriesBookTitle → saga • seriesInfo.bookDisplayNumber → numLibroSaga
	7	-	El backend devuelve IsbnResultDto → 200 OK.
	8	-	ApiService construye un objeto Libro (sin id) con los datos.
	9	-	El sistema compara el título con Libreria.todosLosLibros: - Si ya existe: AlertDialog "¿Añadir igualmente?".
	10	El usuario confirma (o el título no existía).	Navigator.pushReplacement → PantallaGestionLibro(libroPrerellenado).
	11	El usuario revisa y completa los campos → flujo UC-03 desde paso 4.	-
Excepciones	#	Acción (actor)	Reacción (sistema)
	3a	ISBN con formato incorrecto (≠ 10 ni 13 dígitos)	FormatException → clearSnackBars() + Snackbar rojo → _reiniciarEscaner() (espera 3 s, reactiva cámara).
	5a	Google Books API devuelve	GoogleBooksService lanza

		429 Too Many Requests (cuota agotada)	ResponseStatusException(429) → ApiService._errorISBN(429) → Exception → clearSnackBars() + SnackBar "Cuota de Google Books agotada por hoy. Espera unas horas o configura una API key propia en el backend." → _reiniciarEscaner() (espera 3 s).
	5b	Google Books API devuelve 503 / no disponible	ResponseStatusException(503) → SnackBar "Google Books no está disponible. Inténtalo más tarde." → _reiniciarEscaner() (espera 3 s).
	7a	ISBN no encontrado en Google Books (404)	SnackBar "No se encontró información" con acción "Añadir manualmente" → PantallaGestionLibro vacío.
	9a	Duplicado detectado y usuario cancela	la cámara se reactiva para escanear otro código.
Frecuencia	Alta		
Importancia	Alta		
Comentarios	DetectionSpeed.noDuplicates solo previene re-detección dentro del mismo evento de escaneo. El delay de 3 s en _reiniciarEscaner evita que el mismo código se reescanee inmediatamente tras rearrancar _controller.start(). clearSnackBars() elimina SnackBar anteriores antes de mostrar uno nuevo, evitando que se apilen.		

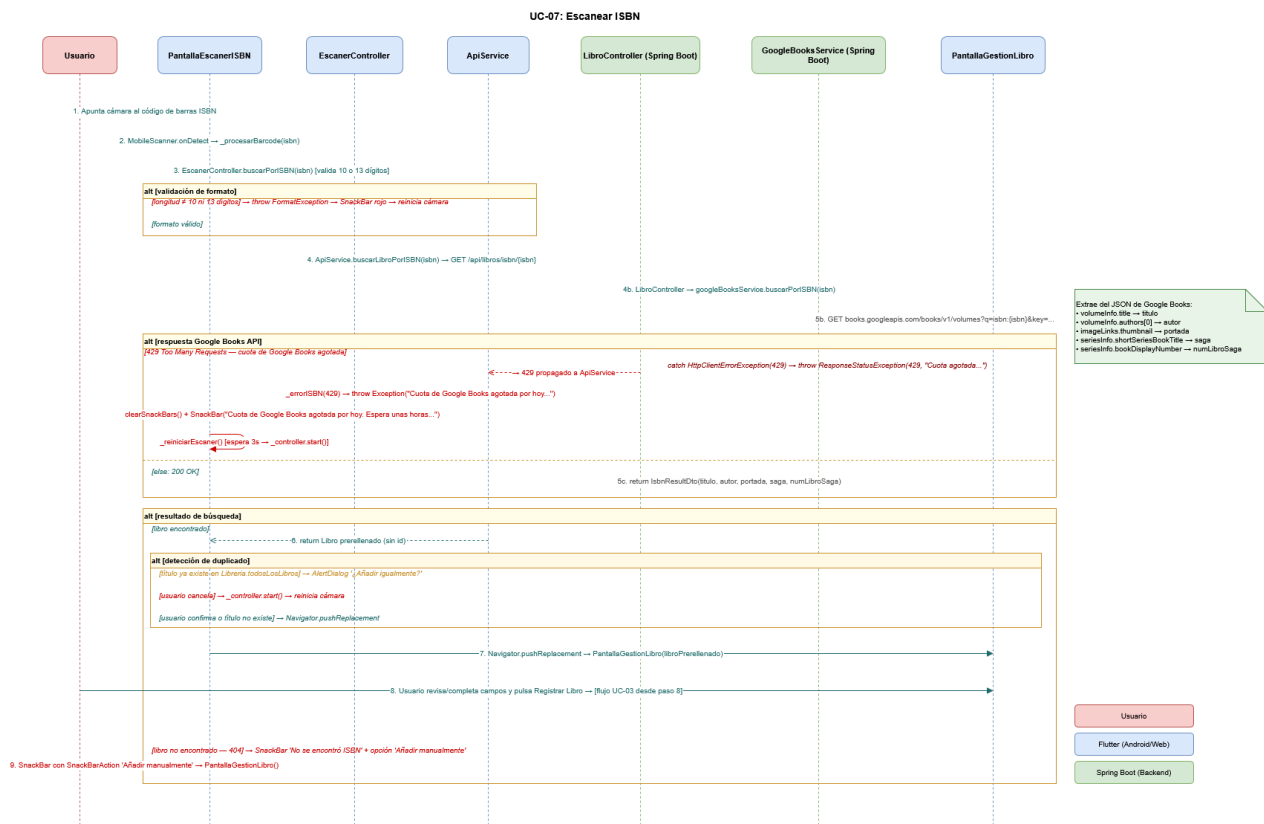


Figura 12: UC-07 Escanear ISBN

UC-08	Gestionar Saga (Crear / Editar / Eliminar)
Actores	Usuario autenticado
Pre condiciones	El usuario está en PantallaColeccionSagas o PantallaDetalleSaga. El backend está operativo.

UC-08-A	Crear Saga		
Post condiciones	La saga queda en la tabla «sagas». La lista de sagas se actualiza.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa FAB (+) en PantallaColeccionSagas.	El sistema navega a PantallaGestionSaga (modo creación).
	2	El usuario rellena nombre (obligatorio) y totalLibros (opcional).	-
	3	El usuario pulsa "Guardar".	ApiService envía POST /api/sagas {nombre, totalLibros}.
	4	-	SagaService.guardar → trim(nombre) → INSERT INTO sagas.
	5	-	El backend devuelve la Saga con id.
	6	-	Navigator.pop(true) → PantallaColeccionSagas llama _cargar().
	7	-	La lista de sagas se actualiza con la nueva saga.
Excepciones	#	Acción (actor)	Reacción (sistema)
	2a	Nombre vacío	validación local → el formulario no se envía.

UC-08-B	Editar Saga		
Post condiciones	Los datos de la saga quedan actualizados.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa el icono de edición en PantallaDetalleSaga.	El sistema navega a PantallaGestionSaga(saga: sagaExistente).
	2	-	El formulario se pre-rellena con los datos actuales.
	3	El usuario modifica campos y pulsa "Guardar".	ApiService envía POST /api/sagas {id, nombre, totalLibros}.
	4	-	SagaService.guardar → save() → UPDATE sagas SET ...

	5	-	Navigator.pop(true) × 2 → PantallaColeccionSagas → _cargar().
--	---	---	---

UC-08-C	Eliminar Saga		
Post condiciones	La saga es eliminada de «sagas». Los libros asociados quedan con saga_id = NULL (NO se eliminan los libros).		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa el icono de eliminar en PantallaGestionSaga.	AlertDialog confirma la acción.
	2	El usuario confirma.	ApiService envía DELETE /api/sagas/{id}.
	3	-	SagaService.eliminar: a. libroRepository.clearSagaById(id) → UPDATE libros SET saga_id = NULL WHERE saga_id = id. b. sagaRepository.deleteById(id) → DELETE FROM sagas.
	4	-	204 No Content → Navigator.pop(true) × 2 → _cargar().
Excepciones	#	Acción (actor)	Reacción (sistema)
	2a	El usuario cancela	el AlertDialog se cierra sin acción.
Frecuencia	Media		
Importancia	Alta		

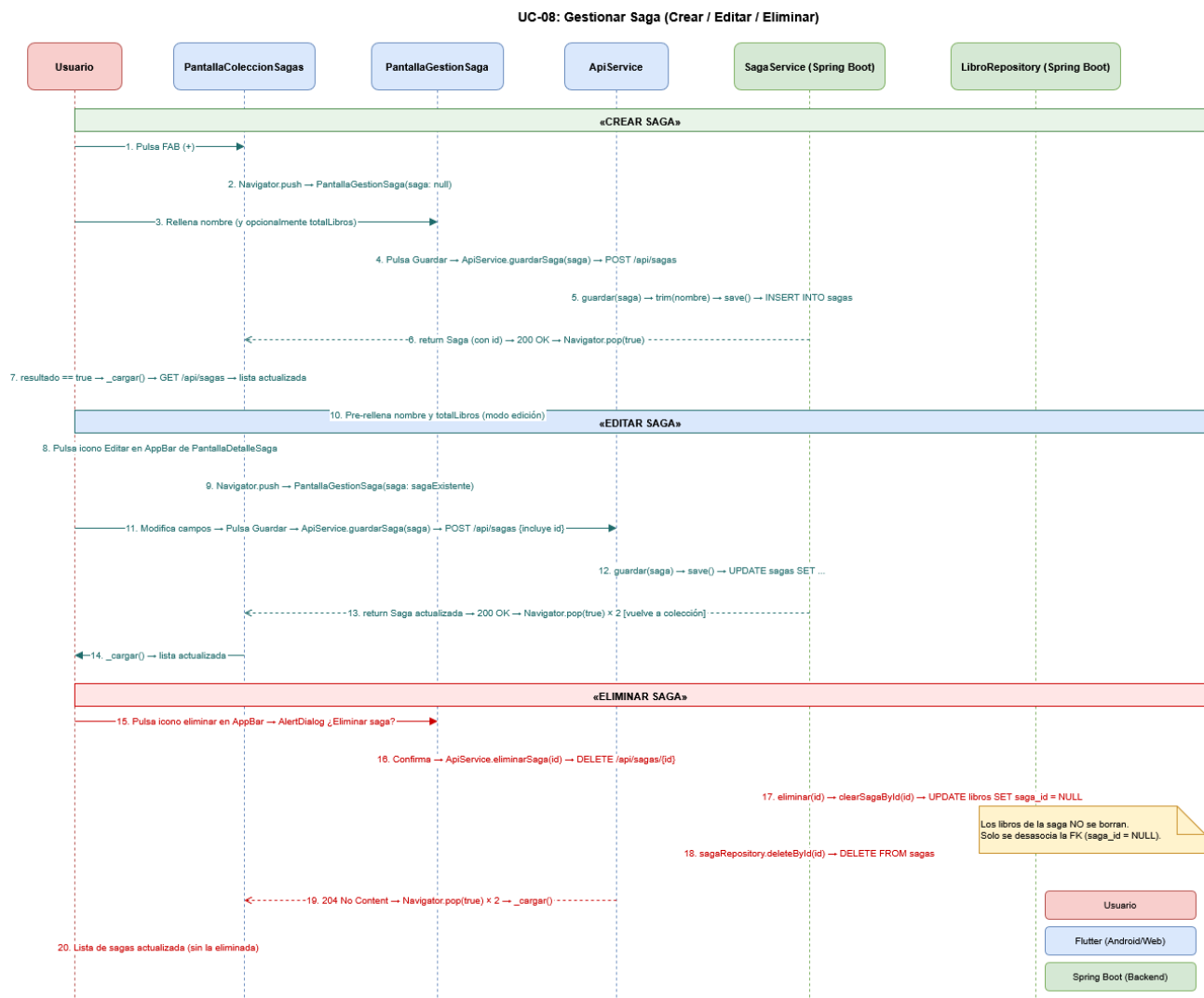


Figura 13: UC-08 Gestionar Saga

UC-09	Asignar Libro Existente a Saga		
Actores	Usuario autenticado		
Pre condiciones	La saga existe en la base de datos. Existen libros en la colección no asignados a esta saga. El backend está operativo.		
Post condiciones	El libro queda con saga_id apuntando a la saga. La pantalla de detalle de la saga muestra el libro.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa el FAB (playlist_add) en PantallaDetalleSaga.	El sistema abre DialogoAsignarSaga con la lista de libros NO asignados a esta saga (filtro local sobre todosLosLibros).
	2	El usuario selecciona un libro de la lista.	Aparece campo opcional para introducir el número de volumen.

	3	El usuario pulsa "Asignar".	ApiService.asignarSagaALibro → PATCH /api/libros/{id}/saga ?saga=NombreSaga&numLibroSaga=N.
	4	-	LibroService.asignarSaga: a. autoCrearSagaSiNecesario(sagaNombre) → si no existe. b. libro.setSaga(resolverSaga(sagaNombre)) → FK actualizada. c. libro.setNumLibroSaga(numLibroSaga). d. libroRepository.save(libro) → UPDATE libros.
	5	-	El backend devuelve el Libro actualizado (sagaNombre @Transient).
	6	-	El diálogo actualiza el modelo local: libro.sagaNombre = saga.nombre.
	7	-	DialogoAsignarSaga devuelve true → PantallaDetalleSaga → setState().
	8	-	El GridView de la saga muestra el libro recién asignado.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1a	Todos los libros ya están asignados a esta saga	el diálogo muestra "No hay libros disponibles para asignar".
	3a	Error de red	SnackBar de error → el diálogo permanece abierto.
Frecuencia	Media		
Importancia	Media		

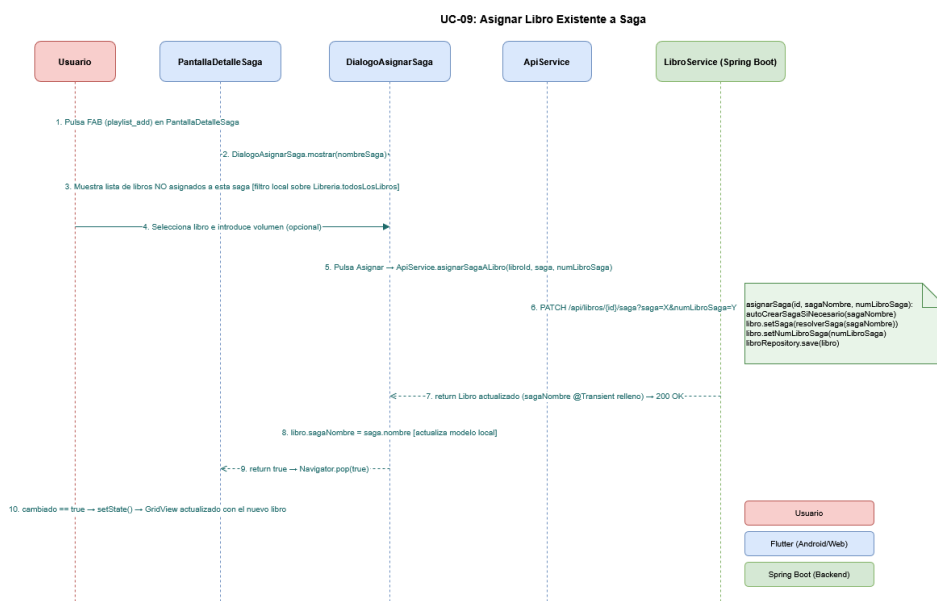
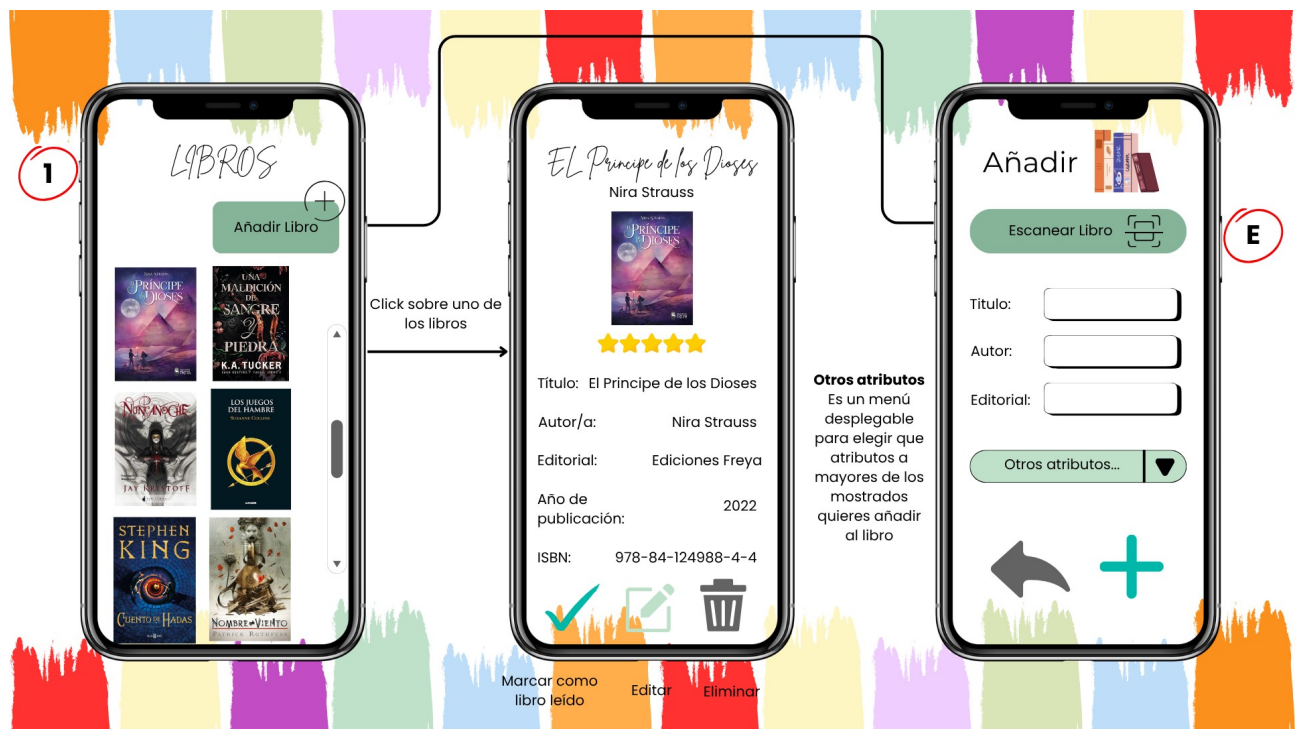
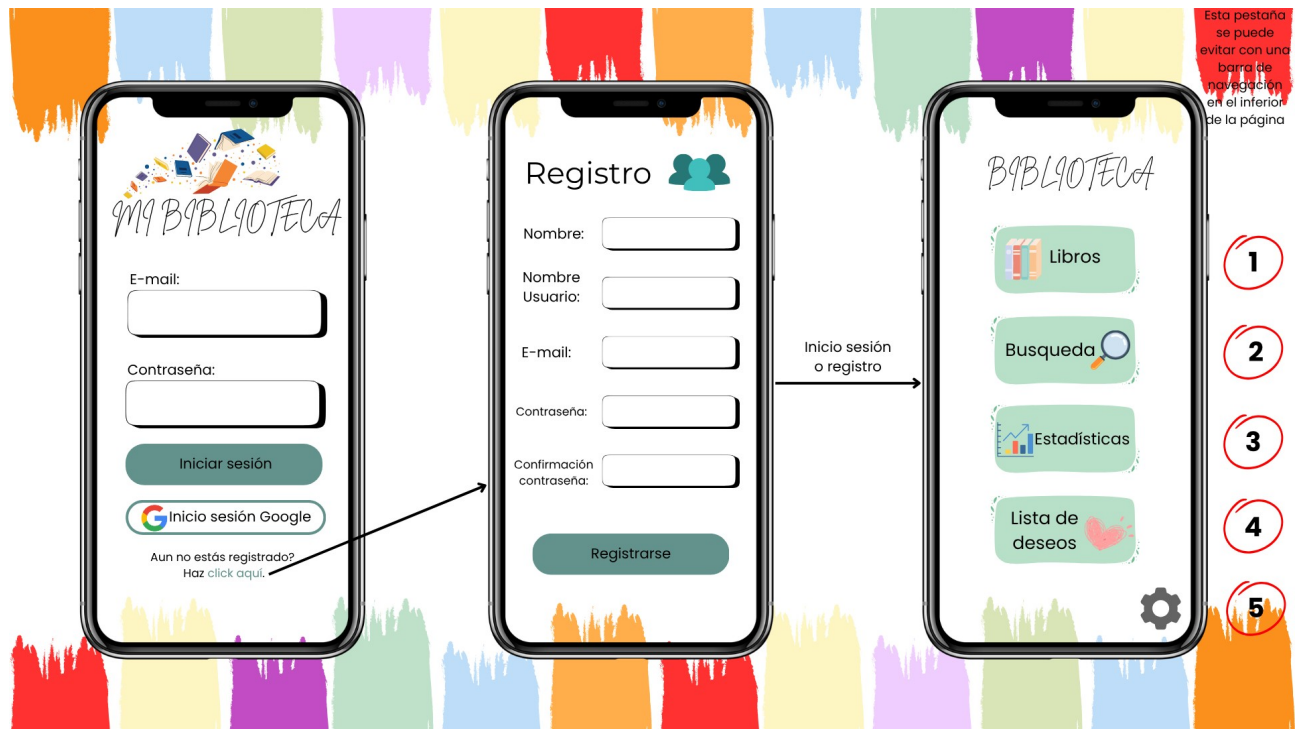
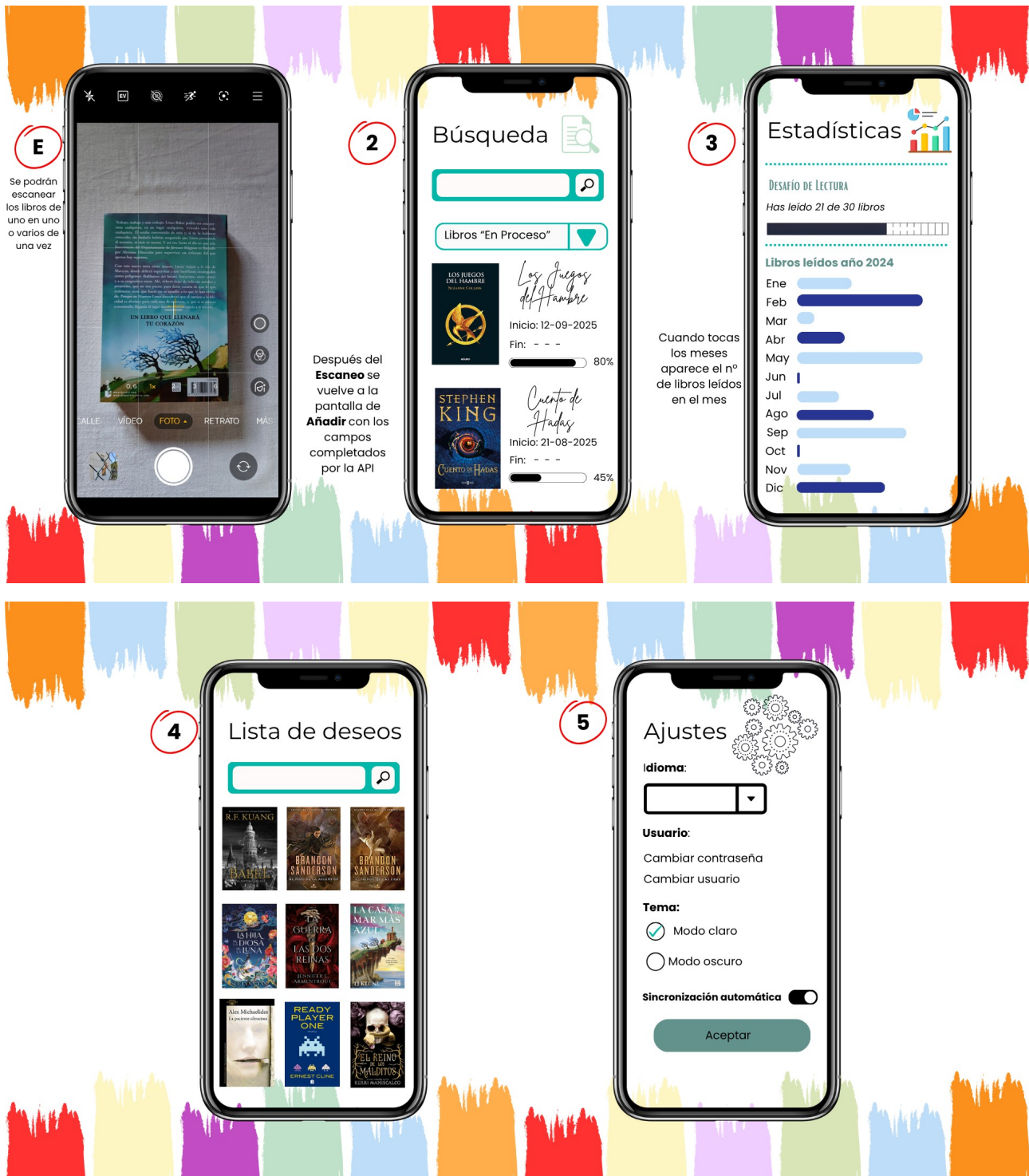


Figura 14: UC-09 Asignar Libro Existente a Saga

Mockups de interfaz





6.5 Presupuesto completo

Mano de Obra (Personal)

Para el cálculo, estimaremos un perfil de **Desarrollador Fullstack Junior** trabajando en el proyecto durante las 12 semanas (aproximadamente 300 horas de trabajo efectivo).

Perfil	Horas Estimadas	Coste/Hora	Total
Analista /Diseñador	40h	35,00€	1.400,00€
Desarrollador Backend (Java/Spring)	120h	30,00€	3.600,00€
Desarrollador Frontend (Flutter/Dart)	120h	30,00€	3.600,00€
Tester / QA	20h	25,00€	500,00€
Total Mano de Obra			9.100,00€

Hardware

Se presupone el equipo necesario para desarrollar, compilar y probar la aplicación en entornos móviles y de servidor.

Concepto	Descripción	Coste Estimado
Ordenador de desarrollo	Portatil/Sobremesa (mín. 16GB RAM para IntelliJ/Emuladores)	1.200€
Dispositivo móvil de pruebas	Smartphone Android/iOS para testing real	300€
Total Hardware		1.500€

Software e Infraestructura

Afortunadamente, el stack tecnológico elegido (**Spring Boot, Flutter, MySQL**) es de código abierto, lo que reduce drásticamente los costes de licencias.

Concepto	Detalles	Coste Estimado
Licencias de Software	IntelliJ IDEA (Community), VS Code, MySQL	0€ (Open Source)
Servidor de Base de Datos	Hosting/Cloud (Ej. AWS/Azure - Tier Gratuito/Básico)	50€/año
Cuenta Desarrollador Google Play	Pago único para subir la App	25€
Cuenta Desarrollador Apple	Pago anual (opcional)	99€/año
Total Software		174€

Resumen Presupuesto Total

Categoría	Importe
Mano de Obra	9.100€
Hardware	1.500€
Software e Infraestructura	174€
SUBTOTAL	10.774€
IVA (21%)	2.262,54€
TOTAL del Proyecto	13.036,54€

7. Desarrollo y ejecución

7.1 Diagrama de Despliegue

El diagrama de despliegue refleja la distribución física de los componentes del sistema en los diferentes escenarios de ejecución: **despliegue en la nube**, orientado al uso permanente de la aplicación desde cualquier dispositivo o lugar y **despliegue local mediante Docker Compose**, pensado para entornos de demostración o presentación sin dependencia de servicios externos.

Ambos escenarios comparten la misma base de código y la misma API REST como punto de comunicación entre cliente y servidor, variando únicamente la infraestructura sobre la que se ejecutan los componentes.

Diagrama de Despliegue — UxiLibris

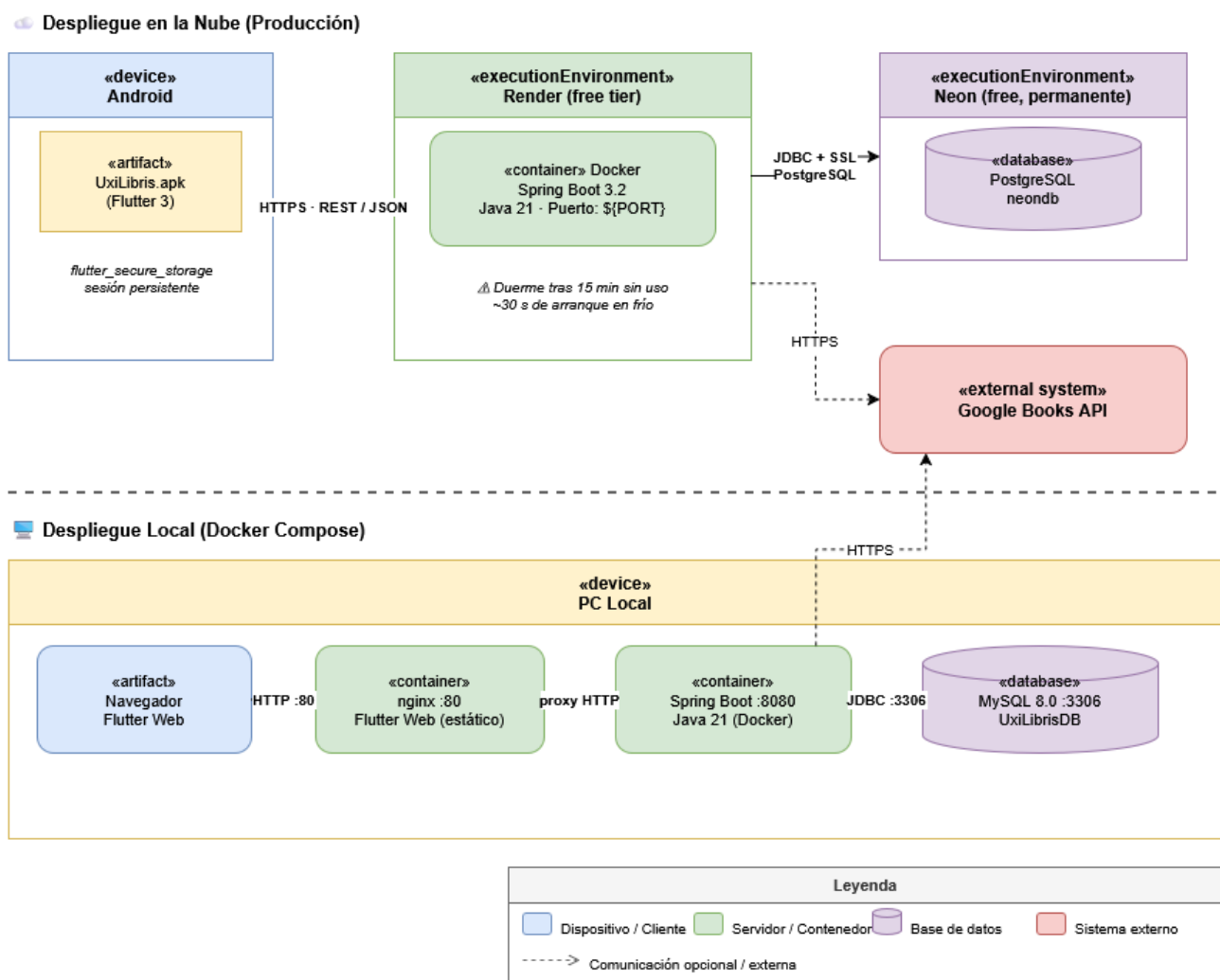


Figura 15: Diagrama de Despliegue

7.1.1 Despliegue en la Nube (Producción)

Nodo: «device» Android

Representa el dispositivo móvil Android del usuario final sobre el que se ejecuta la aplicación. Contiene **UxiLibris.apk**, generado a partir del código fuente Flutter mediante el comando *flutter build apk* con la URL del backend inyectada en tiempo de compilación a través del mecanismo *--dart-define=BACKEND_URL*.

La aplicación hace uso de la librería *flutter_secure_storage* para persistir las credenciales de sesión de forma cifrada en un almacén seguro del dispositivo (Keystore en Android), de modo que el usuario no necesita autenticarse de nuevo al reabrir la app.

La comunicación con el backend se realiza exclusivamente mediante peticiones **HTTPS** hacia la URL pública del servicio desplegado en Render, por lo que el dispositivo no requiere ninguna conexión física ni red local con el servidor.

Nodo: «executionEnviroment» Render

Representa el entorno de ejecución en la nube proporcionado por la plataforma **Render** en su plan gratuito. Actúa como servidor de aplicaciones y expone públicamente la API REST del sistema.

Internamente aloja el siguiente componente:

Componente: «container» Spring Boot 3.2 (Docker)

El backend de UxiLibris se despliega como un contenedor Docker construido a partir de un *Dockerfile* multietapa: una primera etapa compila el proyecto con Maven y Java 21, y una segunda etapa genera la imagen de producción usando únicamente el JRE (Java Runtime Enviroment) de Alpine, reduciendo el tamaño final de la imagen.

Spring Boot arranca en el puerto indicado por la variable del entorno *PORT*, que Render asigna dinámicamente en cada despliegue. El contenedor expone los siguientes grupos de endpoints bajo el prefijo */api*:

- */api/auth* → registro e inicio de sesión de usuarios
- */api/libros* → operaciones CRUD sobre la colección de libros
- */api/sagas* → gestión de sagas literarias
- */api/libros/buscar* y *api/libros/isbn/{isbn}* → integración con Google Books

La lógica de negocio se implementa con Spring Data JPA, que gestiona automáticamente el esquema de la base de datos gracias a la configuración *ddl-auto=update*.

Limitación del plan gratuito: Render suspende el servicio tras 15 minutos de inactividad. La primera petición tras un periodo de reposo provoca un arranque en frío que puede tardar entre 60 y 120 segundos. En ese intervalo el servidor puede devolver 503 inmediatamente (sin mantener la conexión abierta), por lo que la aplicación implementa un mecanismo de reintento automático: hasta

8 intentos con 8 segundos de pausa entre ellos, mostrando al usuario un mensaje de «el servidor está arrancando» a los 8 segundos del primer intento.

Nodo: «executionEnviroment» Neon

Representa el servicio de base de datos relacional en la nube proporcionado por **Neon** en su plan gratuito. Neon ofrece ProsgreSQL como servicio gestionado con datos persistentes, sin fecha de expiración.

Componente: «database» PostgreSQL (neondb)

Almacena de forma persistente toda la información del sistema: usuarios registrados, libros de la biblioteca, sagas, historiales de lectura y propiedades dinámicas de los libros. El esquema es generado y mantenido automáticamente por Hibernate al arrancar el backend.

La conexión desde Render hacia Neon se establece mediante **JDBC sobre SSL** (Secure Sockets Layer) (*sslmode=require*), utilizando la URL de conexión directa de Neon (sin el pooler de PgBouncer) para garantizar la compatibilidad con las operaciones DDL de Hibernate. Las credenciales se configuran como variables de entorno en Render y nunca se incluyen en el código fuente.

Nodo: «external system» Google Books API

Representa el servicio externo de búsqueda bibliográfica de Google, consumido por el backend a través de su API REST pública. UxiLibris lo utiliza para dos funcionalidades:

- **Búsqueda por ISBN:** dado un código de barras escaneado, recupera título, autor, portada y datos de saga del libro correspondiente.
- **Búsqueda por texto libre:** permite al usuario buscar libros por título o autor y seleccionar uno de los resultados para añadirlos a su biblioteca.

La comunicación se produce siempre desde el backend hacia Google Books mediante **HTTPS**, evitando así exponer la clave de la API en el cliente móvil. Si no se configura una clave propia, la cuenta anónima compartida puede agotarse con uso frecuente.

7.1.2 Despliegue Local (Docker Compose)

Nodo: «device» PC Local

Representa el ordenador del usuario sobre el que se ejecuta la totalidad de la infraestructura de forma local y autocontenida, orquestada mediante **Docker Compose**. Este escenario no requiere conexión a servicios externos ni cuentas en plataformas de nube, siendo suficiente con tener instalado Docker Desktop.

Docker Compose define y gestiona cuatro servicios interconectados mediante una red interna privada:

Componente: «**artifact**» **Navegador – Flutter Web**

El usuario accede a la aplicación a través de un navegador web convencional (Chrome, Firefox, etc.) en la dirección `http://localhost`. Los ficheros estáticos de la versión web de Flutter se generan previamente mediante `flutter build web --release` y son servidos por nginx.

Componente: «**container**» **nginx :80**

Servidor web ligero que actúa como punto de entrada para el tráfico HTTP en el puerto 80. Aporta los ficheros estáticos del frontend Flutter Web y está configurado con la directiva `try_files` para soportar el enrutamiento de aplicación de página única (SPA), redirigiendo cualquier ruta desconocida a `index.html`.

Componente: «**container**» **Spring Boot :8080**

Instancia del backend idéntica en código a la del despliegue en Render, pero conectada a la base de datos MySQL local en lugar de a Neon. Arranca únicamente cuando la base de datos ha superado su comprobación de salud (healthcheck), garantizando que las tablas pueden crearse correctamente antes de recibir peticiones.

Componente: «**database**» **MySQL 8.0 :3306 (UxiLibrisDB)**

Sistema gestor de base de datos relacional que almacena localmente los mismos datos que PostgreSQL en el despliegue en la nube. La imagen oficial `mysql:8.0` se configura con las credenciales definidas en el fichero `.env` (nunca en el código fuente) y expone el puerto 3306 únicamente dentro de la red interna de Docker Compose.

7.2 Incidencias técnicas resueltas durante el desarrollo

7.2.1 Backend (Spring Boot)

Error 1: Inyección de dependencia ausente en LibroController.

Causa: `googleBooksService` carecía de `@Autowired`. Spring no inyectó el bean y cualquier llamada al endpoint ISBN lanzaba `NullPointerException` en tiempo de ejecución.

Solución: Añadir `@Autowired` al campo.

Error 2: Rutas duplicadas en LibroController.

Causa: La clase tenía el prefijo de clase `/api/libros` y los métodos usaban rutas como `"/libros/isbn/{isbn}"`, generando `/api/libros/libros/isbn/{isbn}`. Ninguna petición del cliente coincidía.

Solución: Eliminar el prefijo redundante en cada método, dejando solo el segmento relativo (`"/isbn/{isbn}"`).

Error 3: orphanRemoval lanzando excepción al actualizar un libro (primera vez).

Causa: Flutter no enviaba el campo propiedades en el JSON de actualización. Jackson lo deserializaba como null e Hibernate, con orphanRemoval = true, lanzaba excepción al intentar reemplazar la colección gestionada por null.

Solución: En LibroService, si propiedades llega como null, inicializarlo a una lista vacía en lugar de intentar copiar la colección gestionada:

```
if (libro.getPropiedades() == null) libro.setPropiedades(new ArrayList<>());
```

Error 4: orphanRemoval — mismo objeto como origen y destino en sincronizarLecturas.

Causa: En sincronizarLecturas, se pasaba libro.getLecturas() tanto como destino como fuente. El destino.clear() vaciaba también la referencia de origen, por lo que el bucle de sincronización iteraba sobre una lista vacía y el libro se guardaba sin fechas de lectura.

Solución: Pasar una copia de la lista: `sincronizarLecturas(libro, new ArrayList<>(libro.getLecturas()))`.

Error 5: Spring devolvía fechas como array [2024, 1, 15] en lugar de cadena ISO.

Causa: Sin `spring.jackson.serialization.write-dates-as-timestamps=false`, Jackson serializaba LocalDate como array numérico. DateTime.parse() en Dart solo acepta cadenas, por lo que el parseo fallaba silenciosamente y la lista de libros quedaba vacía.

Solución: Añadir la propiedad en application.properties y como variable de entorno en Render (`SPRING_JACKSON_SERIALIZATION_WRITE_DATES_AS_TIMESTAMP=false`).

Error 6: PATCH bloqueado por CORS.

Causa: WebConfig.java solo incluía GET, POST, PUT, DELETE en los métodos CORS permitidos. Las peticiones PATCH (usadas para favorito y saga) fallaban sin error visible en Flutter.

Solución: Añadir "PATCH" a la lista de métodos permitidos en la configuración CORS.

Error 7: @JsonIgnore ausente en PropiedadValor.libro — riesgo de recursión infinita

Causa: La referencia de vuelta PropiedadValor → Libro → PropiedadValor → ... carecía de @JsonIgnore. Jackson intentaba serializar el ciclo completo y lanzaba *StackOverflowError*.

Solución: Anotar el campo libro en PropiedadValor con @JsonIgnore.

Error 8: Valor DESEO del enum rechazado por MySQL

Causa: Al añadir el estado DESEO para la lista de deseos, la columna estado en MySQL seguía definida como ENUM('PENDIENTE','LEYENDO','LEIDO'). Insertar un libro con ese estado provocaba *SQL Error 1265: Data truncated*.

Solución: `ALTER TABLE libros MODIFY COLUMN estado ENUM('PENDIENTE','LEYENDO','LEIDO','DESEO');`

Error 9: Creación de libro siempre usaba POST, generando duplicados al editar.

Causa: ApiService.guardarLibro usaba POST /api/libros independientemente de si el libro tenía ID. El backend, con *@GeneratedValue*, ignoraba el ID recibido y creaba un nuevo registro.

Solución: Usar PUT /api/libros/{id} cuando el libro tiene ID y POST solo para nuevos. Añadir el endpoint PUT al controlador.

Error 10: El libro creado no devolvía el ID del servidor.

Causa: Tras guardar, guardarLibro devolvía solo *bool* descartando la respuesta con el ID generado. El objeto local mantenía id: null y en la siguiente edición se creaba un duplicado.

Solución: Cambiar guardarLibro para devolver el Libro completo deserializado de la respuesta del servidor.

Error 11: Saga no creada automáticamente al guardar un libro con saga nueva.

Causa: Si el usuario escribía un nombre de saga que no existía en la tabla sagas, el libro se guardaba con el nombre, pero la saga no aparecía en la pantalla de colección de sagas.

Solución: Añadir autoCrearSagaSiNecesario en LibroService que verifica si la saga existe antes del *save()* y la crea si no.

Error 12: Google Books API — cuota agotada propagada como HTTP 500

Causa: El bucle de reintentos del escáner (ver más abajo) agotó la cuota diaria de Google Books. La API devolvía 429, pero GoogleBooksService no lo capturaba, propagando un *HttpClientErrorException* como error 500.

Solución: Capturar *HttpClientErrorException* en GoogleBooksService y relanzar como *ResponseStatusException(429)*. En Flutter, *_errorISBN()* traduce 429 y 503 a mensajes legibles.

7.2.2 Base de datos (MySQL)

Error 13: MySQL Workbench: Error 1046 — No database selected

Causa: Ejecutar sentencias SQL sin seleccionar la base de datos activa.

Solución: Hacer doble clic sobre la base de datos en el panel Schemas antes de ejecutar.

Error 14: MySQL Workbench: Error 1175 — Safe update mode

Causa: El modo seguro rechazaba *UPDATE con WHERE I=I* por no usar una columna clave en el filtro.

Solución: Desactivar Safe Updates en Preferencias → SQL Editor, o usar la clave primaria en el *WHERE*.

7.2.3 Frontend (Flutter)

Error 15: Inferencia de tipos en listas vacías en formularioLibroController.

Causa: Dart no podía inferir el tipo de `([], [])` en el retorno de una tupla.

Solución: Tipar explícitamente: `(<String>[], <String>[])`.

Error 16: `Future.wait` con tipos genéricos distintos.

Causa: `Future.wait<T>` requiere que todos los futuros compartan el mismo tipo. Se mezclaba `Future<double>` con `Future<List<double>>`.

Solución: Sustituir `Future.wait` por dos `await` secuenciales.

Error 17: Portada de Google Books no visible en las cards.

Causa: `_buildImagen()` en `cardLibro.dart` solo manejaba rutas `assets/` y ficheros locales. Las URLs `http://...` de Google Books no tenían rama propia y no se renderizaban.

Solución: Añadir rama `Image.network` para rutas que empiecen por `http`, con `errorBuilder` de fallback.

Error 18: Portada local sin `errorBuilder` — pantalla rota si el archivo no existe.

Causa: `Image.file` sin `errorBuilder` mostraba un widget de error rojo si la ruta ya no existía en el dispositivo.

Solución: Añadir `errorBuilder` que muestre la portada por defecto en todos los puntos que usen `Image.file`.

Error 19: `toJson()` del libro no incluía `rutaImagen`.

Causa: El campo `rutaImagen` no se serializaba al guardar. El backend almacenaba siempre la ruta por defecto y al recargar todos los libros aparecían sin portada.

Solución: Añadir `'rutaImagen': rutaImagen` al mapa de `toJson()`.

Error 20: Escáner mostraba "Actualizar Libro" en un libro nuevo.

Causa: La condición `widget.libroParaEditar == null` no distinguía entre un libro recién escaneado (sin ID) y uno existente de la base de datos.

Solución: Cambiar la condición a `widget.libroParaEditar?.id == null`.

Error 21: Clave `estadoLectura` vs `estado` entre Flutter y Spring.

Causa: `toJson()` enviaba el estado bajo la clave `estadoLectura` pero la entidad Java tenía el campo `estado`. Todos los libros se guardaban con `estado` null y cargaban siempre como pendiente.

Solución: Cambiar la clave en `toJson()` y `fromJson()` a `'estado'`.

Error 22: numLibroSaga no se resetea al borrar el campo saga.

Causa: Al vaciar el nombre de saga en el formulario, el número de volumen conservaba su valor anterior y se guardaba en el libro sin saga.

Solución: Poner `numLibroSaga = null` explícitamente cuando el campo saga esté vacío.

Error 23: Fechas enviadas como datetime completo en lugar de solo fecha.

Causa: `Lectura.toJson()` usaba `DateTime.toIso8601String()`, produciendo "2024-01-15T10:30:00". Jackson no podía parsear eso como `LocalDate` y guardaba null.

Solución: Truncar a solo la parte de fecha: `.toIso8601String().substring(0, 10)`.

Error 24: Doble `Navigator.pop` corrompía la pila de navegación.

Causa: `gestionLibro.dart` llamaba a pop en el callback al Guardar, y `formularioLibro.dart` también. Los dos pops consecutivos dejaban una pantalla negra.

Solución: Eliminar el pop de `gestionLibro.dart`; solo `formularioLibro.dart` lo ejecuta.

Error 25: Escáner reintentando en bucle y apilando `SnackBars`.

Causa: Al no encontrar un ISBN, la cámara se reactivaba inmediatamente, detectaba el mismo código y disparaba otra llamada API. Se acumulaban decenas de errores superpuestos.

Solución: Añadir un retardo de 3 segundos antes de reactivar la cámara. Llamar a `ScaffoldMessenger.clearSnackBars()` antes de cada nuevo `SnackBar`.

Error 26: Teclado tapando el botón de inicio de sesión.

Causa: Al enfocar el campo de contraseña el teclado cubría el botón y el scroll no lo compensaba automáticamente.

Solución: Añadir `scrollPadding: EdgeInsets.only(bottom: 100)` al `TextFormField` de la contraseña.

Error 27: Datos del usuario anterior visibles tras cambio de sesión.

Causa: `Libreria.todosLosLibros` es estática. Al cerrar sesión sin limpiarla, el siguiente usuario veía los libros del anterior porque `cargarLibros()` devolvía la caché si no estaba vacía.

Solución: Limpiar `Libreria.todosLosLibros` al cerrar sesión y al iniciar una nueva.

Error 28: `DatePickerDialog` en formato MM/DD/YYYY.

Causa: Sin configurar la localización española, el selector de fecha usaba el formato estadounidense en el modo de teclado.

Solución: Añadir `flutter_localizations` y configurar `MaterialApp` con `localizationsDelegates` y `supportedLocales: [Locale('es')]`.

Error 29: static final del tema no actualizado tras hot reload.

Causa: Los campos static final se calculan una sola vez al cargar la clase. Cambios en el tema durante el desarrollo no se reflejaban con hot reload.

Solución: Realizar hot restart completo tras modificar campos estáticos.

Error 30: SingleChildScrollView anulando *mainAxisAlignment.center*.

Causa: Al envolver una Column centrada en un SingleChildScrollView, la columna recibía altura infinita y el centrado perdía efecto.

Solución: Envolver la Column en un ConstrainedBox con `minHeight: MediaQuery.of(context).size.height`.

Error 31: Card de saga mostrando "6 de 5 libros".

Causa: libros.length contaba todos los volúmenes asignados a la saga sin filtrar por el rango definido en totalLibros. Un volumen con número 6 en una saga de 5 desbordaba el contador.

Solución: Filtrar para contar solo libros cuyo numLibroSaga está en el rango 1..totalLibros.

Error 32: Pantalla de saga no refrescando al volver de la colección.

Causa: Los cambios realizados en libros desde la pantalla de saga actualizaban Libreria.todosLosLibros en memoria pero no disparaban setState en la pantalla de colección abierta.

Solución: La pantalla de colección llama a setState al recuperar el foco mediante observación de rutas.

Error 33: El mismo libro podía escanearse y añadirse varias veces.

Causa: No había comprobación de ISBN duplicado antes de abrir el formulario de nuevo libro.

Solución: Antes de navegar al formulario, verificar si ya existe un libro con ese ISBN en Libreria.todosLosLibros.

Error 34: No se podía eliminar la fecha de lectura al cambiar estado de "leído" a "leyendo".

Causa: La condición puedeEliminar ocultaba el botón de borrado cuando solo existía una lectura y el estado era "leyendo", aunque esa lectura estuviese completada.

Solución: Ampliar la condición: `|| lec.estaCompletada`.

Error 35: Libro eliminado seguía apareciendo en estadísticas

Causa: La pantalla de estadísticas usaba la caché de ColeccionController, que no se actualizaba tras una eliminación.

Solución: La pantalla de estadísticas llama directamente a `ApiService.fetchLibros()` en cada carga, ignorando la caché.

Error 36: Portadas locales perdidas al reinstalar la aplicación

Causa: Las portadas seleccionadas de la galería se copiaban a `getApplicationDocumentsDirectory()`. Android elimina ese directorio al desinstalar la app.

Solución provisional: `errorBuilder` en todos los widgets de imagen para mostrar la portada por defecto. Solución estructural pendiente: almacenamiento en servicio cloud (*Cloudinary*).

7.2.4 Despliegue

Error 37: `application.properties` rastreado por git tras añadirlo a `.gitignore`

Causa: El archivo fue incluido en `.gitignore` después de haber sido ya confirmado. Git continuaba rastreándolo.

Solución: `git rm --cached application.properties`.

Error 38: Regla de negación en `.gitignore` ignorada por espacio

Causa: La regla `!.env.example` (con espacio tras `!`) era inválida según la especificación de `.gitignore`.

Solución: Cambiar a `!.env.example`.

Error 39: Render: "No open ports detected" — puerto estático no compatible con entorno dinámico

Causa: Spring Boot arrancaba siempre en el puerto 8080. Render asigna el puerto en tiempo de ejecución mediante la variable `PORT`.

Solución: `ENTRYPOINT ["sh", "-c", "java -jar app.jar --server.port=${PORT:-8080}"]`

Error 40: Render: tablas no creadas (*SQLException: relation does not exist*)

Causa: Sin `application.properties` en el repositorio, `ddl-auto` tomaba el valor por defecto `none` para PostgreSQL e Hibernate nunca creaba el esquema.

Solución: Configurar `SPRING_JPA_HIBERNATE_DDL_AUTO=update` como variable de entorno en Render.

Error 41: Neon: error de dialecto por URL JDBC malformada

Causa: La URL incluía credenciales embebidas y `?sslmode=require` duplicado, impidiendo que Hibernate determinara el dialecto.

Solución: URL sin credenciales y sin parámetros duplicados; usuario y contraseña en variables separadas.

Error 42: Neon: pooler PgBouncer incompatible con DDL de Hibernate

Causa: El hostname con sufijo `-pooler` usa PgBouncer en modo `transaction`, que cierra la conexión entre transacciones e impide que Hibernate ejecute sentencias DDL.

Solución: Usar el hostname directo (sin `-pooler`) en `SPRING_DATASOURCE_URL`.

Error 43: Flutter apuntando a IP local tras despliegue en Render

Causa: La app compilada seguía usando 192.168.1.200:8080 como backend.

Solución: `flutter build apk --dart-define=BACKEND_URL=https://uxilibris-backend.onrender.com`

Error 44: Timeout insuficiente ante el arranque en frío de Render

Causa: El plan gratuito duerme el servidor tras 15 minutos de inactividad. Android cerraba la conexión (~15 s) antes de que Render terminara de arrancar (~60 s).

Solución: En primer lugar se separaron los timeouts por tipo de petición (`_timeoutLogin = 30 s`, `_timeoutFetch = 90 s`). Sin embargo, dado que Render devuelve 503 inmediatamente durante el arranque en frío (sin mantener la conexión), el timeout por sí solo no era suficiente. La solución definitiva fue implementar un bucle de reintentos automáticos en `_cargarYFiltrar()`: hasta 8 intentos con 8 segundos de pausa entre ellos. Superados todos los intentos sin éxito, se muestra un estado de error con botón de reintento manual.

Error 45: JVM sin límites de memoria superando los 512 MB de Render (OOM silencioso)

Causa: El backend compilaba y arrancaba correctamente, pero Render mostraba «No open ports detected / Port scan timeout reached» sin ningún mensaje de error visible en los logs. Spring Boot llegaba a registrar el arranque de Tomcat en el puerto correcto, pero el proceso moría antes de que Render detectara el puerto.

Solución: Establecer límites explícitos en el ENTRYPOINT del Dockerfile:

```
ENTRYPOINT ["sh", "-c", "java -Xms64m -Xmx280m -XX:MaxMetaspaceSize=96m -XX:+UseSerialGC -jar app.jar --server.port=${PORT:-8080}"]
```

Error 46: El backend arrancaba en Render pero las consultas a sagas lanzaban `SQLException: column s1_0.abandonada does not exist`, impidiendo cargar o listar sagas.

Causa: Se añadió el campo `private boolean abandonada = false;` a la entidad Saga. Hibernate con `ddl-auto=update` intenta ejecutar `ALTER TABLE sagas ADD COLUMN abandonada boolean NOT NULL`, pero PostgreSQL rechaza añadir una columna `NOT NULL` sin valor por defecto a una tabla que ya contiene filas. Hibernate no propaga el error a los logs de arranque, por lo que la columna nunca se creaba.

Solución:

1. Ejecutar manualmente en el SQL Editor de Neon:
`ALTER TABLE sagas ADD COLUMN IF NOT EXISTS abandonada boolean NOT NULL DEFAULT false;`
2. Añadir `columnDefinition` a la anotación de la entidad para que futuros despliegues en bases de datos nuevas funcionen sin intervención manual:
`@Column(nullable = false, columnDefinition = "boolean DEFAULT false")`
`private boolean abandonada = false;`

Error 47: El job de Flutter en GitHub Actions reportaba «34 tests passed, 1 failed» con el mensaje `loading widget_test.dart (failed)`, aunque ese archivo no contenía ningún test.

Causa: Dart exige que todo archivo ejecutado como programa (incluidos los archivos de test que flutter test descubre automáticamente) contenga una función `main()`. El archivo `widget_test.dart` había sido vaciado de contenido —los tests de widget requieren dispositivo real— dejando solo comentarios, lo que hacía que el runner no pudiera cargarlo.

Solución: Añadir una función `main()` vacía al archivo.

8. Conclusiones y reflexiones

8.1 Valoración global

UxiLibris ha alcanzado todos los objetivos funcionales planteados en la fase de análisis. La aplicación permite gestionar una biblioteca personal completa, con registro de lecturas, puntuaciones, sagas, favoritos, lista de deseos y estadísticas anuales. Está operativa en dos plataformas (web y Android) y cuenta con un despliegue real accesible desde internet.

El resultado final es una aplicación funcional, visualmente coherente y con una arquitectura bien definida en capas: cliente Flutter, API REST Spring Boot y base de datos relacional. La separación de responsabilidades entre componentes ha permitido que cada parte del sistema sea independiente, testable de forma aislada y desplegable en distintos entornos sin modificar el código fuente.

La incorporación de una suite de pruebas automatizadas y un pipeline de integración continua con GitHub Actions cierra el ciclo de calidad del proyecto: cada cambio en el repositorio es verificado automáticamente antes de llegar a producción.

Desde el punto de vista personal, el proyecto ha supuesto un recorrido completo por el ciclo de vida del desarrollo de software: análisis de requisitos, diseño de base de datos, implementación de frontend y backend, integración entre servicios externos, y despliegue en producción. Abordar todas esas fases de forma individual ha exigido un nivel de autonomía y resolución de problemas que ha resultado muy formativo.

8.2 Resumen de lo aprendido

Flutter y desarrollo multiplataforma

Se ha adquirido experiencia práctica en el desarrollo de aplicaciones Flutter orientadas tanto a la web como a Android desde un único código base. Se ha trabajado con gestión de estado mediante `StatefulWidget`, comunicación asíncrona con `Future` y `async/await`, navegación con rutas nombradas, formularios con validación, y diseño adaptativo con `MediaQuery`. También se ha aprendido a separar la lógica de presentación en controladores estáticos independientes del árbol de widgets.

Spring Boot y diseño de API REST

El desarrollo del backend ha permitido consolidar los conceptos de diseño de API RESTful con Spring Boot: controladores, servicios, repositorios JPA, entidades con relaciones, y manejo de errores con códigos HTTP apropiados. Se ha trabajado con dos motores de base de datos distintos (MySQL en local y PostgreSQL en producción), lo que ha obligado a entender las diferencias entre ambos y a configurar la capa de persistencia de forma que fuera compatible con los dos.

Despliegue y entornos

Se ha aprendido a crear y gestionar un entorno Docker completo con docker-compose, incluyendo la definición de servicios dependientes, variables de entorno y comprobaciones de salud (healthcheck). Para el despliegue en la nube se ha trabajado con Render (backend) y Neon (base de datos PostgreSQL serverless), lo que ha implicado comprender el comportamiento de plataformas gratuitas con restricciones de recursos.

Integración de servicios externos

La integración con la API de Google Books ha permitido aprender a consumir servicios REST externos desde el backend, actuar como intermediario frente al cliente para proteger las claves de API y gestionar los límites de cuota. El escáner de ISBN mediante la cámara del dispositivo ha introducido el uso de plugins nativos de Flutter.

Depuración en entornos distribuidos

Uno de los aprendizajes más relevantes ha sido aprender a diagnosticar errores en un sistema con varias capas: distinguir si un fallo proviene del cliente, de la red, del servidor o de la base de datos. Herramientas como los logs de Render, los errores HTTP en Flutter DevTools y el cliente REST de prueba han sido fundamentales para aislar problemas que no eran reproducibles localmente.

Testing y calidad del código

Se han implementado pruebas unitarias y de integración para las dos capas del sistema. En el backend, se ha trabajado con JUnit 5 y Mockito para probar la lógica de servicio en aislamiento, y con `@DataJpaTest` junto a H2 para verificar el comportamiento de los repositorios JPA sin necesidad de una base de datos real. En el frontend, se han escrito pruebas en Dart puro —sin dispositivo ni emulador— que validan el parseo, la serialización y la lógica de filtrado de los modelos de dominio. Adicionalmente, se ha configurado un workflow de GitHub Actions que ejecuta ambos conjuntos de pruebas de forma automática en cada push, integrando el resultado como badge de estado en el README del repositorio.

8.3 Dificultades encontradas

Compatibilidad entre entornos

La mayor fuente de incidencias ha sido la diferencia de comportamiento entre el entorno de desarrollo local y el entorno de producción en la nube. Aspectos como la URL de conexión a la base de datos, el puerto dinámico asignado por Render, la incompatibilidad del pooler de Neon con las sentencias DDL de Hibernate, o la política de tráfico HTTP en Android requirieron en cada caso una investigación específica para identificar la causa raíz y aplicar la solución correcta.

Gestión del arranque en frío de Render

El plan gratuito de Render suspende el servidor tras un período de inactividad. Esto causó que la aplicación Android pareciera no cargar los libros, cuando en realidad el servidor estaba despertando. La solución fue aumentar el tiempo de espera de las peticiones HTTP en el cliente, pero identificar que ese era el problema (y no un error de código) requirió comparar el comportamiento de la versión web con el de la app nativa.

Diseño de la base de datos y evolución del modelo

El modelo de datos evolucionó a lo largo del proyecto: se añadieron entidades nuevas (sagas, lista de deseos), se modificaron relaciones y se ajustaron campos. Gestionar esa evolución con `spring.jpa.hibernate.ddl-auto=update` fue práctico durante el desarrollo, pero obligó a prestar atención a los efectos que los cambios de esquema podían tener sobre los datos existentes.

Almacenamiento de archivos en móvil

La gestión de portadas personalizadas descubrió una limitación importante del almacenamiento local en Android: los archivos guardados en el directorio privado de la aplicación se eliminan al desinstalarla. Aunque se valoró implementar almacenamiento en la nube como solución definitiva, la complejidad adicional no se consideró proporcional al alcance del proyecto y se optó por documentar el comportamiento y mantener un fallback visual adecuado.

9. Bibliografía y Webgrafía

- Abadi, D., Madden, S., y Ferreira, M. (2006). Integrating compression and execution in column-oriented database systems. Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data, 671–682. <https://doi.org/10.1145/1142473.1142548>
- Bien, B. (s. f.). flutter_secure_storage [Paquete de software]. pub.dev. https://pub.dev/packages/flutter_secure_storage
- imaNNeo (s. f.). fl_chart [Paquete de software]. pub.dev. https://pub.dev/packages/fl_chart
- García, J. A. (s. f.). *Modelo entidad-atributo-valor* [Presentación]. Prezi. https://prezi.com/p/oqdpgz1e7jt/_modelo-entidad-atributo-valor/
- Google. (s. f.). Books API overview. Google Developers. <https://developers.google.com/books/docs/v1/using>https://pub.dev/packages/flutter_secure_storage
- Google. (s. f.). *Flutter documentation*. <https://docs.flutter.dev/>
- Google. (s. f.). Material Design 3. <https://m3.material.io/>
- Provos, N., y Mazières, D. (1999). A future-adaptable password scheme. Proceedings of the USENIX Annual Technical Conference (FREENIX Track), 81–91.
- Rowe, R., y Stonebraker, M. (1987). The POSTGRES data model. Proceedings of the 13th International Conference on Very Large Data Bases, 83–96.
- Spring Team. (s. f.). *Spring Boot reference documentation*. <https://docs.spring.io/spring-boot/documentation.html>
- The Bouncy Castle Project. (s. f.). BCrypt [Implementación de referencia]. <https://www.bouncycastle.org/>
- Voorhees, R. (s. f.). provider [Paquete de software]. pub.dev. <https://pub.dev/packages/provider>
- Windmill, E. (2020). Flutter in action. Manning Publications.
- Oracle Corporation. (s. f.). MySQL 8.0 reference manual. <https://dev.mysql.com/doc/refman/8.0/en/>

10. Anexo I – Manual técnico de instalación o puesta en marcha

10.1 Introducción

10.1.1 Propósito del documento

Este Anexo I describe los procedimientos necesarios para la instalación, configuración y puesta en marcha de UxiLibris, una aplicación móvil de gestión de biblioteca personal desarrollada como Trabajo Final de DAM. Su objetivo es proporcionar a cualquier persona técnica la información suficiente para desplegar el sistema en alguna de sus modalidades disponibles, desde un entorno de desarrollo local hasta un despliegue completo en la nube.

10.1.2 Alcance

Este manual está dirigido a perfiles con conocimientos técnicos en desarrollo de software. No se asume experiencia previa con las tecnologías concretas utilizadas en el proyecto, pero sí familiaridad con conceptos generales como el uso de la terminal, la gestión de dependencias y el despliegue de aplicaciones web.

El documento no cubre el uso de la aplicación desde el punto de vista del usuario final, para ello existe el Manual de Usuario (Anexo II). Tampoco describe las decisiones de diseño o arquitectura del sistema, que se recogen en la Memoria del proyecto (Apartados 1 a 9).

10.1.3 Descripción general del sistema

UxiLibris está compuesto por dos módulos principales que deben estar operativos para que la aplicación funcione correctamente:

1. **Frontend** móvil, desarrollado con Flutter y distribuido como APK para dispositivos Android. Es la interfaz con la que interactúa el usuario.
2. **Backend**, desarrollo con Spring Boot (Java). Expone una API REST que gestiona la lógica de negocio y la persistencia de datos. Se comunica con una base de datos relacional y con la API externa de Google Books para la búsqueda e identificación de libros por ISBN.

La base de datos puede ser una instancia de MySQL en entornos locales o PostgreSQL alojado en Neon para el despliegue en producción. El backend puede ejecutarse localmente mediante Docker o desplegarse en la plataforma Render.

10.1.4 Modalidades de despliegue

El sistema admite cuatro configuraciones de despliegue, descritas en detalle en la sección 10.4:

Modalidad	Frontend	Backend	Base de datos
A – Producción en nube	APK compilado	Render	Neon – PostgreSQL
B – Docker local (navegador)	Web local	Docker	MySQL en Docker
C – APK con backend local	APK compilado	Docker	MySQL en Docker
D – Desarrollo	Futter run	IDE / Maven	MySQL local

10.2 Arquitectura del sistema

10.2.1 Diagrama de componentes

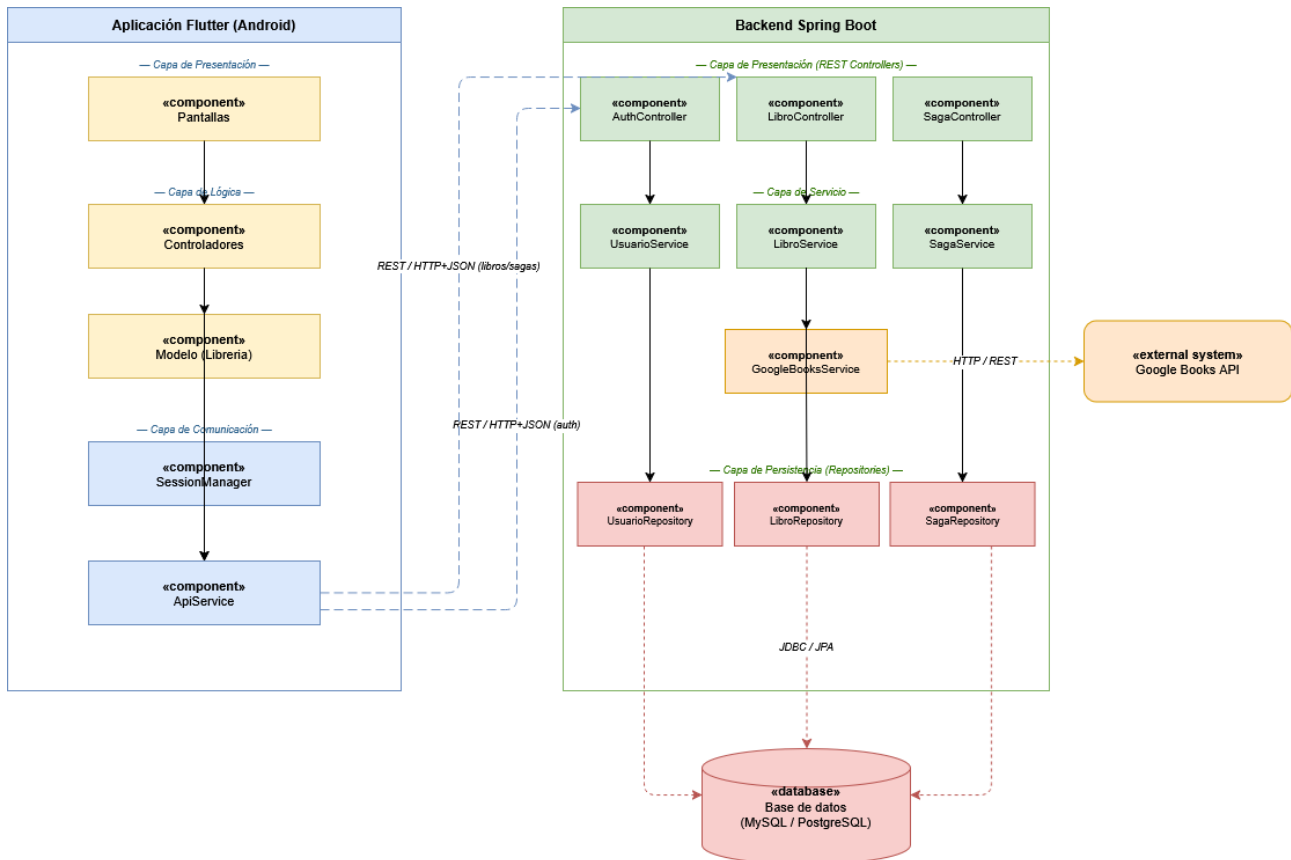


Figura 16: Diagrama de componentes

10.2.2 Descripción de los componentes del sistema

Aplicación Flutter (Android)

La aplicación móvil constituye la capa de presentación e interacción del sistema. Está desarrollada con el framework Flutter y distribuida como APK para dispositivos Android. Internamente se organiza en capas:

1. **Pantallas:** conjunto de widgets que componen la interfaz de usuario. Cada pantalla representa una funcionalidad principal de la aplicación: colección de libros, gestión de sagas, estadísticas, escáner ISBN, ajustes y autenticación. Las pantallas delegan toda la lógica a los controladores.
2. **Controladores:** clases que encapsulan la lógica de cada pantalla. Procesan las acciones del usuario, transforman los datos del modelo y coordinan las llamadas al servicio de comunicación. Entre ellos se encuentran ColeccionController, EstadisticasController, FormularioLibroController y LoginController, entre otros.

3. Modelo (Librería): conjunto de clases de dominio que representan las entidades del sistema (Libro, Saga, Lectura, Propiedad, etc.) y la colección en memoria (Librería.todosLosLibros). Actúa como caché local de los datos cargados desde el servidor.
4. SessionManager: componente encargado de mantener la sesión activa del usuario. Almacena el identificador de usuario y los datos de la cuenta de forma persistente entre sesiones mediante SharedPreferences.
5. ApiService: capa de comunicación con el backend. Centraliza todas las llamadas HTTP al servidor REST, serializa y deserializa objetos JSON, y gestiona los códigos de respuesta. Es el único punto de la aplicación que accede directamente a la red.

Backend Spring Boot

El servidor de la aplicación está desarrollado con Spring Boot (Java 17) y expone una API REST que la aplicación móvil consume. Implementa la lógica de negocio y gestiona la persistencia de todos los datos. Se organiza en cuatro capas:

1. Controllers (capa de presentación REST): reciben las peticiones HTTP del cliente, validan los parámetros de entrada y delegan el procesamiento en los servicios. El sistema dispone de tres controladores: AuthController (registro, inicio de sesión y actualización de cuenta), LibroController (gestión completa de libros, propiedades, lecturas e ISBN) y SagaController (gestión de sagas).
2. Services (capa de servicio): contienen la lógica de negocio de la aplicación. UsuarioService gestiona la autenticación y el cifrado de contraseñas con BCrypt. LibroService orquesta la creación, edición y eliminación de libros, incluyendo la sincronización de lecturas y propiedades dinámicas. SagaService gestiona el ciclo de vida de las sagas. GoogleBooksService realiza las consultas a la API externa de Google Books para la búsqueda por ISBN y texto libre.
3. Repositories (capa de persistencia): interfaces que extienden JpaRepository y proporcionan acceso a la base de datos mediante Spring Data JPA. LibroRepository, UsuarioRepository y SagaRepository son los únicos puntos de acceso a las tablas de la base de datos.
4. Entities y DTOs: las entidades (Libro, Usuario, Saga, LecturaLibro, DefinicionPropiedad, PropiedadValor) son las clases mapeadas a las tablas de la base de datos mediante JPA/Hibernate. Los DTOs (LoginRequest, LoginResponse, IsbnResultDto, ActualizarUsuarioRequest) desacoplan la representación interna de los datos de los contratos de la API.

Base de datos (MySQL / PostgreSQL)

Almacena de forma persistente todos los datos de la aplicación: usuarios registrados, libros de cada biblioteca, sagas, lecturas, propiedades personalizadas y sus valores. El esquema es generado y mantenido automáticamente por Hibernate (ddl-auto=update), por lo que no requiere scripts de creación manual.

En entorno local se utiliza MySQL 8, ejecutado como contenedor Docker. En el entorno de producción en la nube se emplea PostgreSQL alojado en Neon, que ofrece una instancia gratuita y permanentemente accesible mediante conexión SSL.

Google Books API

Servicio externo proporcionado por Google que ofrece información bibliográfica a partir de un ISBN o una búsqueda por texto libre. El backend actúa como intermediario entre la aplicación móvil y esta API: recibe la petición del cliente, consulta Google Books, transforma la respuesta al formato interno del sistema y la devuelve a Flutter. De este modo, la clave de acceso a la API nunca abandona el servidor y el cliente móvil no depende de cuotas ni credenciales propias.

10.2.3 Relaciones entre componentes

El diagrama de componentes (10.2.1) refleja las siguientes relaciones:

Flutter → Spring Boot

La aplicación móvil se comunica con el backend exclusivamente a través de ApiService, mediante llamadas HTTP con intercambio de datos en formato JSON. Existen dos canales diferenciados:

1. ApiService → AuthController: peticiones de registro, inicio de sesión y actualización de datos de cuenta. SessionManager aporta el identificador de usuario que se incluye en las consultas posteriores.
2. ApiService → LibroController y SagaController: operaciones CRUD sobre libros, sagas, lecturas, propiedades y favoritos.

Esta separación permite que el frontend sea completamente independiente de la tecnología del backend, cualquier cambio en la implementación del servidor es transparente para la app siempre que el contrato de la API se mantenga.

Spring Boot → Base de datos

Los tres repositorios (LibroRepository, UsuarioRepository, SagaRepository) acceden a la base de datos mediante JDBC sobre JPA/Hibernate. Ninguna otra clase del backend accede directamente a la base de datos, toda consulta o persistencia pasa obligatoriamente por la capa de repositorios.

Spring Boot → Google Books API

LibroService delega en GoogleBooksService las consultas bibliográficas. Este servicio realiza peticiones HTTP a la API de Google Books, parsea la respuesta y devuelve un IsbnResultDto o una lista de libros candidatos. En caso de error (cuota agotada, servicio no disponible), el servicio traduce el código HTTP externo en una excepción con mensaje legible que el controlador propaga al cliente.

Flutter → Modelo (Librería)

Los controladores de Flutter leen y escriben sobre Librería.todosLosLibros, que actúa como caché en memoria de los datos del usuario. Esta caché se actualiza tras cada operación que modifique el

estado del servidor (guardar, eliminar, actualizar). La pantalla de estadísticas evita deliberadamente esta caché y realiza siempre una consulta directa a la API para garantizar que los datos reflejan el estado real de la base de datos.

10.3 Requisitos del entorno

Herramienta	Versión mínima	Necesaria para
Java JDK	17+	Desarrollo backend (modalidad D)
Flutter SDK	3.x	Compilar APK (modalidades A, C, D)
Docker Desktop	Cualquier reciente	Despliegue local (modalidades B, C)
Git	Cualquier reciente	Clonar el repositorio (todas)
MySQL Workbench	Opcional	Gestión BD local
Android Studio / ADB	Opcional	Instalar APK por cable

Java JDK no hace falta para las modalidades B y C ya que Docker descarga y ejecuta el backend internamente sin necesitar Java instalado en el equipo anfitrión. Solo es necesario para desarrollo local (modalidad D).

Para la modalidad A (nube) tampoco hace falta Java ni Docker en local porque Render construye el contenedor en sus servidores a partir del Dockerfile del repositorio. Solo se necesita Flutter SDK para compilar el APK y Git para subir los cambios.

Requisitos no instalables

- Cuenta en Render y cuenta en Neon (modalidad A)
- API key de Google Books (cualquier modalidad con backend activo)
- Dispositivo Android con instalación desde fuentes desconocidas habilitada (modalidades A y C).

10.4 Modalidades de despliegue

10.4.1 Opción A — Despliegue en la nube (Render + Neon)

Esta modalidad permite utilizar la aplicación desde cualquier dispositivo Android con conexión a internet, sin necesidad de mantener ningún servidor activo en local. El backend se ejecuta en Render y la base de datos reside en Neon (PostgreSQL). Es la configuración recomendada para el uso real de la aplicación.

Prerrequisitos

- Cuenta en GitHub con el repositorio del proyecto subido.
- Flutter SDK instalado en el equipo para compilar el APK.
- Dispositivo Android con la opción de Instalar aplicaciones de fuentes desconocidas habilitadas.

Paso 1 – Crear la base de datos en Neon

1. Acceder a neon.tech y crear una cuenta gratuita.
2. Crear un nuevo proyecto. Seleccionar la región más próxima (recomendado: eu-west-2 para España).
3. Una vez creado el proyecto, acceder a la sección Connection Details.
4. En el desplegable de tipo de conexión, seleccionar JDBC y asegurarse de que el hostname no contiene el sufijo -pooler. La URL tendrá el formato: `jdbc:postgresql://ep-<nombre>.eu-west-2.aws.neon.tech/<nombre-bd>?sslmode=require&channel_binding=require`
5. Anotar por separado la URL de conexión, el nombre de usuario y la contraseña. Se necesitarán en el paso siguiente.

Nota: La URL con sufijo -pooler en el hostname utiliza PgBouncer en modo transaction, incompatible con las operaciones DDL de Hibernate. Utilizar siempre el hostname directo.

Paso 2 – Crear el servicio en Render

1. Acceder a render.com y crear una cuenta gratuita.
2. Desde el panel principal, seleccionar New → Web Service.
3. Conectar la cuenta de GitHub y seleccionar el repositorio del proyecto.
4. Configurar los parámetros del servicio:

Campo	Valor
Name	uxilibris-backend (o el nombre deseado)
Root	backend_spring/uxilibris-backend
Runtime	
Instance	Free
Branch	

5. Antes de confirmar el despliegue, acceder a la sección Environment Variables y añadir las siguientes variables:

Variable	Valor
SPRING_DATASOURCE_URL	URL JDBC obtenida de Neon (sin credenciales embebidas)
SPRING_DATASOURCE_USERNAME	Usuario de Neon
SPRING_DATASOURCE_PASSWORD	Contraseña de Neon
SPRING_JPA_HIBERNATE_DDL_AUTO	update
SPRING JACKSON SERIALIZATION WRITE DATES AS TIMESTAMPS	false
GOOGLE_BOOKS_API_KEY	Clave de la API de Google Books

6. Hacer clic en Create Web Service. Render iniciará automáticamente el primer despliegue, que puede tardar entre 3 y 5 minutos mientras construye la imagen Docker.

Nota: A partir de este momento, cualquier git push a la rama configurada disparará un redespiegue automático del backend sin necesidad de intervención manual.

Paso 3 — Verificar el despliegue del backend

1. Una vez que Render muestre el estado Liv en el panel del servicio, comprobar que el backend responde correctamente abriendo la siguiente URL en un navegador:
https://<nombre-servicio>.onrender.com/api/libros
2. La respuesta esperada es un array JSON vacío ([]) o un error de autenticación, nunca un error de conexión. Si el servidor devuelve una página de error de Render, revisar los logs del despliegue en la pestaña Logs del panel.

Comportamiento del plan gratuito: Render apaga el servidor tras 15 minutos de inactividad. La primera petición tras un período de inactividad puede tardar entre 60 y 120 segundos mientras el servidor arranca. La aplicación gestiona este comportamiento de forma automática mediante un mecanismo de reintento, realiza hasta 8 intentos separados por 8 segundos, mostrando un aviso al usuario transcurridos los primeros 8 segundos. No es necesaria ninguna acción por parte del usuario.

Paso 4 – Compilar e instalar el APK

Con el backend activo, compilar la aplicación Flutter apuntando a la URL de Render: `flutter build apk --dart-define=BACKEND_URL=https://<nombre-servicio>.onrender.com`

El APK generado se encontrará en: *build/app/outputs/flutter-apk/app-release.apk*

Para instalarlo en el dispositivo existen dos opciones:

- Opción 1 – Por cable (ADB): con el dispositivo conectado al equipo en modo depuración USB. La compilación e instalación se automatizó y la carpeta scripts/ incluye dos ficheros .bat:
 - `modoA_compilar_apk_nube.bat` → Compila el APK apuntando a `uxilibris-backend.onrender.com` y muestra la ruta del fichero resultante.
 - `modoA_actualizar_apk_nube.bat` → Compila el APK y lo instala directamente en el dispositivo conectado por USB mediante `adb install -r`, conservando los datos de sesión existentes
- Opción 2 – Transferencia manual: copiar el archivo APK al dispositivo (por correo, cable o cualquier medio), abrirlo desde el explorador de archivos del dispositivo y aceptar la instalación.

Si ya existe una versión instalada de la aplicación, la instalación con `-r` (replace) la actualizará conservando los datos de sesión. Si se desinstala primero, se perderán las portadas almacenadas localmente (ver sección 10.8 – Solución de problemas).

10.4.2 Opción B – Despliegue local con Docker

Esta modalidad levanta el sistema completo (base de datos, backend y frontend) en el equipo local mediante Docker. No requiere tener instalado Java, Maven ni MySQL, Docker gestiona todas las dependencias internamente. La aplicación se consume desde el navegador del mismo equipo.

Prerrequisitos

- Docker Desktop instalado y en ejecución.
- Flutter SDK instalado (necesario para el paso de compilación web).
- El repositorio clonado en el equipo local.

Paso 1 – Crear el fichero de variables de entorno

1. El `docker-compose.yml` lee las credenciales de la base de datos desde un fichero `.env` situado en la raíz del proyecto. Este fichero no está incluido en el repositorio por seguridad. Para crearlo, copiar el fichero de ejemplo e introducir una contraseña: `cp .env.example .env`
2. Abrir el fichero `.env` y establecer la contraseña de MySQL:
`MYSQL_ROOT_PASSWORD=contraseña_elegida`
`DB_NAME=UxiLibrisDB`

La contraseña puede ser cualquier cadena, se usará exclusivamente en el entorno local.

Paso 2 – Compilar en frontend web

Antes de levantar los contenedores, es necesario generar los ficheros estáticos de la versión web de la aplicación. El contenedor nginx servirá directamente desde la carpeta de salida.

1. Desde el directorio `frontend_flutter/`, ejecutar: `flutter build web --release`
Esto generará los ficheros en `frontend_flutter/build/web/`.
2. Este paso debe repetirse cada vez que se modifique el código del frontend.

Paso 3 – Levantar los contenedores

1. Desde la raíz del proyecto (donde se encuentra el fichero `Docker-compose.yml`), ejecutar:
`docker-compose up --build`

El flag `--build` indica a Docker que construya la imagen del backend a partir del `Dockerfile`. En el primer arranque, Maven descargará las dependencias de Spring Boot, por lo que este proceso puede tardar varios minutos. Los siguientes arranques serán significativamente más rápidos al reutilizar las capas cacheadas.

El sistema arrancará en el siguiente orden, garantizado por el healthcheck configurado en el `compose`:

- MySQL – espera hasta estar listo para aceptar conexiones.
- Backend Spring Boot – arranca solo tras confirmar que MySQL responde.
- Nginx (frontend) – arranca tras el backend y sirve la aplicación web.

Cuando los tres contenedores estén activos, la terminal mostrará los logs de los tres servicios de forma entrelazada. El backend estará listo cuando aparezca una línea similar a:

backend | Started UxilibrisBackendApplication in X.XXX seconds

Paso 4 – Acceder a la aplicación

1. Abrir un navegador y acceder a: <http://localhost> . La aplicación Flutter web cargará completamente en el navegador. El backend es accesible directamente en: <http://localhost:8080/api>

Nota: la versión web detecta automáticamente que está ejecutándose en un navegador (kIsWeb = true) y utiliza <http://localhost:8080/api> como dirección del backend, No es necesario configurar ninguna URL manualmente.

2. Detener y gestionar los contenedores
 - Detener los contenedores (sin eliminar los datos): [docker-compose down](#)
 - Detener y eliminar todos los datos (la base de datos se borrará): [docker-compose down -v](#)
 - Levantar sin reconstruir la imagen (cuando solo ha cambiado el frontend): [docker-compose up](#)

Los datos de la base de datos persisten entre reinicios gracias al volumen `nysql_data`, por lo que detener los contenedores con `docker-compose down` no implica pérdida de información.

10.4.3 Opción C – APK con backend local

Esta modalidad permite usar la aplicación en un dispositivo real mientras el backend se ejecuta en el equipo local. El dispositivo y el equipo deben estar conectados a la misma red WiFi. Es útil para pruebas con datos reales sin depender del servidor en la nube.

Prerrequisitos

- Docker Desktop en ejecución, o un IDE con soporte para Spring Boot (IntelliJ IDEA, VS Code con extensión Java).
- Flutter SDK instalado.
- Dispositivo Android con Instalar aplicaciones de fuentes desconocidas habilitado, conectado a la misma red WiFi que el equipo.

Paso 1 – Obtener la IP local del equipo

El dispositivo Android necesita conocer la dirección IP del equipo en la red local para conectarse al backend. En Windows, abrir un terminal y ejecutar: `ipconfig`

Localizar la sección correspondiente al adaptador WiFi activo y anotar el valor de Dirección IPv4. Tendrá un formato similar a 192.168.1.X.

Paso 2 – Levantar el backend

- Opción 2a – Con Docker (recomendado, no requiere Java instalado):

Desde la raíz del proyecto, levantar únicamente los servicios de base de datos y backend: `docker-compose up db backend`

No es necesario levantar el servicio frontend (nginx) en esta modalidad, ya que la interfaz será el APK.

- Opción 2b – Desde el IDE (para desarrollo).

Abrir el proyecto `backend_spring/uxilibris-backend` en IntelliJ IDEA o VS Code y ejecutar la clase `UxilibrisBackendApplication`. Es necesario tener configurado el fichero `application.properties` con los datos de la base de datos local.

En ambos casos, verificar que el backend responde antes de continuar accediendo desde el navegador del equipo a: `http://localhost:8080/api/libros`

Paso 3 – Compilar el APK apuntado a la IP local

Desde el directorio `frontend_flutter/`, ejecutar el comando de compilación sustituyendo 192.168.1.X por la IP obtenida en el Paso 1: `flutter build apk --dart-define=BACKEND_URL=http://192.168.1.X:8080`

Nota: La aplicación ya tiene configurado `android:usesCleartextTraffic="true"` en el `AndroidManifest.xml`, por lo que las conexiones HTTP a direcciones locales están permitidas sin configuración adicional.

El APK generado se encontrará en: `build/app/outputs/flutter-apk/app-release.apk`

Paso 4 – Instalar el APK en el dispositivo

- Opción 4a – Por cable (ADB): con el dispositivo conectado al equipo en modo depuración USB `adb install -r build\app\outputs\flutter-apk\app-release.apk`
- Opción 4b – Transferencia manual: copiar el APK al dispositivo por cualquier medio (cable, correo, aplicación de transferencia de archivos), abrirlo desde el explorador de archivos del dispositivo y aceptar la instalación.

Consideraciones de uso

El dispositivo debe permanecer en la misma red WiFi que el equipo mientras usa la aplicación. Si cambia de red, perderá la conexión con el backend.

Si el equipo se reinicia o la IP local cambia (por reasignación del router), será necesario recompilar el APK con la nueva IP.

Al detener los contenedores Docker con Docker-compose down, los datos de la base de datos se conservan gracias al volumen persistente mysql_data. Al levantar de nuevo el compose, los datos estarán disponibles.

10.4.4 Opción D – Entorno de desarrollo

Esta modalidad está orientada a quien quiera modificar, depurar o ampliar el código del proyecto. El backend se ejecuta directamente desde el IDE o con el Maven Wrapper incluido en el repositorio, y el frontend se lanza con flutter run sobre un dispositivo o emulador conectado. No se utiliza Docker.

Prerrequisitos

- Git instalado.
- Java JDK 21 instalado y configurado en el PATH.
- Flutter SDK 3.10.4 o superior instalado y configurado en el PATH.
- MySQL 8 instalado y en ejecución en local.
- IDE recomendado: IntelliJ IDEA (backend) + VS Code con extensión Flutter (frontend).

Paso 1 – Clonar el repositorio

`git clone https://github.com/<usuario>/uxilibris.git`

`cd uxilibris`

Paso 2 – Configurar el backend

El fichero application.properties está excluido del repositorio mediante .gitignore porque contiene credenciales y la API key de Google Books. Es necesario crearlos manualmente antes de lanzar el backend.

Crear el fichero en la ruta `backend_spring/uxilibris-backend/src/main/resources/application.properties` con el contenido incluido la [Sección 10.5.1](#)

Nota: con `ddl-auto=update` Hibernate creará automáticamente las tablas en la base de datos UxiLibrisDB al arrancar por primera vez. No es necesario ejecutar ningún script SQL previo.

Nota: el parámetro `createDatabaseIfNotExists=true` en la URL hace que MySQL cree la base de datos si aún no existe. El usuario configurado debe tener permisos suficientes para ello.

Paso 3 – Instalar dependencias del frontend

Desde el directorio `frontend_flutter/`, instalar los paquetes de Flutter declarados en `pubspec.yaml`
`flutter pub get`

Paso 4 – Levantar el backend**Opción 4a – Con el Maven Wrapper** (sin necesidad de Maven instalado globalmente)

- `cd backend_spring/uxilibris-backend`
- `./mvnw spring-boot:run` # Linux / macOS
- `mvnw.cmd spring-boot:run` # Windows

Opción 4b – Desde IntelliJ IDEA

Abrir el proyecto `backend_spring/uxilibris-backend` como proyecto Maven. IntelliJ detectará automáticamente de la clase principal `UxilibrisBackendApplication`. Ejecutarla con el botón Run o con Shift+F10.

El backend estará listo cuando el log muestre una línea similar a:

Started UxilibrisBackendApplication in X.XXX seconds (process running for X.XXX)

Paso 5 – Lanzar el frontend

Con el backend en ejecución, lanzar la aplicación Flutter sobre un dispositivo físico o emulador Android conectado

`cd frontend_flutter`

`flutter run`

Si hay varios dispositivos disponibles, Flutter mostrará un selector. El frontend detecta automáticamente que no se ha definido `BACKEND_URL` en tiempo de compilación y utiliza la dirección local configurada por defecto en `ApiService`:

// Dispositivo físico → IP del equipo en la red local

`return 'http://192.168.1.200:8080/api';`

// Emulador Android → descomentar esta línea en `ApiService.dart`

`// return 'http://10.0.2.2:8080/api';`

Si se usa un emulador en lugar de un dispositivo físico, editar `ApiService.dart` y sustituir la URL por `http://10.0.2.2:8080/api` antes de lanzar.

Resumen de Comandos

Acción	Comando
Instalar dependencias Flutter	<code>flutter pub get</code>
Arrancar backend (Maven Wrapper)	<code>mvnw.cmd spring-boot:run</code>
Arrancar frontend en dispositivo	<code>flutter run</code>
Compilar APK de desarrollo	<code>flutter build apk</code>
Compilar APK apuntando a Render	<code>flutter build apk</code> <code>--dart-define=BACKEND_URL=https://<servicio>.onrender.com</code>
Ejecutar tests backend	<code>mvnw.cmd test</code>

10.5 Configuración del backend

El comportamiento del backend se controla mediante el fichero `application.properties`. Este fichero no está incluido en el repositorio porque contiene credenciales y claves de acceso a servicios externos. Dependiendo del entorno de despliegue, la configuración se aporta de formas distintas: mediante el fichero físico en desarrollo local, o mediante variables de entorno en producción.

10.5.1 Estructura del fichero `application.properties`

A continuación, se muestra la estructura completa del fichero con valores de ejemplo. Los campos marcados con `<valor>` deben sustituirse por los datos reales del entorno:

```
spring.application.name=uxilibris-backend
# Puerto de escucha. Lee la variable PORT del entorno si existe; si no, usa 8080.
# En Render el valor lo asigna la plataforma dinámicamente.
server.port=${PORT:8080}
server.address=0.0.0.0
# — Base de datos —
# Desarrollo local con MySQL:
spring.datasource.url=jdbc:mysql://localhost:3306/UxiLibrisDB?createDatabaseIfNotExists=true
# Producción con Neon (PostgreSQL):
# spring.datasource.url=jdbc:postgresql://ep-<nombre>.region.neon.tech/<bd>?sslmode=require
spring.datasource.username=<usuario_bd>
spring.datasource.password=<contraseña_bd>
# — JPA / Hibernate —
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.open-in-view=false
# — Jackson —
# Serializa LocalDate como "2024-01-15" en lugar de [2024,1,15]
spring.jackson.serialization.write-dates-as-timestamps=false
# — Google Books API —
google.books.api.key=<api_key_google_books>
```

Importante: este fichero nunca debe subirse al repositorio. Está incluido en `.gitignore`. Si se añade accidentalmente, eliminarlo del índice con `git rm --cached application.properties`.

10.5.2 Variables de entorno equivalentes para producción

En el entorno de producción (Render), el fichero `application.properties` no está disponible. Spring Boot permite sobrescribir cualquier propiedad mediante variables de entorno siguiendo la convención de convertir los puntos en guiones bajos y pasar todo a mayúsculas. Las variables que deben configurarse en el panel de Render son las siguientes:

Variable de entorno	Equivalente en properties	Valor de ejemplo
SPRING_DATASOURCE_URL	spring.datasource.url	jdbc:postgresql://ep- <nombre>.region.neon.tech/<bd>? sslmode=require
SPRING_DATASOURCE_USERNAME	spring.datasource.username	neondb_owner
SPRING_DATASOURCE_PASSWORD	spring.datasource.password	<contraseña_neon>
SPRING_JPA_HIBERNATE_DDL_AUTO	spring.jpa.hibernate.ddl-auto	update
SPRING_JACKSON_SERIALIZATION_WRITE_DATES_AS_TIMESTAMPS	spring.jackson.serialization.write-dates-as-timestamps	false
GOOGLE_BOOKS_API_KEY	google.books.api.key	<api_key_google_books>

Nota sobre la URL de Neon: utilizar siempre el hostname directo, sin el sufijo -pooler. La variante con pooler usa PgBouncer en modo transaction, incompatible con las operaciones DDL de Hibernate que crean y actualizan el esquema de la base de datos.

10.5.3 La propiedad ddl-auto y por qué se usa update

La propiedad spring.jpa.hibernate.ddl-auto controla qué hace Hibernate con el esquema de la base de datos al arrancar la aplicación. Los valores posibles más relevantes son:

Valor	Comportamiento
none	No toca el esquema. Las tablas deben existir previamente.
validate	Comprueba que el esquema coincide con las entidades, pero no lo modifica.
update	Crea las tablas que faltan y añade columnas nuevas. No elimina nada.
create	Recrea todas las tablas en cada arranque. Borra todos los datos.
create-drop	Como create, pero además elimina las tablas al apagar el servidor.

En UxiLibris se usa update por las siguientes razones:

- No requiere scripts SQL manuales. Al desplegar por primera vez, Hibernate crea automáticamente todas las tablas a partir de las anotaciones JPA de las entidades (@Entity, @Column, @OneToMany, etc.). No es necesario ejecutar ningún CREATE TABLE previo.
- Permite evolucionar el modelo sin perder datos. Si se añade un nuevo campo a una entidad, Hibernate añade la columna correspondiente en el siguiente arranque sin borrar los registros existentes.
- Comportamiento seguro. A diferencia de create, el valor update nunca elimina tablas ni columnas, por lo que los datos están protegidos ante reinicios del servidor.

Nota: update no gestiona migraciones complejas como renombrar columnas o cambiar tipos de datos. Para proyectos en producción con esquemas más elaborados se recomienda complementarlo con una herramienta de migraciones como Flyway o Liquibase. En el alcance de este proyecto, update es suficiente.

10.5.4 Obtener y configurar la API key de Google Books

La API de Google Books tiene una cuota pública compartida que se agota con facilidad si varios usuarios o aplicaciones la utilizan sin identificarse. Registrar una clave propia proporciona una cuota de 1.000 consultas gratuitas por día asociadas al proyecto.

Pasos para obtener la clave:

1. Acceder a console.cloud.google.com con una cuenta de Google.
2. Crear un nuevo proyecto o seleccionar uno existente.
3. En el menú lateral, navegar a APIs y servicios → Biblioteca.
4. Buscar Books API y hacer clic en Habilitar.
5. Ir a APIs y servicios → Credenciales y hacer clic en Crear credenciales → Clave de API.
6. Copiar la clave generada.
7. Configuración en desarrollo local: Añadir la clave al fichero `application.properties:google.books.api.key=AIza...`
8. Configuración en producción (Render): Añadir la variable de entorno `GOOGLE_BOOKS_API_KEY` con el valor de la clave en el panel de configuración del servicio en Render.

Nota de seguridad: la clave de API nunca debe incluirse en el código fuente ni subirse al repositorio. El backend actúa como intermediario entre la aplicación móvil y Google Books precisamente para que la clave permanezca en el servidor y no quede expuesta en el APK distribuido.

10.6 Compilación del APK

La aplicación UxiLibris se distribuye como un fichero APK que puede instalarse directamente en cualquier dispositivo Android. La URL del backend queda fijada en tiempo de compilación, por lo que es necesario indicarla explícitamente al construir el APK.

Prerrequisitos

- Flutter SDK instalado y en el PATH.
- Dependencias instaladas (`flutter pub get` ejecutado al menos una vez en `frontend_flutter/`).
- Conexión a internet durante la compilación (Flutter descarga herramientas de construcción en el primer uso).

Paso 1 – Comando base

Desde el directorio `frontend_flutter/`, ejecutar: `flutter build apk`

Este comando genera un APK en modo release (optimizado, sin información de depuración) apuntando a la dirección por defecto definida en `ApiService` para dispositivos físicos

(http://192.168.1.200:8080/api). Es el comando apropiado para la modalidad de desarrollo local (opción C) si la IP del equipo coincide con ese valor.

Paso 2 – El parámetro `--dart-define=BACKEND_URL`

Para compilar un APK que apunte a un backend distinto al definido por defecto, se utiliza el parámetro `--dart-define`. Este mecanismo inyecta constantes en el código Dart en tiempo de compilación, de forma que el valor queda integrado directamente en el binario.

La sintaxis general es: `flutter build apk --dart-define=BACKEND_URL=<url_del_backend>`

Ejemplos según modalidad:

- Apuntando al backend en la nube (modalidad A): `flutter build apk --dart-define=BACKEND_URL=https://uxilibris-backend.onrender.com`
- Apuntando a un backend Docker en la red local (modalidad C), con la IP del equipo: `flutter build apk --dart-define=BACKEND_URL=http://192.168.1.X:8080`

Internamente, ApiService lee este valor de la siguiente forma:

```
static const String _backendUrl =  
    String.fromEnvironment('BACKEND_URL', defaultValue: "");  
static String get baseUrl {  
    if (_backendUrl.isNotEmpty) return '$_backendUrl/api';  
    if (kIsWeb) return 'http://localhost:8080/api';  
    return 'http://192.168.1.200:8080/api'; // fallback dispositivo físico  
}
```

Si no se proporciona `--dart-define=BACKEND_URL`, `String.fromEnvironment` devuelve la cadena vacía y el código cae en el fallback correspondiente al entorno detectado.

Importante: la URL del backend queda fijada permanentemente en el APK en el momento de la compilación. Si el backend cambia de dirección, es necesario recompilar e instalar un nuevo APK.

Paso 3 – Donde encontrar el APK generado

Una vez finalizada la compilación, el APK se encuentra en la siguiente ruta relativa al directorio `frontend_flutter/`: `build/app/outputs/flutter-apk/app-release.apk`

La compilación puede tardar entre 1 y 3 minutos dependiendo del equipo. Al finalizar correctamente, Flutter muestra en la terminal la ruta al fichero y el tamaño del APK generado:

✓ *Built build/app/outputs/flutter-apk/app-release.apk (XX.X MB)*

Paso 4 – Instalación en el dispositivo**Opción 4a — Por cable mediante ADB (recomendado para desarrollo):**

Con el dispositivo Android conectado al equipo mediante cable USB y la depuración USB habilitada en las opciones de desarrollador del dispositivo, ejecutar:

```
adb install -r build\app\outputs\flutter-apk\app-release.apk
```

El flag -r (replace) actualiza la aplicación si ya existe una versión instalada, conservando los datos de sesión del usuario. Si se omite y la aplicación ya está instalada, ADB devuelve un error.

Para comprobar que el dispositivo es reconocido correctamente antes de instalar: [adb devices](#)

Opción 4b – Transferencia manual:

Copiar el fichero app-release.apk al dispositivo por cualquier medio (cable USB en modo transferencia de archivos, correo electrónico, aplicación de transferencia inalámbrica, etc.). Una vez en el dispositivo, abrir el fichero desde el explorador de archivos y aceptar la instalación.

Para que Android permita instalar APKs externos, el dispositivo debe tener habilitada la opción Instalar aplicaciones de fuentes desconocidas. En Android 8 o superior esta opción se activa por aplicación: al intentar instalar el APK, el sistema mostrará automáticamente el diálogo correspondiente si no está ya habilitada.

Nota sobre reinstalación: desinstalar la aplicación antes de instalar una nueva versión elimina el directorio privado de la app, incluyendo las portadas de libros almacenadas localmente. Para preservar los datos, instalar siempre encima de la versión existente en lugar de desinstalar primero.

10.7 Verificación del sistema

Una vez completado el despliegue en cualquiera de las modalidades descritas, se deben seguir la siguiente lista de comprobaciones para confirmar que todos los componentes funcionan correctamente.

10.7.1 El backend responde correctamente

Abrir un navegador y acceder al endpoint de libros del backend. La URL varía según la modalidad de despliegue.

Modalidad	URL de verificación
A — Nube (Render)	<a href="https://<nombre-servicio>.onrender.com/api/libros">https://<nombre-servicio>.onrender.com/api/libros
B / C — Docker local	http://localhost:8080/api/libros
D — Desarrollo	http://localhost:8080/api/libros

Respuesta esperada: un array JSON vacío ([]) si no hay libros registrados, o un array con objetos JSON si los hay. Cualquier respuesta JSON válida indica que el backend está activo y la base de datos es accesible.

Respuestas que indican un problema:

Respuesta	Causa probable
Página de error de Render / sin respuesta	El servidor está arrancando (esperar 60 s) o hay un fallo en el despliegue
relation "libros" does not exist	SPRING_JPA_HIBERNATE_DDL_AUTO no está configurado como update
Connection refused	El backend no está en ejecución o el puerto es incorrecto
Error de SSL / certificado	La URL usa http en lugar de https en la modalidad nube

Modalidad A — plan gratuito de Render: si el servidor lleva más de 15 minutos sin recibir peticiones, el primer acceso puede tardar hasta 60 segundos mientras arranca. Si transcurrido ese tiempo el navegador sigue sin responder, revisar los logs en el panel de Render (pestaña Logs).

10.7.2 Registro e inicio de sesión

1. Abrir la aplicación (APK o versión web según la modalidad).
2. Desde la pantalla de inicio de sesión, acceder al formulario de registro.
3. Introducir un nombre de usuario, correo electrónico y contraseña y confirmar el registro.
4. Verificar que la aplicación redirige automáticamente a la colección (pantalla principal) sin mostrar ningún error.
5. Cerrar la aplicación y volver a abrirla. Verificar que la sesión se mantiene activa y no es necesario iniciar sesión de nuevo.

Si el registro falla, los motivos más frecuentes son que el backend no está accesible (verificar el paso 10.7.1) o que el nombre de usuario o el correo ya existen en la base de datos.

10.7.3 Añadir y listar libros

1. Desde la pantalla de colección, añadir un libro manualmente mediante el botón de nuevo libro. Rellenar al menos el título y guardar.
2. Verificar que el libro aparece en la lista de la colección tras guardar.
3. Cerrar la aplicación completamente y volver a abrirla. Verificar que el libro persiste (se ha guardado en el servidor, no solo en memoria).
4. Editar el libro añadido: cambiar algún campo y guardar. Verificar que el cambio se refleja en la lista.

5. Eliminar el libro y verificar que desaparece de la colección.

10.7.4 Portadas de Google Books

Este paso verifica que el backend puede comunicarse con la API de Google Books y que la clave de API está correctamente configurada.

1. Desde la pantalla de nuevo libro, seleccionar la opción de búsqueda por título o autor.
2. Introducir el nombre de un libro conocido (por ejemplo, «El nombre del viento») y lanzar la búsqueda.
3. Verificar que aparece una lista de resultados con portadas visibles. Si las portadas no cargan pero los títulos sí, la conexión con Google Books funciona pero las URLs de imagen no son accesibles (problema de red del dispositivo, no del backend).
4. Seleccionar un resultado de la lista y verificar que el formulario se rellena automáticamente con el título, autor y portada del libro seleccionado.
5. Guardar el libro y verificar que la portada se muestra correctamente en la tarjeta de la colección.

Si la búsqueda no devuelve resultados o devuelve un error:

Síntoma	Causa probable
Sin resultados, sin error	La API key no está configurada y la cuota pública está agotada
Error «Cuota de Google Books agotada»	La variable <code>GOOGLE_BOOKS_API_KEY</code> no está definida o la cuota diaria se ha consumido
Error de conexión	El backend no puede acceder a internet (solo puede ocurrir en despliegues locales sin salida a internet)

10.8 Solución de problemas comunes

Esta sección recoge los problemas más frecuentes durante la instalación y puesta en marcha del sistema, junto con sus causas y los pasos necesarios para resolverlos.

10.8.1 El servidor en Render no responde (arranque en frío)

Síntoma: la aplicación se queda cargando durante varios segundos al iniciar sesión o al abrir la colección de libros, y finalmente muestra un error de conexión o no carga ningún dato.

Causa: el plan gratuito de Render apaga automáticamente el servidor tras 15 minutos de inactividad. El primer acceso tras este período requiere que Render arranque el contenedor desde cero, proceso que puede tardar entre 30 y 60 segundos.

Solución: Abrir la URL del backend directamente en el navegador del dispositivo o del equipo y esperar a que devuelva una respuesta: `https://<nombre-servicio>.onrender.com/api/libros`

Una vez que el navegador muestra una respuesta JSON (aunque sea []), el servidor está activo. Abrir la aplicación y volver a intentar la operación.

La aplicación ya tiene configurado un timeout de 90 segundos en las llamadas HTTP para tolerar este arranque. Si el error persiste pasado ese tiempo, revisar los logs en el panel de Render.

10.8.2 Las tablas no se crean en la base de datos

Síntoma: el backend arranca correctamente pero cualquier operación devuelve un error similar a *PSQLException: relation "usuarios" does not exist* o *Table 'UxiLibrisDB.libros' doesn't exist*.

Causa: la propiedad `spring.jpa.hibernate.ddl-auto` tiene el valor por defecto `none` cuando se usa PostgreSQL sin `application.properties`. Hibernate no intenta crear ni modificar el esquema y las consultas fallan porque las tablas no existen.

Solución: añadir o verificar la siguiente variable de entorno en el panel de Render: `SPRING_JPA_HIBERNATE_DDL_AUTO = update`

Tras guardar la variable, Render redesplegará el servicio automáticamente. En el siguiente arranque, Hibernate creará todas las tablas necesarias.

10.8.3 Error de dialecto o conexión con Neon

Síntoma: el log de Render muestra *HibernateException: Unable to determine Dialect* o *PSQLException: FATAL: password authentication failed*.

Causa más frecuente A - URL malformada: la URL de `SPRING_DATASOURCE_URL` contiene credenciales embebidas (`usuario:contraseña@host`) o tiene el parámetro `?sslmode=require` duplicado.

Solución A: usar la URL sin credenciales y sin parámetros duplicados, y separar usuario y contraseña en sus propias variables:

```
SPRING_DATASOURCE_URL      = jdbc:postgresql://ep-<nombre>.region.neon.tech/<bd>?  
sslmode=require
```

```
SPRING_DATASOURCE_USERNAME = <usuario_neon>
```

```
SPRING_DATASOURCE_PASSWORD = <contraseña_neon>
```

Causa B - Hostname con pooler: la URL incluye el sufijo `-pooler` en el hostname (por ejemplo, `ep-nombre-pooler.region.neon.tech`). PgBouncer en modo `transaction` impide que Hibernate ejecute las sentencias DDL de creación de tablas.

Solución B: eliminar el sufijo `-pooler` del hostname, usando el hostname directo:

Incorrecto:

```
jdbc:postgresql://ep-nombre-pooler.eu-west-2.aws.neon.tech/neondb?sslmode=require
```

Correcto:

`jdbc:postgresql://ep-nombre.eu-west-2.aws.neon.tech/neondb?sslmode=require`

10.8.4 La aplicación no encuentra el backend (IP incorrecta)

Síntoma: la aplicación muestra un error de conexión inmediatamente al iniciar sesión o al cargar los libros, sin período de espera.

Causa: el APK fue compilado con una URL de backend (`--dart-define=BACKEND_URL`) que ya no es válida, bien porque la IP local del equipo ha cambiado (reasignación por el router), bien porque el APK apunta a localhost en lugar de a la IP real de la red local.

Diagnóstico: verificar la URL con la que fue compilado el APK comprobando si el backend responde desde el propio dispositivo. Abrir el navegador del dispositivo y acceder a: `http://<ip_del_equipo>:8080/api/libros`. Si el navegador no responde pero sí lo hace desde el equipo, el problema es la IP.

Solución: obtener la IP actual del equipo con `ipconfig` (Windows) y recompilar el APK con la nueva dirección: `flutter build apk --dart-define=BACKEND_URL=http://192.168.1.X:8080`

Instalar el nuevo APK sobre el existente con `adb install -r` o por transferencia manual.

10.8.5 Render no detecta el puerto ("No open ports detected")

Síntoma: el log de Render muestra *No open ports detected* y el contenedor se detiene con código de salida 1 poco después de arrancar.

Causa: Spring Boot está configurado para escuchar siempre en el puerto 8080, pero Render asigna el puerto de forma dinámica en cada despliegue mediante la variable de entorno `PORT`. Al no coincidir, Render considera que no hay ningún proceso activo.

Solución: verificar que el Dockerfile del backend usa la forma `shell` en el `ENTRYPOINT` para que `{PORT}` se resuelva en tiempo de ejecución:

```
ENTRYPOINT ["sh", "-c", "java -jar app.jar --server.port=${PORT:-8080}"]
```

Tras corregir el Dockerfile, hacer `git push` para que Render redespliegue con la nueva configuración.

10.8.6 Las portadas de los libros no se muestran

Síntoma A: las portadas de libros buscados por ISBN o texto no se muestran; en su lugar aparece la imagen por defecto.

Causa: la cuota diaria de la API de Google Books está agotada (límite de 1.000 consultas por día sin clave propia) o la clave `GOOGLE_BOOKS_API_KEY` no está configurada.

Solución: configurar una clave de API propia siguiendo los pasos del apartado 5.4 de este manual y añadirla como variable de entorno en Render o en application.properties.

Síntoma B: los libros muestran la imagen por defecto tras reinstalar la aplicación, aunque anteriormente tenían portadas asignadas.

Causa: las portadas seleccionadas manualmente desde la galería del dispositivo se almacenaban en el directorio privado de la aplicación, que Android elimina al desinstalar. Las rutas almacenadas en la base de datos ya no apuntan a ningún fichero.

Solución: no es posible recuperar las imágenes eliminadas. Es necesario reasignar las portadas afectadas editando cada libro. Las portadas procedentes de Google Books (URLs remotas) no se ven afectadas por este problema y se recuperan automáticamente.

11. Anexo II – Documentación de uso (manuales de usuario)

11.1 Introducción

11.1.1 ¿Qué es UxiLibris?

UxiLibris es una aplicación de gestión de biblioteca personal que permite al usuario llevar un registro organizado de sus lecturas. Con ella es posible catalogar libros, registrar cuándo se empezó y terminó cada lectura, puntuar las obras, agruparlas en sagas, marcar favoritos y consultar estadísticas de hábitos de lectura a lo largo del tiempo.

La aplicación está pensada para lectores que quieren tener en un único lugar toda la información sobre sus libros, sin depender de servicios externos de terceros ni de hojas de cálculo. Los datos de cada usuario quedan almacenados en una cuenta propia y son accesibles desde cualquier dispositivo con conexión a internet.

11.1.2 Plataformas disponibles

UxiLibris está disponible en dos formatos:

Aplicación web

Accesible desde cualquier navegador moderno (Chrome, Firefox, Edge, Safari) sin necesidad de instalación. Basta con introducir la dirección del servidor en la barra del navegador e iniciar sesión. Es la opción recomendada para usar la aplicación desde un ordenador de sobremesa o portátil.

Aplicación Android (APK)

Disponible como archivo de instalación .apk para dispositivos Android. Ofrece la misma funcionalidad que la versión web y, además, permite escanear el código de barras (ISBN) de un libro físico mediante la cámara del dispositivo para añadirlo automáticamente a la biblioteca. La instalación se realiza de forma manual desde el archivo descargado (no está disponible en Google Play).

Nota: Ambas versiones comparten la misma cuenta y los mismos datos. Los cambios realizados desde una plataforma son visibles inmediatamente desde la otra.

11.1.3 Requisitos del usuario

No es necesario instalar ningún software adicional para usar UxiLibris. El único requisito es disponer de una cuenta registrada en la aplicación.

El proceso de registro es gratuito y solo requiere:

- Un nombre de usuario
- Una dirección de correo electrónico
- Una contraseña

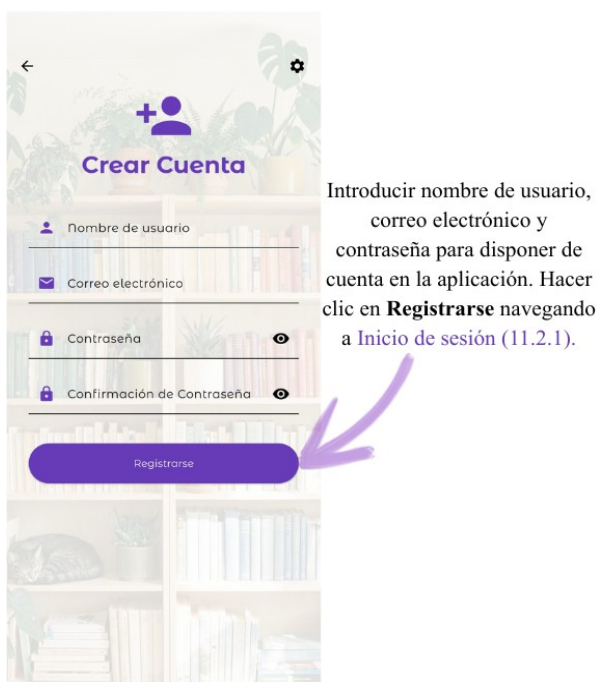
Una vez registrada la cuenta, el usuario puede iniciar sesión desde cualquier dispositivo y acceder a su biblioteca personal completa.

11.2 Primeros pasos

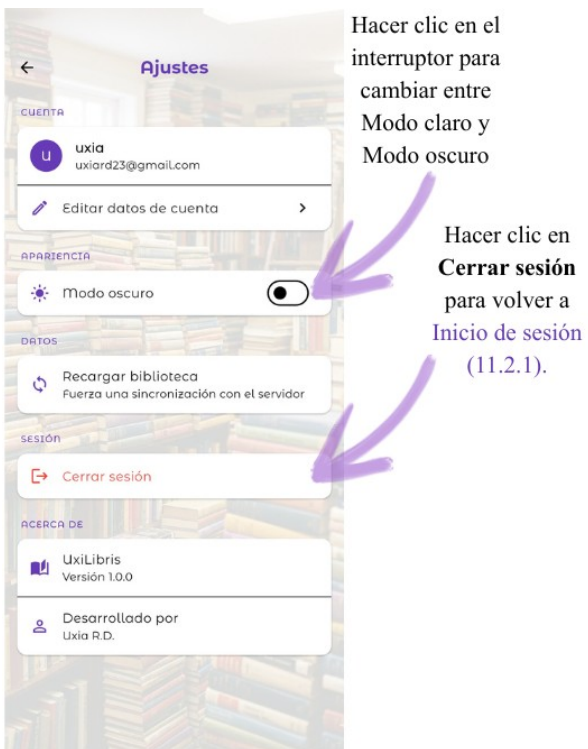
11.2.1 Inicio de sesión



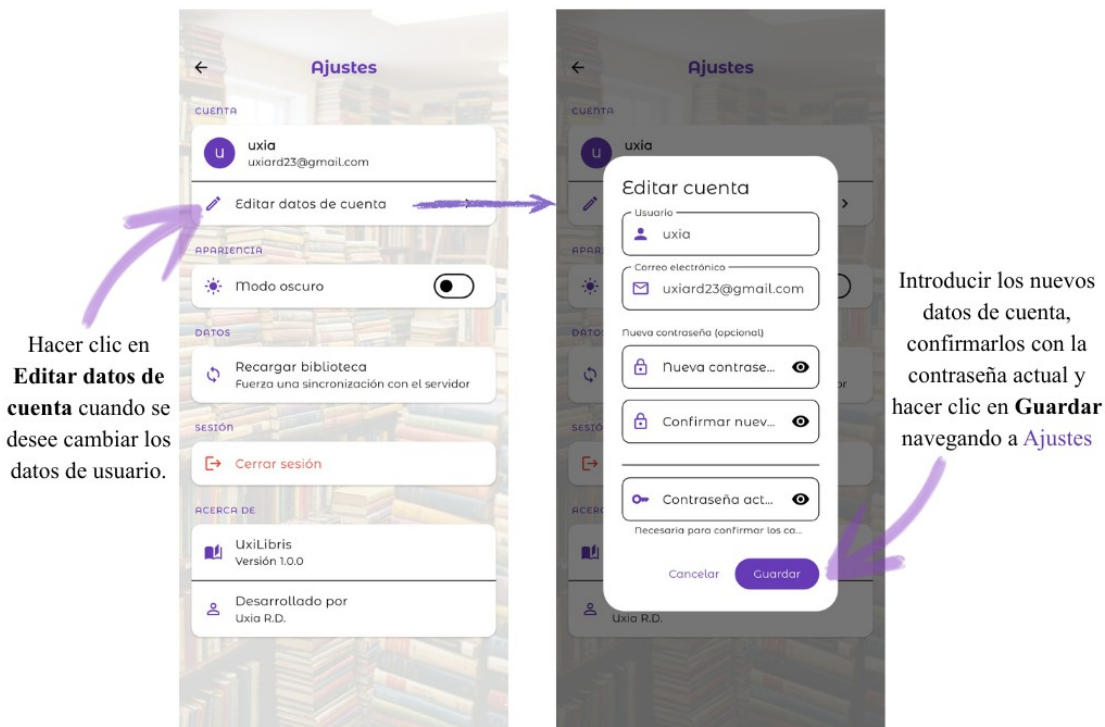
11.2.2 Registro de cuenta



11.2.3 Cierre de sesión y cambio de modo de visualización

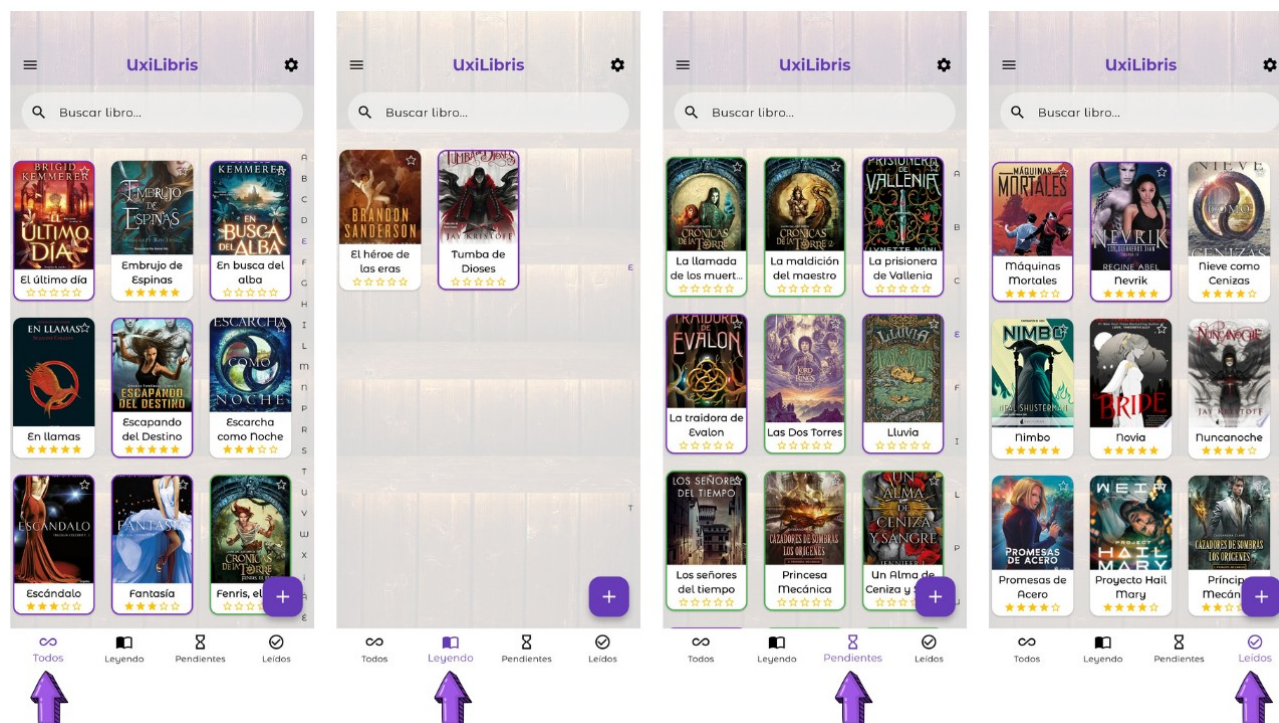


11.2.4 Modificación datos de la cuenta (username, email, contraseña)



11.3 La biblioteca

11.3.1 Vista general (tarjetas de libros)



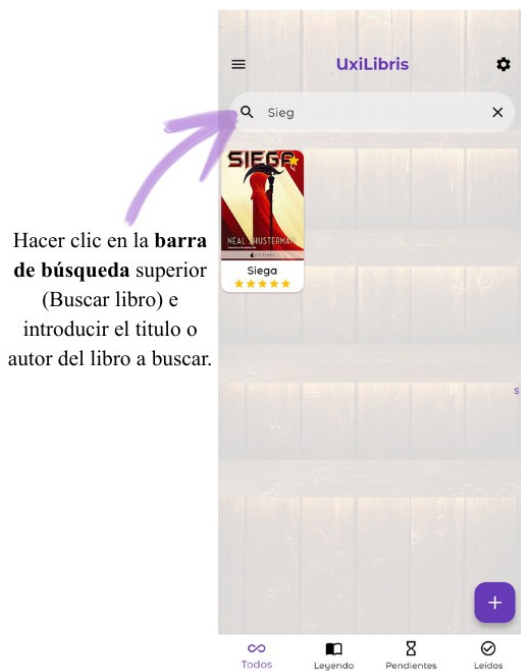
Pantalla principal en la que se pueden ver los libros registrados en la cuenta, visualizar su estado, buscar por título o autor, acceder a los datos de un libro, o añadir un nuevo libro haciendo clic en el botón + (abajo, derecha) navegando a Gestión de libros (11.4).

En la barra superior de la pantalla se accede al menú principal (izquierda) y a los ajustes (derecha).

Los bordes de las tarjetas aportan la siguiente información:

- Sin borde: libro físico
- Borde violeta: libro digital
- Borde verde: libro físico cuyo estado es pendiente

11.3.2 Filtros y búsqueda

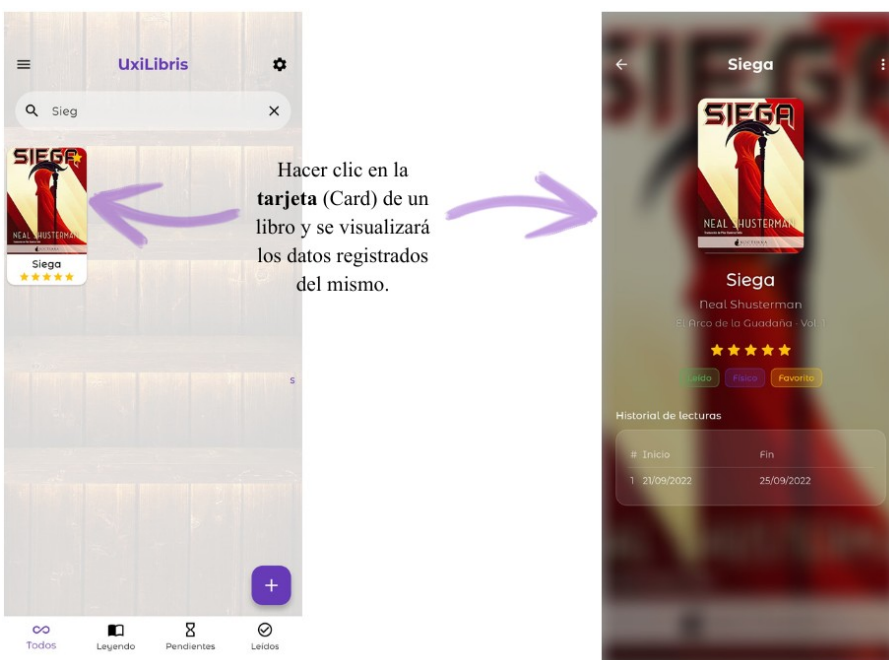


11.3.3 Estados de un libro (Pendiente, Leyendo, Leído)



Hacer clic en el estado del libro por el que se desea filtrar su biblioteca.

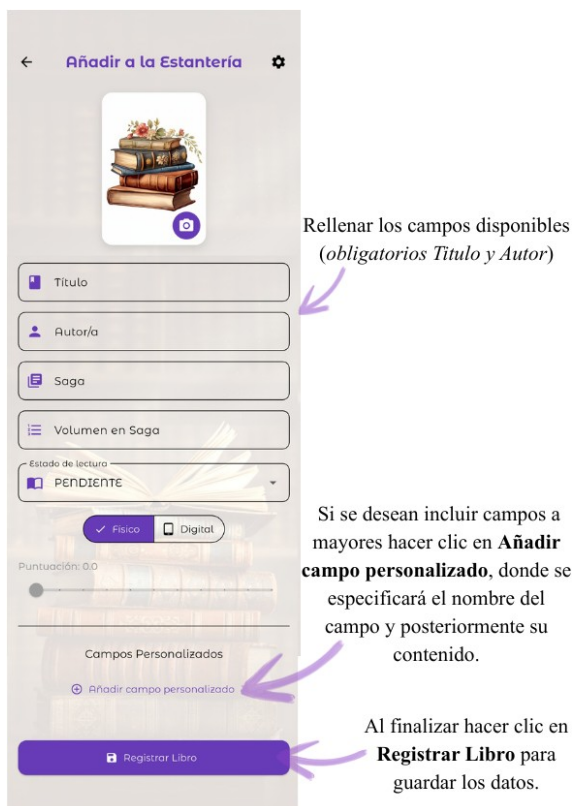
11.3.4 Detalle Libro



11.4 Gestión de libros

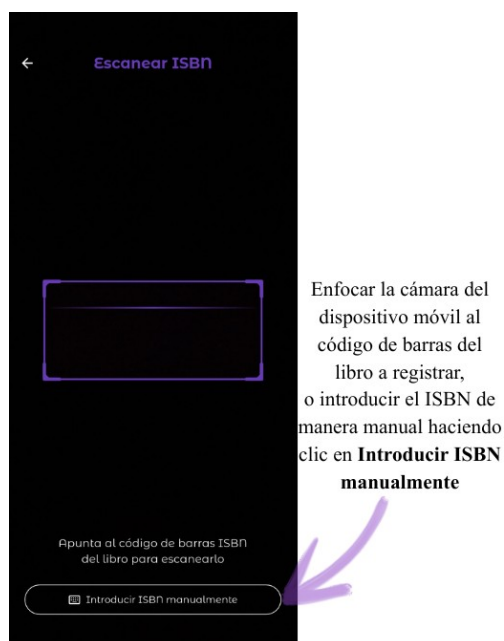


11.4.1 Añadir un libro manualmente



El campo de portada del libro selecciona el fichero del sistema local del dispositivo en el que se está ejecutando la aplicación, por lo que no se conserva si se carga en otro dispositivo.

11.4.2 Añadir un libro por ISBN (escáner)



Enfocar la cámara del dispositivo móvil al código de barras del libro a registrar, o introducir el ISBN de manera manual haciendo clic en **Introducir ISBN manualmente**

La API de Google Books no siempre completa todos los campos por lo que después de la búsqueda la aplicación navega Añadir un libro manualmente (11.4.1) con los campos que ha obtenido cubiertos, pero ofrece la posibilidad de editarlos y completar aquellos que no se han recuperado.

Función no disponible en formato web.

11.4.3 Buscar un libro en Google Books

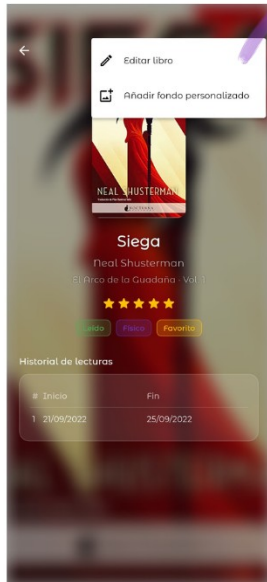


Introducir el título o el autor del libro y hacer clic en **Buscar**.

Las limitaciones de la API de Google Books hacen que los autores se localicen mejor si solo se incluye una palabra de su nombre completo. Los títulos de los libros se localizan mejor en inglés ya que es el idioma en el que están la mayoría de los libros de la API.

11.4.4 Editar un libro

En **Detalle libro** (11.3.4) hacer clic en los **3 puntos** (derecha) de la barra superior, hacer clic en **Editar libro** el cual navega al formulario de **Añadir un libro manualmente** (11.4.1) pero con los campos cubiertos del libro a editar.



Hacer clic en el **campo** a modificar e introducir el nuevo valor,

Hacer clic en el **campo personalizado** para editarlo

Hacer clic en el **icono de lapiz** para editar el nombre del campo personalizado y en el **icono de papelera** para eliminarlo

Por último, hacer clic en **Actualizar Libro**.

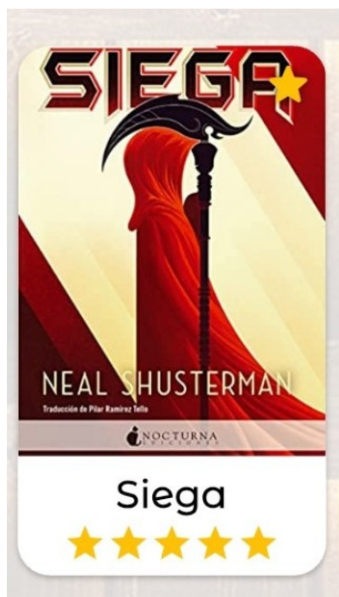
11.4.5 Eliminar un libro

En **Editar un libro** (11.4.4) en la barra superior (derecha) hacer clic en el **icono de la papelera** (rojo)



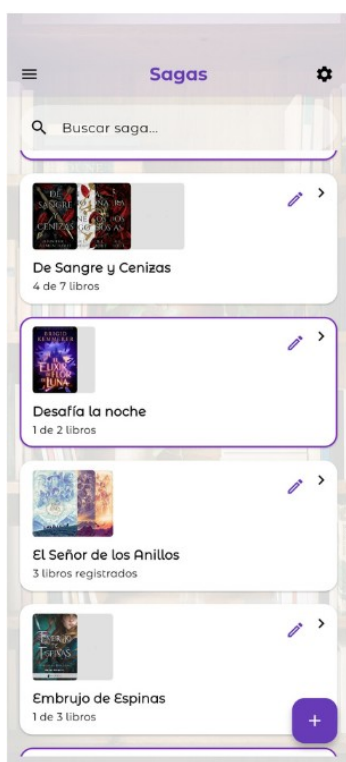
Confirmar la acción en la ventana emergente.

11.4.6 Marcar / desmarcar como favorito



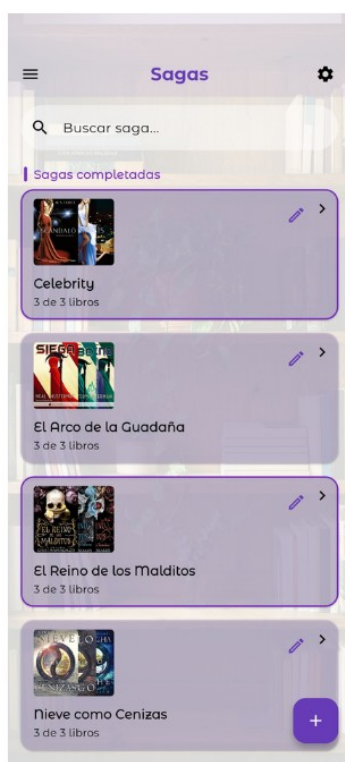
En la tarjeta de un libro, hacer clic en la **estrella** (parte superior, derecha) para marcarlo como favorito, la estrella se coloreará de amarillo. Para desmarcarlo hacer clic en la estrella de nuevo.

11.5 Sagas



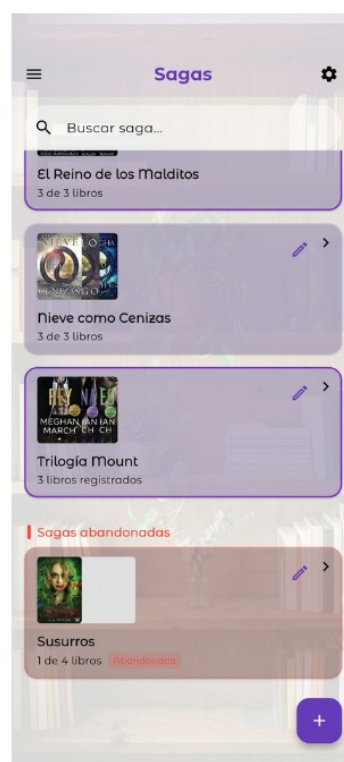
1

En lectura



2

Completadas

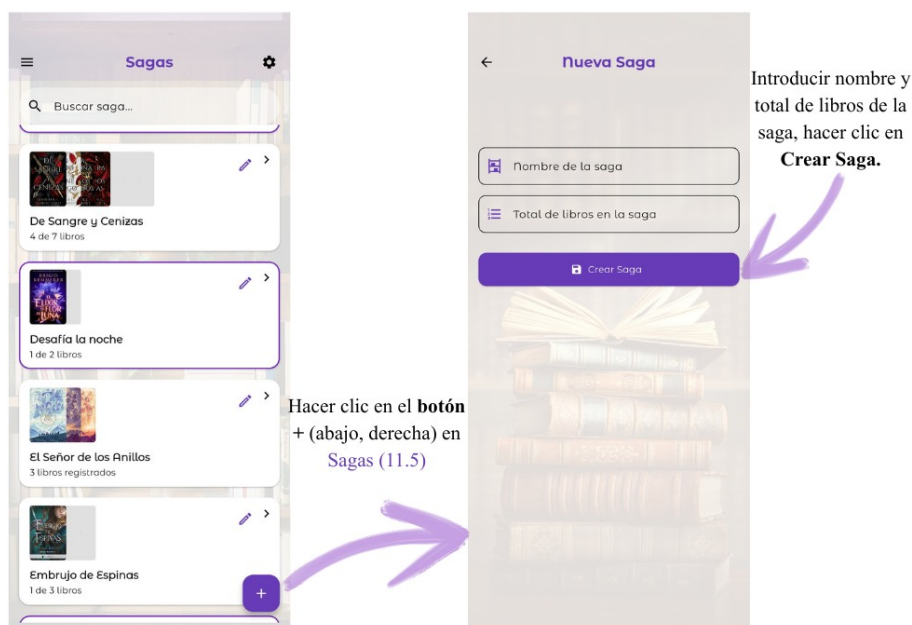


3

Abandonadas

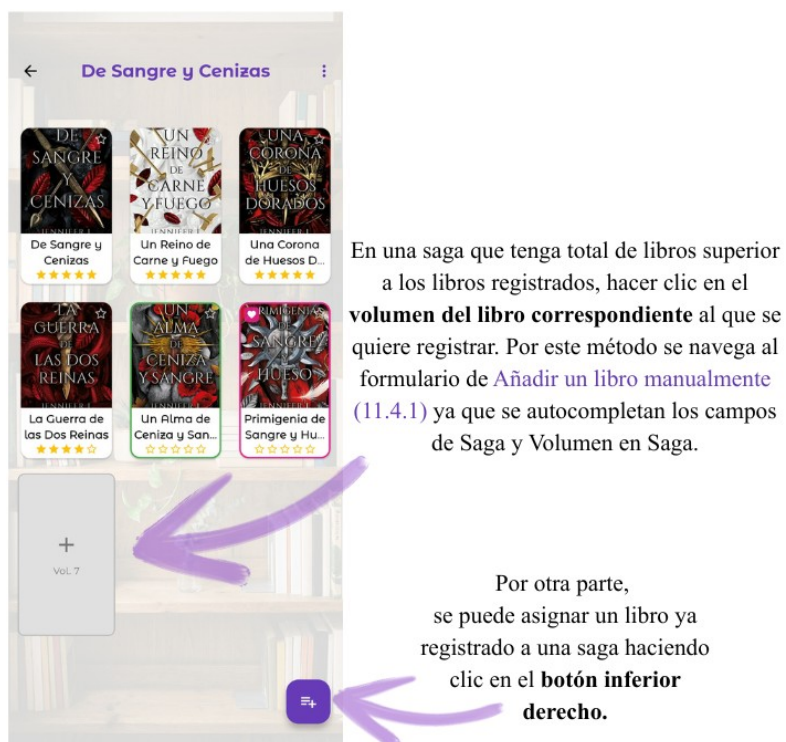
Si en una saga todos sus libros son digitales tendrá un borde violeta.

11.5.1 Crear una saga



Otra opción, al añadir un nuevo libro indicar una saga que aun no está registrada. Esta nueva saga se creará con un total de libros igual al número de libros registrados en ella, pudiendo editar este valor posteriormente.

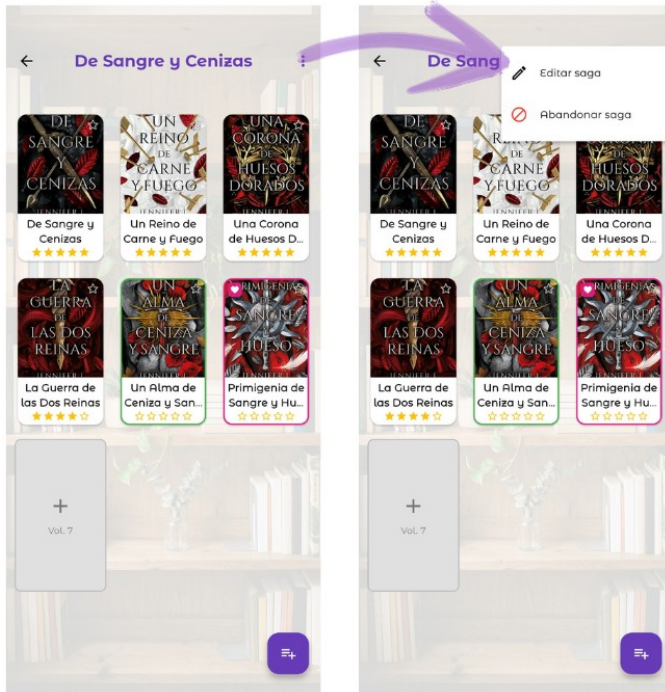
11.5.2 Asignar un libro a una saga



Además, si en el formulario de **Añadir un libro manualmente** (11.4.1) se le asigna una saga al libro este se añade automáticamente.

11.5.3 Eliminar o abandonar una saga

Hacer clic en los **tres puntos** de la barra superior (derecha), se puede acceder a [Editar una saga](#) (misma pantalla que Crear una saga – 11.5.1) y a abandonar una saga.

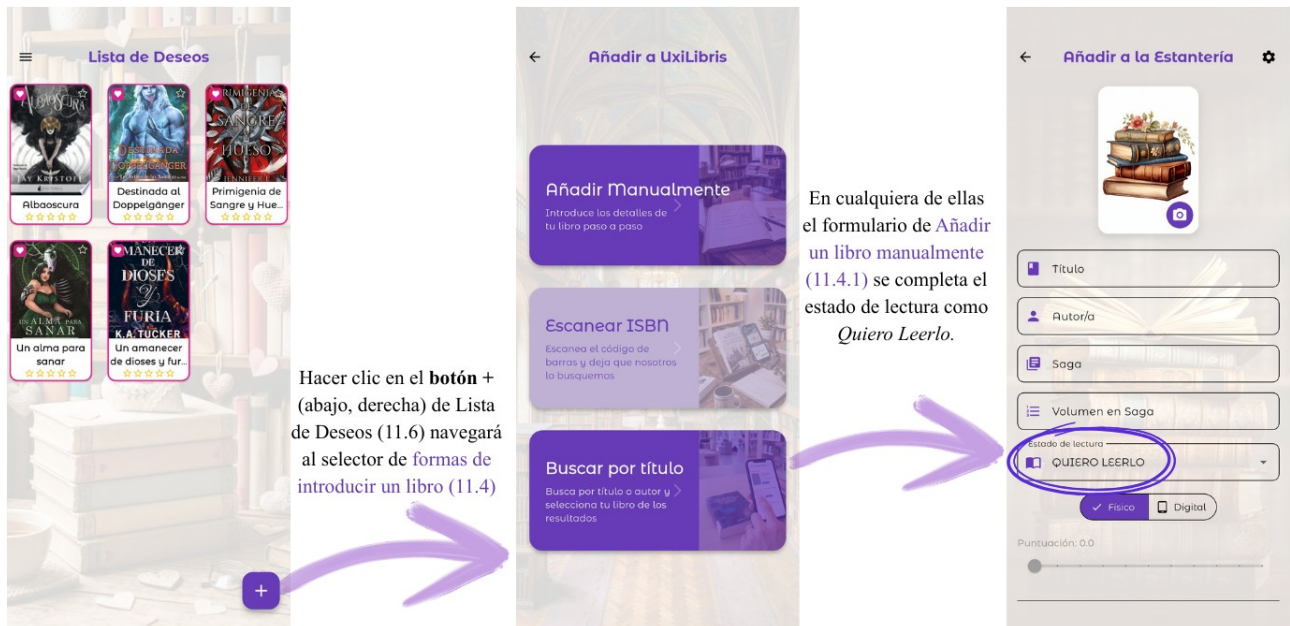


11.6 Lista de deseos

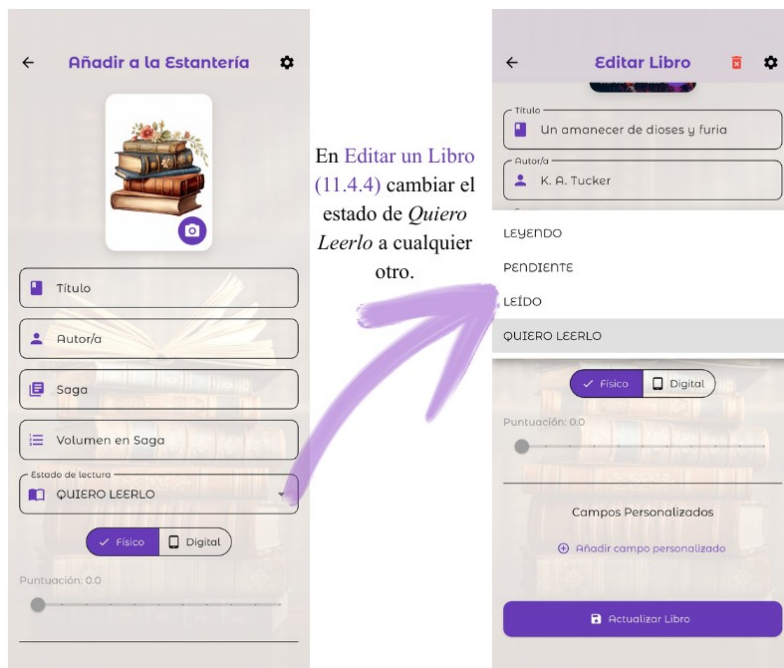


Existen libros que aunque deseas leerlos no dispones de un ejemplar, ni físico ni digital para este caso se encuentra la Lista de Deseos. Estos libros tienen un borde rosa y un icono de corazón y se podrán visualizar en la Lista de Deseos y en las Sagas que los tengan asignados.

11.6.1 Añadir un libro a la lista



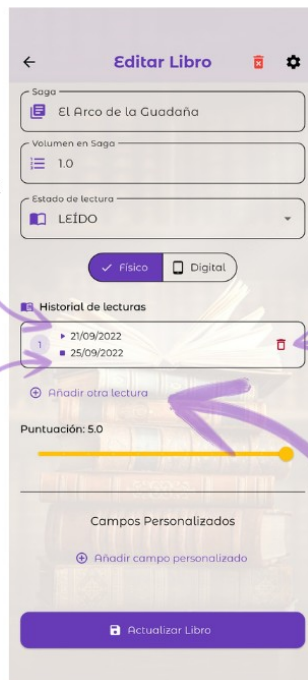
11.6.2 Mover un libro a la biblioteca



11.7 Lecturas

11.7.1 Registrar fechas de una lectura

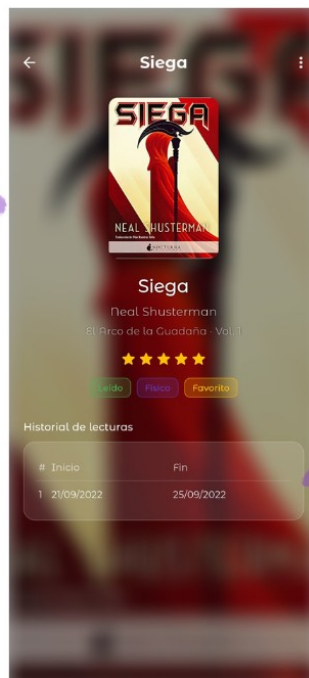
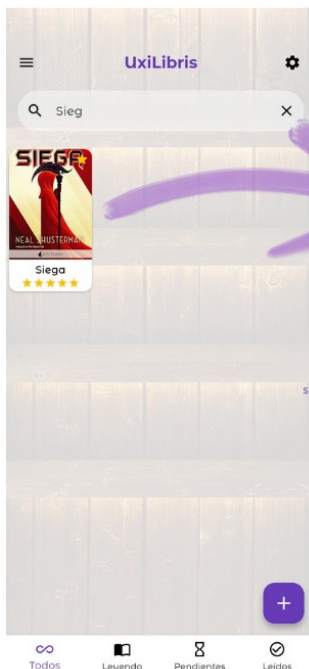
En **Añadir un libro manualmente (11.4.1)** o en **Editar un Libro (11.4.4)** hacer clic en la **primera fecha (icono play)** para registrar la fecha de inicio y en la **segunda (icono stop)** para la fecha de fin.



Para eliminar una lectura, hacer clic en el **icono de papelera (rojo)**

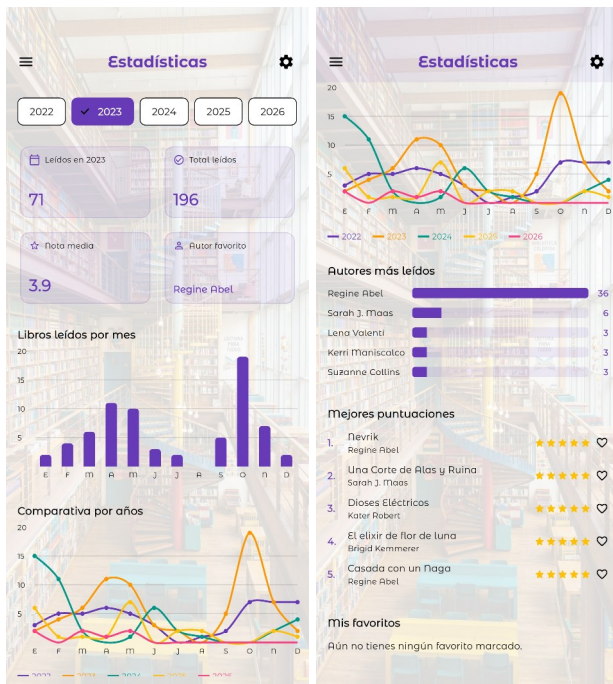
Si se quieren añadir más fechas de lectura hacer clic en **Añadir otra lectura**.

11.7.2 Consultar el historial de lecturas de un libro



En **Detalle Libro (11.3.4)** se visualizará una tabla con todas las fechas de lectura registradas.

11.8 Estadísticas



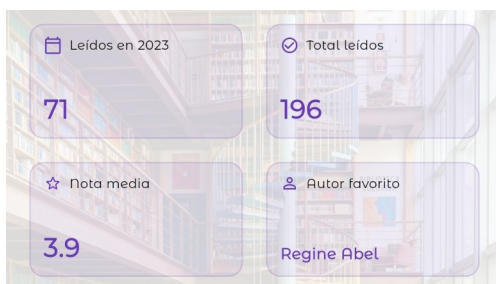
Pantalla con tiempo de espera debido al filtro principal (selector de año) debe realizar la petición a la base de datos y no tener los datos precargados.

11.8.1 Selector de año



Hacer clic en el año por el que se quieren filtrar las estadísticas.

11.8.2 Tarjetas resumen



Se visualiza el numero de lecturas hechas ese año, el número de lecturas registradas en total (suma de todos los años), la puntuación media de los libros leídos ese año y el autor favorito de ese año.

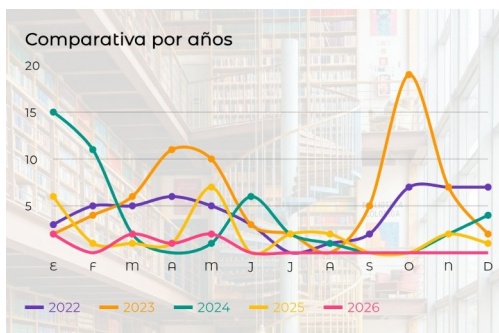
Nota: Se tienen en cuenta todas las lecturas no solo los libros, es decir, si lees un libro tres veces en un año se contabilizan las tres aunque sea el mismo libro.

11.8.3 Gráfica mensual



Gráfica que ilustra el numero de lecturas hechas cada mes del año.

11.8.4 Comparativa por años



Gráfica global que compara las gráficas mensuales de todos los años.

11.8.5 Top autores



Comparativa del numero de lecturas hechas por cada autor, se muestran los 5 más leídos ese año.

11.8.6 Mejores puntuaciones



Comparativa de los libros a los que se le asigna mejor puntuación en el año, se muestran 5 de ellos. El orden no es relevante ya que la puntuación es la más alta.

11.8.7 Mis favoritos

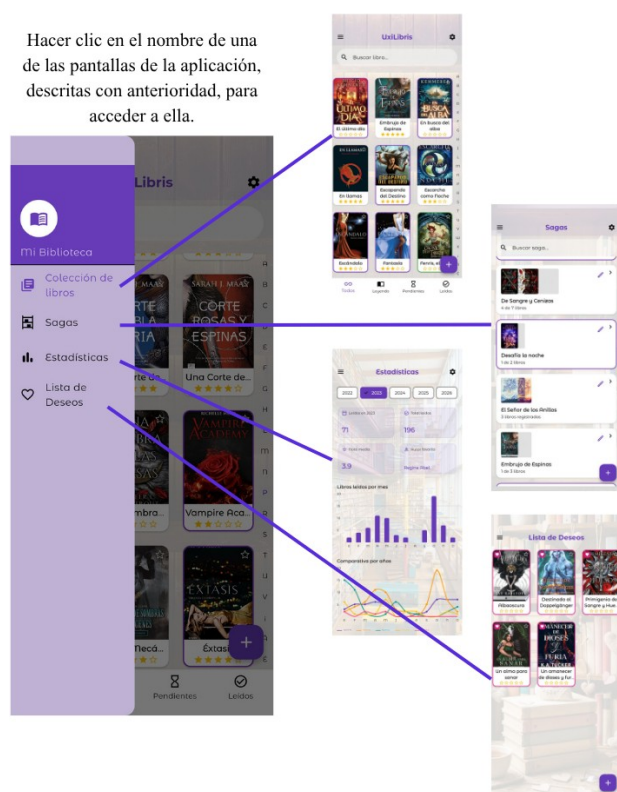


Libros marcados como **Favoritos** (11.4.6) y leídos ese año.

11.9 Navegación

11.9.1 Menú lateral (drawer)

Hacer clic en el nombre de una de las pantallas de la aplicación, descritas con anterioridad, para acceder a ella.



11.10 Preguntas frecuentes

¿Por qué no cargan los libros al abrir la app?

Si al iniciar sesión la pantalla de biblioteca permanece en blanco o muestra el indicador de carga durante más tiempo de lo habitual, lo más probable es que el servidor esté en reposo. El servidor de UxiLibris utiliza un plan gratuito en la nube que suspende el servicio automáticamente tras un período de inactividad. Al recibir la primera petición, tarda entre 30 y 60 segundos en volver a arrancar. Pasado ese tiempo, los libros cargarán con normalidad. No es necesario realizar ninguna acción: basta con esperar en la pantalla de carga.

¿Puedo usar la app sin cuenta?

No. UxiLibris requiere registro e inicio de sesión para acceder a cualquier funcionalidad. Esto garantiza que cada usuario disponga de su propia biblioteca privada y que los datos estén siempre asociados a una cuenta concreta, accesible desde cualquier dispositivo.

¿Se sincronizan los datos entre la versión web y la app Android?

Sí. Tanto la versión web como la app Android se conectan al mismo servidor y a la misma base de datos. Cualquier libro añadido, editado o eliminado desde un dispositivo aparecerá reflejado automáticamente al acceder desde el otro. La sincronización es inmediata: no existe un sistema de caché offline.

¿Qué ocurre con las portadas si desinstalo la app de Android?

Las portadas que el usuario asigna manualmente desde la galería del dispositivo se guardan en el almacenamiento interno de la aplicación. Cuando la app se desinstala, Android elimina toda esa carpeta junto con la aplicación. Al reinstalarla, los libros seguirán existiendo en la biblioteca (los datos están en el servidor), pero las portadas personalizadas no podrán recuperarse y los libros mostrarán la imagen por defecto.

Las portadas obtenidas automáticamente de Google Books no se ven afectadas, ya que se cargan directamente desde internet cada vez que se accede a la aplicación.

12. Anexo III – Otra documentación

12.1 Test y Calidad del Código

12.1.1 Introducción y estrategia de pruebas

El proyecto UxiLibris cuenta con un conjunto de pruebas automatizadas que cubre las dos capas principales de la aplicación: el backend en Java (Spring Boot) y el frontend en Dart (Flutter). El objetivo es verificar que la lógica de negocio es correcta de forma aislada, sin depender de un entorno de ejecución completo ni de bases de datos de producción.

La estrategia adoptada distingue dos niveles de prueba:

- **Pruebas unitarias:** verifican una sola clase o método aislado, sustituyendo las dependencias externas por dobles de prueba (mocks). Son rápidas y deterministas.
- **Pruebas de integración:** verifican que varios componentes colaboran correctamente. En este proyecto se aplican al acceso a datos del backend, usando una base de datos en memoria (H2) que se crea y destruye en cada ejecución.

Las pruebas de interfaz gráfica (widgets de Flutter y flujos end-to-end) requieren un dispositivo o emulador real y quedan fuera del alcance de la integración continua automática, dado que el plan gratuito de los runners de GitHub Actions no incluye emuladores.

Todas las pruebas se ejecutan automáticamente mediante un pipeline de GitHub Actions en cada push al repositorio.

12.1.2 Pruebas del backend (Java / Spring Boot)

A) Tecnologías utilizadas

Herramienta	Propósito
JUnit 5	Framework de pruebas unitarias para Java
Mockito	Creación de mocks y verificación de interacciones
Spring Boot Test (@DataJpaTest)	Contexto de integración ligero para la capa de datos
H2	Base de datos relacional en memoria para pruebas
AssertJ	API fluida para aserciones expresivas

Las dependencias de prueba se declaran con `scope test` en el *pom.xml* y no se incluyen en el artefacto (archivo final que se genera después de compilar y empaquetar el código) de producción.

La configuración de la base de datos en memoria se encuentra en:

backend_spring/uxilibris-backend/src/test/resources/application-test.properties

(.properties)

`spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1`

`spring.datasource.driver-class-name=org.h2.Driver`

```
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=create-drop
google.books.api.key=test-key
```

El flag `create-drop` hace que Hibernate cree el esquema completo al inicio de cada ejecución y lo elimine al terminar, garantizando el aislamiento total entre ejecuciones.

B) Pruebas unitarias de servicio – UsuarioServiceTest

Ubicación → `src/test/java/com/uxia/uxilibris/service/UsuarioServiceTest.java`

Clase base prueba → `UsuarioService`

Esta clase gestiona el registro y la autenticación de usuarios. Para aislarla de la base de datos se usa Mockito, que simula el comportamiento del repositorio sin realizar ninguna consulta real.

(java)

```
@ExtendWith(MockitoExtension.class)
class UsuarioServiceTest{
    @Mock
    private UsuarioRepository usuarioRepository;
    @InjectMocks
    private UsuarioService usuarioService;
}
```

La anotación `@ExtendWith(MockitoExtension.class)` activa el procesamiento de las anotaciones de Mockito. `@Mock` crea un doble del repositorio y `@InjectMocks` inyecta ese doble en el servicio que se va a probar.

Casos cubiertos

Método	Caso	Resultado esperado
guardar	Usuario nuevo con credenciales únicas	Devuelve <i>true</i> , la contraseña queda hasheada con BCrypt y se llama a <i>save</i>
guardar	Email ya registrado	Devuelve <i>false</i> , no se llama a <i>save</i>
guardar	Username ya registrado	Devuelve <i>false</i> , no se llama a <i>save</i>
verificar	Credenciales correctas	Devuelve <i>Optional</i> con <i>LoginResponse</i> con id, username y email
verificar	Contraseña incorrecta	Devuelve <i>Optional.empty()</i>
verificar	Usuario inexistente	Devuelve <i>Optional.empty()</i>

El tests de hashing verifica explícitamente que la contraseña almacenada no contiene el texto original y que `BCryptPasswordEncoder.matches` la valida:

```
assertThat(usuario.getPassword()).doesNotContain("plaintextpassword");
assertThat(encoder.matches("plaintextpassword", usuario.getPassword())).isTrue();
```

C) Pruebas de integración del repositorio – UsuarioRepositoryTest

Ubicación → `src/test/java/com/uxia/uxilibris/repository/UsuarioRepositoryTest.java`

Clase bajo prueba → `UsuarioRepository`

La anotación `@DataJpaTest` levanta un contexto de Spring limitado a la capa JPA (entidades, repositorios y H2), sin cargar controladores ni servicios. `TestEntityManager` permite persistir entidades directamente para preparar el estado de cada prueba.

(java)

`@DataJpaTest`

```
class UsuarioRepositoryTest{
    @Autowired
    private TestEntityManager em;
    @Autowired
    private UsuarioRepository usuarioRepository;
}
```

Casos cubiertos

Método del repositorio	Caso	Resultado esperado
<code>findByUsername</code>	Username existente	<i>Optional</i> presente con el email correcto
<code>findByUsername</code>	Username inexistente	<i>Optional.empty()</i>
<code>findByEmail</code>	Email existente	<i>Optional</i> presente con el username correcto
<code>findByEmail</code>	Email inexistente	<i>Optional.empty()</i>
<code>save</code>	Usuario nuevo	El objeto guardado tiene un <i>id</i> no nulo asignado por la base de datos

Cada prueba parte de un estado limpio porque `@DataJpaTest` envuelve cada método en una transacción que se revierte al terminar.

D) Prueba de contexto – UxilibrisBackendApplicationTests

Ubicación → `src/test/java/com/uxia/uxilibris/UxilibrisBackendApplicationTests.java`

Esta prueba verifica que el contexto de Spring Boot arranca sin errores cuando se usa el perfil `test`. Es la primera línea de defensa frente a errores de configuración o dependencias circulares.

(java)

`@SpringBootTest`

`@ActiveProfiles("test")`

```
class UxilibrisBackendApplicationTests{
    @Test
```



```
void contextLoads(){}
}
```

E) Ejecución local

(bash)

```
cd backend_spring/uxilibris-backend
```

Mac/Linux

```
./mvnw test -Dspring.profiles.active=test
```

Windows

```
mvnw.cmd test -Dspring.profiles.active=test
```

12.1.3 Pruebas del frontend (Dart / Flutter)

A) Tecnologías utilizadas

Herramienta	Propósito
flutter_test (SDK)	Framework de pruebas incluido en el SDK de Flutter
flutter test (CLI)	Ejecuta los test en la máquina virtual de Dart, sin dispositivo

Los tests de modelos son pruebas de Dart puro: no renderizan widgets, no acceden a la red ni al sistema de ficheros, y no requieren ningún emulador ni dispositivo físico.

B) Estructura de los tests

```
frontend_flutter/test/
```

```
├─ widget_test.dart      # Placeholder (test de UI requieren dispositivo)
└─ modelo/
```

```
    ├─ lectura_test.dart  # 12 casos – modelo Lectura
```

```
    ├─ libro_test.dart    # 12 casos – modelo Libro
```

```
    └─ librería_test.dart # 10 casos – modelo Librería
```

C) Tests del modelo Lectura

Lectura representa un periodo de lectura con fecha de inicio y fecha de fin opcionales.

Casos cubiertos

Lectura.fromJson

Caso	Descripción
Formato string ISO	“2024-01-15” → DateTime(2024, 1, 15)
Formato array	[2024, 3, 10] → DateTime(2024, 3 10)
fechaFin ausente	Se admite <i>null</i> (lectura en curso)
JSON vacío	Todas las fechas se quedan en <i>null</i> sin lanzar excepción

El backend Java puede devolver las fechas en dos formatos distintos según el campo, el parser los maneja a ambos.

Lectura.toJson

Caso	Descripción
Serialización de fechas	Se produce en formato “YYYY-MM-DD”
fechaFin nula	La clave <i>fechaFin</i> no aparece en el mapa resultado
id nulo	La clave <i>id</i> no aparece en el mapa resultado

Lectura.estaActiva / estaCompletada

Caso	Descripción
Sin fecha de fin	<i>estaActiva</i> == <i>true</i> , <i>estaCompletada</i> == <i>false</i>
Con fecha de fin	<i>estaCompletada</i> == <i>true</i> , <i>estaActiva</i> == <i>false</i>

Lectura.withFechaInicio / withFechaFin

Verifican que los métodos crean una copia con el nuevo valor sin modificar el objeto original (inmutabilidad).

D) Test del modelo Libro

Libro es la entidad central de la aplicación. Sus tests validan el ciclo completo de serialización/deserialización y la inmutabilidad al cambiar de estado.

Casos cubiertos**Libro.fromJson**

Caso	Descripción
Todos los campos presentes	Todos los atributos se parsean con el tipo correcto
Campos ausentes o nulos	Se aplican los valores por defecto: puntuacion=0.0, estado=pendiente, formato=físico, favorito=false, ruta de imagen por defecto
Estado desconocido	Se convierte a EstadoLibro.pendiente sin lanzar excepción
Estado en mayúsculas	“LEYENDO” se parsea correctamente a EstadoLibro.leyendo
Lecturas anidadas	La lista de Lectura se deserializa correctamente, incluyendo el flag <i>estaActiva</i>

Libro.toJson

Caso	Descripción
Estado	Se serializa en mayúsculas (“LEYENDO”) para compatibilidad con el backend
Formato	Se serializa en mayúsculas (“DIGITAL”)
id nulo	La clave <i>id</i> no aparece en el mapa
Round-trip	fromJson → toJson → fromJson preserva todos los valores

El test de round-trip garantiza que no se pierde información durante el ciclo de comunicación con el servidor:

```
final reconstructido = Libro.fromJson(original.toJson());
expect(reconstructido.titulo, original.titulo);
expect(reconstructido.estado, original.estado);
expect(reconstructido.puntuacion, original.puntuacion);
```

Libro.withEstado

Caso	Resultado esperado
Crea copia con nuevo estado	El objeto nuevo tiene el estado indicado
No modifica el original	El objeto fuente mantiene su estado tras la llamada
Preserva todos los campos	id, sagaNombre, puntuación, favorito se copian sin alteración

E) Test del modelo Libreria

Librería agrupa la colección de libros del usuario y expone los métodos de filtrado que usa la interfaz principal.

Cada prueba parte de un estado limpio gracias: `setUp(() => Libreria.todosLosLibros = []);`

Casos cubiertos

Libreria.filtrarPorEstado

Caso	Resultado esperado
Filtro “todos” con libros en deseo	Los libros con estado deseo quedan excluidos
Filtro por estado concreto (“leído”)	Solo devuelve los libros de ese estado
Filtro por estado sin coincidencias	Devuelve lista vacía
Filtro “todos” sobre colección vacía	Devuelve lista vacía

El filtro “todos” excluye siempre los libros en lista de deseos, ya que estos pertenecen a una pantalla dedicada y no deben aparecer en la colección principal.

Libreria.filtrarPorBusqueda

Caso	Resultado esperado
Consulta vacía	Devuelve todos los libros
Búsqueda por título	Case-insensitive; “señor” encuentra “El Señor de los Anillos”
Búsqueda por autor	“asimov” encuentra el libro cuyo autor es “Isaac Asimov”
Búsqueda por saga	“galactica” encuentra el libro de la saga “Saga Galáctica”
Sin coincidencias	Devuelve lista vacía
Coincidencia en título y saga simultánea	El libro aparece una sola vez (sin duplicados)

F) Ejecución local

```
cd frontend_flutter
```

```
flutter test
```

La salida muestra el resultado de cada caso individualmente. Para obtener más detalle en caso de fallo:

```
flutter test --reporter expanded
```

12.1.4 Integración continua – GitHub Actions

A) Configuración de workflow

Ubicación: .github/workflows/ci.yml

El pipeline se activa automáticamente en cada push o pull_request sobre cualquier rama. Ejecuta dos jobs en paralelo:

CI

- └─ Backend – Test (ubuntu-latest, JDK 21, Maven)
- └─ Flutter – Analyze & Tests (ubuntu-latest, Flutter stable)

B) Job: Backend – Tests

```
uses: actions/setup-java@v4
```

```
with:
```

```
  java-version: '21'
```

```
  distribution: temurin
```

```
  cache: maven
```

```
run: mvn test -Dspring.profiles.active=test
```

1. Descarga el JDK 21 (distribución Temurin / Eclipse Adoptium).
2. Activa la caché de dependencias Maven para acelerar ejecuciones posteriores.
3. Ejecuta todos los tests con el perfil test, que carga application-test.properties y usa H2 en lugar de MySQL o PostgreSQL.

C) Job: Flutter – Analyze & Tests

```
uses: subosito/flutter-action@v2
```

```
with:
```

```
  channel: stable
```

```
  cache: true
```

```
run: flutter pub get
```

```
run: flutter analyze --no-fatal-infos
```

```
run: flutter test
```

1. Instala la última versión estable de Flutter (con caché activado).
2. Descarga las dependencias del proyecto (pubspec.yaml).
3. Analiza el código estáticamente con las reglas de flutter_lints. El flag --no-fatal-infos permite que avisos de nivel informativo no interrumpan el pipeline.
4. Ejecuta todos los tests unitarios de la carpeta test/.

D) Resultado Visible

El estado del pipeline se muestra como badge en el README.md del repositorio:

`[![CI](https://github.com/UxiaRD/uxilibris_project/actions/workflows/ci.yml/badge.svg)](...)`

- Verde (*passing*): todos los jobs han terminado correctamente.
- Rojo (*failing*): al menos un job ha fallado; el log del run concreto indica cuál y por qué.

12.1.5 Cobertura y limitaciones

La cobertura actual se centra en la lógica central donde los errores tienen mayor impacto. Los controladores REST se pueden validar manualmente con las opciones de instalación descritas en el README, y los flujos de interfaz se verifican en dispositivo real durante el desarrollo.